Prototipo de sistema de traducción de Lengua de Señas Argentina con procesamiento en tiempo pseudo-real

Maite Nigro Francisco Veron

Director: Esp. Ing. Gonzalo Avalos Ribas

Co-director: Ing. Bruno Constanzo

Universidad Nacional de Mar del Plata
Facultad de Ingeniería
Departamento de Ingeniería en Informática

Octubre 2026

Agradecimientos

De Maite: Agradezco sinceramente a Avalos Ribas, Gonzalo, por su guía y acompañamiento constante en este proyecto. A la Facultad de Ingeniería, por la formación brindada durante estos años de estudio. Y especialmente a mi familia y amigos, por su aliento e incondicionalidad en cada etapa.

De Francisco: Para la facultad de Ingeniería de Mar del Plata, y sus docentes que, con todo su cariño y pasión, nos enseñaron a lo largo de estos años. Para Gonzalo Avalos Ribas y Bruno Constanzo, director y co-director del proyecto final, que nos acompañaron en esta instancia final de carrera. Para mi familia, amigos, personas que pasaron por mi camino, sin todos ellos no podría haber llegado a donde estoy, ni ser quien soy. Agradecimiento al cielo para Carmen Beatríz Loyola
A Miriam Rolls, por su apoyo y ayuda en este proyecto.

Índice general

1. Introducción **1**
- 1.1. Contexto . . . 1
- 1.2. Motivación . . . 1
- 1.3. Problemática . . . 1
- 1.4. Alcance y limitaciones . . . 2
  - 1.4.1. Delimitación del vocabulario y asesoramiento lingüístico . . . 2
  - 1.4.2. Límites operativos y comportamiento ante entradas no contempladas 3
  - 1.4.3. Infraestructura de despliegue . . . 4

2. Objetivos **5**
- 2.1. Objetivo general . . . 5
- 2.2. Objetivos específicos . . . 5

3. Marco teórico, estado del arte y metodología **7**
- 3.1. Marco teórico . . . 7
  - 3.1.1. Componentes articulatorios y estructura fonológica de la LSA . . . 7
  - 3.1.2. Estructura lingüística de la LSA y orden manual del prototipo . . . 7
  - 3.1.3. Sistemas de tiempo real y tiempo pseudo-real . . . 8
  - 3.1.4. Visión computacional y reconocimiento basado en esqueletos . . . 8
  - 3.1.5. Modelado de secuencias temporales y atención . . . 9
  - 3.1.6. Modelos de lenguaje pequeños, adaptación *LoRA* y cuantización . . 9
  - 3.1.7. Estándares de captura web y arquitecturas de extensión web . . . 11
- 3.2. Estado del arte . . . 12
- 3.3. Metodología de desarrollo . . . 13
  - 3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial (IA) 14
- 3.4. Herramientas y tecnologías empleadas . . . 16

4. Análisis de requerimientos **17**
- 4.1. Requerimientos funcionales . . . 17
- 4.2. Requerimientos no funcionales . . . 18
- 4.3. Restricciones del sistema . . . 19
  - 4.3.1. Especificación del entorno de ejecución cliente . . . 19
- 4.4. Casos de uso . . . 20
  - 4.4.1. Casos de uso esenciales en formato breve . . . 20
  - 4.4.2. Diagrama de casos de uso . . . 21
  - 4.4.3. Casos de uso expandidos . . . 21

5. Diseño del Sistema **27**
5.1. Arquitectura del sistema . . . 27
5.1.1. Justificación de la partición y modelo de privacidad . . . 27
5.1.2. Variación artificial del conjunto de señas . . . 28
5.1.3. Flujo integral de datos entre componentes . . . 28
5.2. Diseño de contratos de API REST . . . 29
5.2.1. API local del motor ILSA . . . 30
5.2.2. API del servicio semántico remoto . . . 30
5.2.3. API del servicio de archivos del avatar . . . 30
5.3. Diseño de la representación gestual inversa . . . 32
5.3.1. Procesamiento geométrico de coordenadas y segmentación temporal 32
5.3.2. Preparación, acondicionamiento y ensamblado cinemático . . . 33
5.3.3. Estrategia de contingencia léxica . . . 34
5.4. Diseño de la extensión de videollamada . . . 34
5.5. Patrones de diseño de *software* aplicados . . . 34
5.6. Diseño conceptual de la arquitectura escalable de producción en la nube . . 35
5.6.1. Ingreso . . . 35
5.6.2. Traductor . . . 36
5.6.3. Archivos de seña . . . 36
5.6.4. Monitoreo . . . 36
5.6.5. Qué queda fuera de las pruebas . . . 36

6. Implementación y Desarrollo **37**
6.1. Estructura del código . . . 37
6.1.1. Organización del Repositorio . . . 37
6.1.2. Descripción de Componentes y Responsabilidades . . . 38
6.1.3. Convenciones de Nomenclatura y Estándares . . . 38
6.1.4. Archivos de Configuración y Entorno de Pruebas . . . 39
6.2. Implementación del subsistema de reconocimiento gestual . . . 39
6.2.1. Preprocesamiento de datos y pipeline de extracción . . . 39
6.2.2. Arquitectura neuronal TinySkeletonClassifier . . . 40
6.3. Implementación del subsistema de traducción semántico bidireccional . . . 42
6.3.1. Selección y especificación de la suite de modelos de lenguaje . . . 42
6.3.2. Régimen de entrenamiento y ajuste LoRA . . . 43
6.3.3. Canal directo (LSA $\rightarrow$ Español) . . . 45
6.3.4. Canal inverso (Español $\rightarrow$ LSA) . . . 46
6.3.5. Cuantización GGUF Q4_K_M y estimación de memoria . . . 47
6.4. Integración en Google Meet y arquitectura de la extensión . . . 47
6.4.1. Secuencia completa de inferencia y comunicación multidireccional . 47
6.4.2. Motor local ILSA (FastAPI) . . . 50
6.5. Implementación del avatar . . . 51
6.5.1. Vía descartada: retarget hacia un avatar VRM . . . 51
6.5.2. Decisiones técnicas de la reproducción . . . 51
6.6. Instancia de validación del prototipo mediante microservicios en la nube . . 52
6.7. Contratos HTTP de las APIs . . . 53
6.7.1. Motor local ILSA . . . 54
6.7.2. Servicio semántico remoto . . . 60
6.7.3. Servicio de poses del avatar . . . 61

7. Pruebas y validación **63**
7.1. Plan de pruebas . . . 63
7.1.1. Alcance y tipos de pruebas . . . 63
7.1.2. Entornos de evaluación . . . 64
7.1.3. Equipo portátil de las pruebas de cliente . . . 64
7.1.4. Criterios de aceptación y trazabilidad . . . 64
7.2. Resultados de la verificación . . . 65
7.2.1. Evaluación del clasificador gestual (TinySkeletonClassifier) . . . 65
7.2.2. Resultados de los módulos de traducción semántica . . . 69
7.2.3. Pruebas de la lógica de control del motor local . . . 70
7.2.4. Tratamiento de fallas de integración en Google Meet . . . 71
7.3. Métricas de calidad . . . 71
7.3.1. Pruebas de latencia y end-to-end en llamada real . . . 72
7.3.2. Evaluación del uso de recursos de hardware en el dispositivo cliente 74
7.3.3. Consumo de memoria y huella del sistema en servidor . . . 74
7.3.4. Comparativa de rendimiento según arquitectura de hardware . . . 76
7.3.5. Señas aceptadas y rechazadas en llamada . . . 78

8. Evaluación **79**
8.1. Cumplimiento de objetivos . . . 79
8.1.1. Evaluación del objetivo general . . . 79
8.1.2. Evaluación de los objetivos específicos . . . 79
8.2. Limitaciones . . . 83
8.3. Riesgos detectados y estrategias de mitigación . . . 84

9. Conclusiones y trabajos futuros **86**
9.1. Conclusiones . . . 86
9.1.1. Síntesis del cumplimiento de objetivos . . . 86
9.1.2. Aportes técnicos e ingenieriles . . . 87
9.1.3. Impacto social y marco regulatorio . . . 87
9.2. Trabajos futuros . . . 88
9.2.1. Inclusión de la comunidad sorda y validación participativa . . . 88
9.2.2. Ampliación del dataset y captura multicámara . . . 88
9.2.3. Incorporación de rasgos no manuales y reconocimiento continuo . . 88
9.2.4. Evolución hacia un avatar tridimensional (3D) . . . 88
9.2.5. Despliegue del clúster elástico multinodo en la nube . . . 89
9.2.6. Generalización a nuevas plataformas y ejecución local de SLM . . . 89

Bibliografía **90**

A. Vocabulario del clasificador **94**

B. Ramas de entrenamiento y de servicios **96**
B.1. *scratch-mediapipe-v3*: Repositorio de entrenamiento del clasificador . . . 96
B.2. *semantic-cloud*: Servicio semántico en la nube . . . 97
B.3. *semantic-v2*: Entrenamiento del canal directo . . . 97
B.4. *semantic-reverse*: Entrenamiento del canal inverso . . . 98
B.5. *avatar-cloud*: Servicio de poses del avatar . . . 99

C. Arquitectura escalable de producción en la nube **101**

Resumen

En la actualidad, la interacción virtual entre personas sordas y oyentes se limita casi exclusivamente a la comunicación escrita o a la transcripción unilateral de voz a texto, lo que genera una marcada asimetría en la comunicación expresiva durante videollamadas. Este Trabajo Final de Grado desarrolla un prototipo de traducción bidireccional entre la Lengua de Señas Argentina (LSA) y el español oral estructurado bajo una arquitectura híbrida de computación en el borde y servicios en la nube con procesamiento en tiempo pseudo-real.

El sistema articula dos canales comunicacionales complementarios. En el sentido señante-oyente, la computadora del usuario captura localmente el flujo de video desde la cámara web, extrae coordenadas articulares tridimensionales mediante visión computacional y clasifica un vocabulario de 100 señas mediante una red neuronal ligera (Conv1D + Tiny Transformer con *attention pooling*). Con el fin de evitar la saturación de memoria y cómputo que provocaría ejecutar modelos de lenguaje generativos en dispositivos convencionales, las glosas clasificadas son enviadas hacia un backend semántico en la nube que transforma la secuencia en oraciones en español rioplatense, las cuales retornan al cliente para su proyección en subtítulos o síntesis de voz. En sentido inverso (oyente-señante), el sistema procesa la voz y/o texto del interlocutor oyente mediante un módulo semántico inverso en la nube que estructura las glosas, enviándolas a un avatar en el cliente que ejecuta las animaciones correspondientes o recurre al deletreo dactilológico para términos fuera del léxico. El módulo clasificador gestual local alcanza un 99,0 % de exactitud en validación fuera de línea y un 99,6 % de acierto en cámara real (contabiliza como respuestas válidas los pares homógrafos (`O` $\equiv$ `0` y `V` $\equiv$ `viernes`). Asimismo, se explicita que, al exigir coincidencia exacta de etiquetas, la métrica es del 97,8 %.). Por su parte, el modelo elegido para el servicio, Qwen2.5 1,5B, adaptado mediante LoRA y cuantizado a GGUF `Q4_K_M`, alcanza un acierto exacto del 94,96 % en el canal directo y del 91,18 % en el inverso. La generación local de ese modelo, en la estación de desarrollo y con las capas en la GPU, es de 256,30 ms y de 290,82 ms. En la sesión integrada, el tiempo medio de inferencia remota es de 1444,4 ms en el canal directo y de 1368,2 ms en el inverso. Como mecanismo de integración para videollamadas, el sistema se acopla a Google Meet mediante una extensión de navegador articulada con el motor local Interprete de Lengua de Señas Argentina (ILSA). Esta arquitectura resguarda la privacidad del usuario al evitar la transmisión del video crudo hacia los servicios remotos de traducción y poses; el flujo audiovisual se deriva de forma exclusiva hacia la llamada de Google Meet con las capas gráficas superpuestas (subtítulos y avatar). Finalmente, se deja documentada la propuesta de diseño de una arquitectura en la nube escalable para soportar la demanda concurrente en entornos de producción.

Palabras clave Lengua de Señas Argentina, Small Language Model, Transformer, glosas, MediaPipe, extensión de navegador.

Capítulo 1

Introducción

1.1. Contexto

Tradicionalmente, la sociedad abordó a la comunidad sorda desde una perspectiva clínica centrada en la rehabilitación y la corrección auditiva. Ese enfoque omite que la comunidad sorda se identifica, ante todo, como una minoría lingüística con cultura y gramática propias, independientes del español oral. En Argentina, la Lengua de Señas Argentina cuenta con reconocimiento legal formal a través de la Ley 27.710 [1], la cual garantiza su preservación y el derecho de las personas sordas a comunicarse en su lengua natural en todos los ámbitos sociales.

En el contexto digital contemporáneo, las plataformas de videoconferencia se han convertido en herramientas centrales para el ámbito laboral, educativo y de la vida cotidiana. Sin embargo, los *software* de uso masivo presentan una escasa integración de herramientas automatizadas que admitan la LSA como un flujo comunicacional bidireccional en tiempo pseudo-real. Tal como se analiza en el estudio comparativo de la Sección 3.2, las soluciones tecnológicas revisadas para la LSA se limitan predominantemente a diccionarios estáticos, repositorios pedagógicos de video o servicios de interpretación humana síncrona, por lo que carecen de capacidad de interpretación integrada de forma directa en una videollamada.

1.2. Motivación

Este proyecto nace de la aspiración de reducir las barreras comunicacionales de las personas señantes en interacciones cotidianas y de asistencia mediante herramientas informáticas. Resulta indispensable avanzar hacia esquemas de integración lingüística en los que la tecnología actúe como un puente transparente en ambos sentidos de la conversación: expresivo (señante $\rightarrow$ oyente) y receptivo (oyente $\rightarrow$ señante).

1.3. Problemática

La problemática central radica en la asimetría comunicacional presente en las plataformas de videoconferencia. Si bien las herramientas de transcripción automática de voz a texto han alcanzado un alto grado de adopción, resultan insuficientes como solución de accesibilidad integral: para diversas personas sordas, el texto escrito opera como una segunda lengua con reglas gramaticales ajenas [2], e incluso genera barreras de acceso para quienes no se encuentran alfabetizados en la lengua oral.

En el extremo expresivo, la comunicación mediante LSA carece de soporte nativo en el *software* de videollamadas comercial. La falta de módulos capaces de capturar e interpretar la LSA hacia voz o texto audible fuerza la dependencia de interpretación humana o restringe la interacción al chat escrito.

A esta brecha se añade la especificidad lingüística local: los modelos de visión y lenguaje desarrollados internacionalmente se encuentran enfocados mayoritariamente en la Lengua de Señas Americana (ASL). Las técnicas y arquitecturas neuronales resultan reutilizables, pero los modelos preentrenados y conjuntos de datos de ASL no se extrapolan directamente a la LSA debido a diferencias léxicas, dactilológicas y sintácticas, por lo que es necesaria la construcción de un *corpus* propio y de modelos adaptados al contexto argentino.

1.4. Alcance y limitaciones

El sistema desarrollado comprende un prototipo funcional de traducción bidireccional en tiempo pseudo-real concebido como una extensión para navegadores basados en la tecnología Chromium (como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi), articulada con un motor de ejecución en segundo plano denominado ILSA.

Para abordar la interpretación de la LSA se introduce el concepto de **glosa**: representación escrita en letras mayúsculas asignada a una seña para identificarla, y analizar su gramática sin constituir una traducción literal término a término.

El sistema opera a través de dos canales comunicacionales integrados:

- **Canal directo (Señante → Oyente):** En el cliente señante, el sistema toma señas aisladas de la cámara, extrae coordenadas articulares y clasifica cada glosa. Una seña empieza cuando las manos entran en el cuadro y termina cuando salen. La frase se cierra tras 4 segundos sin manos en el cuadro. Una sola glosa léxica, un deletreo de dos o más letras, o una seguidilla de dígitos se resuelven en ILSA y no consultan la nube. Hay excepciones cuando la forma coincide con otra clase: un deletreo que contiene I o T, y las glosas `OJO`, `VIERNES` o `AÑOS` cuando van solas. Esos casos van al traductor, al igual que el resto de las frases. El servicio semántico recibe ese texto de glosas y devuelve una oración en español rioplatense. Se brindan las opciones de Subtítulo, Audio y Ambos. Con Subtítulo, la oración queda en el video. Con Audio o Ambos, ILSA sintetiza un WAV con la voz de Windows y el *script* de la página lo mezcla en la pista de audio que Meet envía. Quien seña la oye por los parlantes y el resto de la llamada, por esa pista.

- **Canal inverso (Oyente → Señante):** La voz del oyente se transcribe en su computadora, o el sistema toma el texto del chat. La oración va a la nube, que la reordena según el orden manual de este prototipo. Se dibuja el clip de pose de cada glosa sobre un esqueleto cuando el archivo existe. Si la glosa no tiene archivo, ILSA pide cada letra y la dibuja con el alfabeto manual. Ese deletreo no asegura que el interlocutor comprenda el significado de la palabra.

1.4.1. Delimitación del vocabulario y asesoramiento lingüístico

El alcance funcional del clasificador gestual está delimitado a un vocabulario de 100 señas de LSA (Apéndice A). El recorte se orientó a la interacción cotidiana y a un primer contacto o auxilio (verbos de asistencia, identificación, parentesco y datos temporales). La delimitación léxica se acordó con Miriam Rolls, profesora e intérprete de LSA, en

una reunión presencial. Se le envió el listado de 100 glosas; lo consideró adecuado para probar el prototipo. Para aprender y filmar cada seña se usó el Señario de la Confederación Argentina de Sordos [3] y la aplicación *LSA en Familia* [4]. Rolls facilitó el contacto con Maximiliano Godoy, presidente de la Asociación de Sordos de Mar del Plata (ASMAR) [5]. En una reunión presencial se presentó el proyecto y se evaluó ampliar el *corpus* con señantes sordos. Esa grabación no se concretó en el calendario de este trabajo. El dataset de este prototipo lo filmaron los autores, que no son usuarios de LSA. La ejecución se revisó con una toma por clase, en una carpeta compartida. Godoy aceptó el conjunto e indicó desvíos menores, esperables en señantes no nativos. Esos desvíos no se tomaron como un bloqueo de este recorte: el *corpus* admite recaptura en una etapa posterior. Esa revisión cubre la conveniencia del léxico y una muestra de ejecución por seña. No es una evaluación de usabilidad ni un ensayo con usuarios sordos en una emergencia.

1.4.2. Límites operativos y comportamiento ante entradas no contempladas

El prototipo presenta las siguientes delimitaciones operativas:

- **Delimitación de señas y de frases:** Una seña empieza cuando las manos entran en el cuadro y termina cuando salen. La frase termina cuando pasan 4 segundos sin manos en el cuadro.
- **Mano hábil:** ILSA muestra dos botones, diestro y zurdo, con diestro seleccionado por *default*. Con zurdo, el vector se espeja en horizontal y se cruzan las dos manos, porque el modelo se entrenó con mano derecha. Ese espejo es una normalización de entrada. En este trabajo la evaluación en cámara se hizo con mano diestra; no se informa el mismo protocolo con señantes zurdos.
- **Rasgos no manuales y marcas manuales:** El clasificador excluye la malla facial y no trata la boca, las cejas ni la cabeza como rasgo gramatical. Una pregunta, una negación o un tiempo solo entran al enunciado si se producen con una seña o una marca manual. El prototipo no recupera un significado que exista únicamente en la información que descarta.
- **Predicción, glosa incorporada e indicador del HUD:** Una predicción es la clase que devuelve el clasificador, con su confianza. Una glosa incorporada es la que quedó en la frase. Ver el nombre en el HUD no significa que vaya a transmitirse. El color informa el resultado:
  - **Cian, «En el mensaje: HOLA»:** la confianza es de al menos 0,75 y la glosa quedó en la frase.
  - **Coral, «Poca confianza: HOLA»:** el clasificador reconoció la seña, pero la confianza quedó bajo 0,75 y no se agregó.
  - **Gris, «No se agregó: HOLA»:** la confianza alcanzó 0,75 y un filtro de repetición la dejó fuera. Ese filtro rechaza la repetición inmediata de una glosa léxica, admite la repetición en los dígitos y admite hasta dos letras iguales seguidas.

Solo el cian entra al enunciado. Una seña ausente del vocabulario puede parecerse a una clase conocida y salir con confianza de al menos 0,75. En ese caso el HUD la muestra en cian y la glosa incorrecta entra a la frase.

- **Reproducción del avatar:** El dibujo está garantizado para el clip cuyo archivo existe, hasta 24 archivos por enunciado. Si una glosa no tiene archivo, se deletrea letra por letra, y ese deletreo no asegura la comprensión del significado. Lo que supera los 24 archivos queda en texto y no se anima. Si el servidor de movimientos no responde, las glosas quedan en texto y el clip queda vacío. El avatar mantiene, además, una cola de hasta 20 animaciones. Una oración nueva espera su turno y no interrumpe la animación en curso.
- **Resiliencia ante fallas de red:** Si el traductor no responde, ILSA conserva las glosas ya aceptadas y las muestra en texto, sin rearmar la oración.
- **Encender y Reiniciar:** Al iniciar la aplicación, al Encender un modo o al Reiniciar, ILSA vuelve a la pantalla de carga y exige que respondan el traductor semántico y el servidor de poses. Si responden, se abre el panel principal. Si alguno falla, queda el botón Reintentar y el panel principal no se abre.

1.4.3. Infraestructura de despliegue

En la red, el alcance incluye el diseño de una arquitectura elástica, con contenedores y balanceo de carga, para varias videollamadas a la vez. Las pruebas no despliegan ese esquema. La evaluación usa una instancia de Cloud Run por servicio, en CPU.

Capítulo 2

Objetivos

2.1. Objetivo general

Desarrollar un prototipo de sistema de traducción bidireccional entre la LSA y el español rioplatense con procesamiento en tiempo pseudo-real, diseñado para operar sobre plataformas de videoconferencia mediante una arquitectura híbrida cliente-nube, con el fin de mitigar la asimetría en la comunicación expresiva y receptiva, lo que ayudaría a fomentar la autonomía de las personas sordas en la interacción con interlocutores oyentes no señantes.

2.2. Objetivos específicos

- Implementar un módulo local de visión computacional y clasificación gestual basado en la extracción de puntos de referencia corporales para el reconocimiento de un vocabulario de 100 señas de LSA.
- Diseñar e integrar el servicio semántico del canal directo (LSA → Español) para transformar secuencias de glosas aisladas en una oración en español rioplatense.
- Diseñar e integrar el servicio semántico del canal inverso (Español → LSA) para reestructurar oraciones del español en la secuencia manual de glosas que fija este prototipo, dentro del orden de la Sección 3.1.2.
- Implementar un módulo de visualización gestual inversa mediante un avatar capaz de animar el clip de cada glosa con archivo y de deletrear los términos ausentes, con el límite de garantía de la Sección 1.4.2.
- Diseñar una arquitectura escalable en la nube para el aprovisionamiento concurrente de los servicios, lo que desacopla el procesamiento del hardware cliente.
- Resguardar la privacidad de los datos biométricos de los usuarios al ejecutar la extracción de características y la clasificación de forma local en el dispositivo cliente, y, por consiguiente, limitar la transmisión hacia la nube a cargas puramente textuales.
- Desarrollar la arquitectura de *software* de una extensión para navegadores basados en la tecnología Chromium, articulada con un motor de ejecución en segundo plano (ILSA).

- Evaluar el prototipo en cuatro ejes:
  - **Clasificador.** Reconocer, seña por seña y con un *script* local, el vocabulario de 100 clases según la Sección 7.2.1, con una meta de más del 75 % de acierto Top-1 en luz doméstica.
  - **Canales semánticos.** Medir la fidelidad de ambos sentidos con las métricas de la Sección 3.3. Esa medición no incluye una revisión humana de persona, tiempo verbal, negación y pregunta.
  - **Latencia y hardware.** Medir los tramos de la Sección 3.1.3 y el consumo de CPU y memoria. En el canal directo, medir la espera desde que las manos salen del cuadro hasta el subtítulo. En el canal inverso, medir la transcripción, la cola, la inferencia, el transporte y la entrega hasta la animación.
  - **Avatar.** Evaluar la comprensión del dibujo de forma aparte de la fidelidad automática.

Capítulo 3

Marco teórico, estado del arte y metodología

3.1. Marco teórico

3.1.1. Componentes articulatorios y estructura fonológica de la LSA

La articulación de la LSA se sostiene sobre la interacción simultánea de dos grandes conjuntos de parámetros lingüísticos:

- **Rasgos manuales:** Comprenden la configuración de la mano, la ubicación, la orientación y la dirección del movimiento [6, pág. 43].
- **Rasgos no manuales:** Comprenden la cara, la cabeza, los ojos y el cuerpo [6, pág. 43]. Esos rasgos marcan la interrogación, la negación y la topicalización [6, págs. 12, 30].

3.1.2. Estructura lingüística de la LSA y orden manual del prototipo

La LSA y el español rioplatense no comparten la misma gramática. Las afirmaciones generales sobre la lengua y la regla de trabajo de este prototipo van separadas.

Sobre la LSA, la bibliografía de referencia sostiene tres puntos. La organización *Topic-Comment* presenta un tema y después lo que se dice de ese tema [6, pág. 30]. Esa organización no equivale, por sí sola, a un orden fijo de todas las palabras. La lengua no usa los artículos, las preposiciones gramaticales abstractas ni los verbos copulativos del español [7].

Este prototipo no implementa esa gramática completa. El clasificador no toma la boca, las cejas ni la cabeza como rasgos. Una pregunta, una negación o un tiempo solo se recuperan si la persona los produce con una seña manual. La ecuación siguiente es la regla simplificada con la que el prototipo ordena las glosas, no una descripción de toda la LSA:

$$\begin{aligned} &\text{Tiempo} \rightarrow \text{Lugar} \rightarrow \text{Sujeto} \\ &\qquad \rightarrow \text{Sustantivo} \rightarrow \text{Posesivo} \rightarrow \text{Numeral} \\ &\qquad \rightarrow \text{Verbo} \rightarrow \text{Negación o pregunta} \end{aligned} \tag{3.1}$$

Por ejemplo:
Canal directo, entrada `AYER CASA YO HERMANO A MIO LLAMAR`: «Ayer, en casa, llamé a mi hermana.»
Canal inverso, «Tengo un cuchillo.»: `YO CUCHILLO TENER`.

Debido a estas divergencias morfosintácticas, la equivalencia entre ambas lenguas no admite un mapeo léxico directo o traducción término a término. Quedan fuera la pregunta o la negación hechas solo con la cara, y cualquier significado que no tenga una seña manual en este recorte.

3.1.3. Sistemas de tiempo real y tiempo pseudo-real

El tiempo pseudo-real, en este prototipo, es una suma de esperas distintas:

- **Pausa.** Empieza cuando las manos salen del cuadro y termina a los 4 segundos si no vuelven. Cierra la frase. No es tiempo del modelo.
- **Cola.** Empieza cuando la solicitud llega al servidor y termina cuando el modelo la toma. El servidor informa ese tiempo aparte de la inferencia.
- **Inferencia.** Empieza cuando el modelo toma la solicitud y termina cuando devuelve el texto. Es el único tramo al que el requerimiento RNF-01 exige un máximo de 2 segundos en banda ancha.
- **Transporte.** Es el tiempo de red. Se obtiene restando, al intervalo entre el envío del cliente y la llegada del texto, la cola y la inferencia que informa el servidor. No incluye esos dos tramos.
- **Entrega.** Empieza cuando el texto vuelve al cliente. En el canal directo incluye el subtítulo y la voz. En el canal inverso incluye la descarga de los movimientos, el dibujo y la salida del video ya compuesto hacia la llamada.

El ciclo que espera el interlocutor suma pausa, transporte, cola, inferencia y entrega. La meta de 2 segundos cubre solo la inferencia. Este capítulo define los tramos; el procedimiento que los mide está en la Sección 7.3.1.

3.1.4. Visión computacional y reconocimiento basado en esqueletos

El reconocimiento de señas a partir de flujo de video crudo opera, en cada fotograma, sobre millones de valores cromáticos. Un cuadro de 1920×1080 píxeles en espacio RGB contiene:

$$1920 \times 1080 \times 3 = 6\,220\,800 \tag{3.2}$$

escalares. Esta representación resulta altamente sensible a las variaciones de iluminación, al fondo y a la vestimenta del usuario. El reconocimiento basado en esqueletos reemplaza esta representación densa mediante la clasificación de un conjunto reducido de coordenadas articulares, denominadas *landmarks*.

Para la estimación postural se utiliza *MediaPipe Holistic*, una arquitectura de percepción distribuida para dispositivos de borde [8]. Este marco no constituye un detector único, sino una serie de modelos preentrenados encadenados, que se ejecuta de forma local en el

cliente sin transmitir la imagen cruda. El primer modelo, *BlazePose*, localiza al sujeto y estima 33 puntos de pose corporal [9]. Dichos puntos abarcan la estructura facial gruesa, el torso y las extremidades. A partir de las coordenadas de las muñecas, el pipeline delimita una región de interés (*ROI*) para cada mano, donde el modelo específico de manos (*hand*) infiere 21 puntos tridimensionales por mano (la muñeca y cuatro articulaciones por dedo) [10]. La coordenada z de cada mano es una profundidad relativa al origen en la muñeca, no una distancia en metros.

Asimismo, los datos cinemáticos de referencia gestual se archivan en un formato binario especializado con extensión `.pose` [11]. La especificación del formato y su esquema de serialización provienen de la biblioteca *pose-format*, desarrollada para el almacenamiento y manipulación de datos posturales en lenguas de señas, la cual organiza los puntos en un encabezado de metadatos globales y un tensor multidimensional de coordenadas espacializadas.

3.1.5. Modelado de secuencias temporales y atención

La clasificación gestual requiere procesar secuencias temporales de coordenadas articulares para asignar una clase o glosa. Un modelo recurrente, por ejemplo una red LSTM, condensa la historia en un estado de tamaño fijo. Ese estado puede diluir los instantes iniciales del gesto al llegar a la pose final.

El mecanismo de atención permite que cualquier instante consulte otros instantes de la secuencia en una operación matricial, sin pasar por un único estado acumulado [12]. Eso no elimina todas las pérdidas: el *softmax* concentra el peso y la dimensión de las claves es finita. Para cada entrada, el modelo proyecta tres vectores: consulta (Q), clave (K) y valor (V). La respuesta de atención se calcula mediante la relación:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{3.3}$$

Q, K y V son las proyecciones de consulta, clave y valor. d_k es la dimensión de las claves; el cociente $1/\sqrt{d_k}$ evita que el producto $QK^\top$ crezca con esa dimensión y sature el *softmax*.

3.1.6. Modelos de lenguaje pequeños, adaptación *LoRA* y cuantización

Los modelos de lenguaje pequeños (SLM, por sus siglas en inglés) poseen un número reducido de parámetros en comparación con las arquitecturas de gran escala. Su función principal consiste en tomar una secuencia de texto de entrada para predecir la continuación lógica del mensaje. En este prototipo, los SLM asumen la traducción semántica bidireccional. Esta dimensión reducida facilita la etapa de ajuste fino, además de permitir la ejecución posterior del traductor, previamente cuantizado, directamente en la CPU.

El procesamiento de información requiere la conversión del texto en *tokens*, los cuales constituyen los componentes básicos del vocabulario del tokenizador. Cabe destacar que un *token* no equivale necesariamente a una palabra completa, sino que puede abarcar caracteres individuales, subpalabras o palabras enteras. Estos segmentos representan la unidad fundamental mediante la cual la arquitectura divide la entrada y cuantifica la salida generada.

La adaptación de estos modelos se ejecuta mediante la técnica de Adaptación de Bajo Rango (*LoRA*) [13], la cual congela la matriz de pesos preentrenados $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$

e introduce dos matrices entrenables de bajo rango $\mathbf{B} \in \mathbb{R}^{d\times r}$ y $\mathbf{A} \in \mathbb{R}^{r\times k}$, con rango $r \ll \text{mín}(d, k)$. La actualización lineal adopta la forma:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA}) \tag{3.4}$$

donde α actúa como un factor de escala constante. La Figura 3.1 dibuja el caso cuadrado $k = d$: $\mathbf{B}$ mide $d \times r$ y $\mathbf{A}$ mide $r \times d$. El producto BA recupera el tamaño de ΔW.

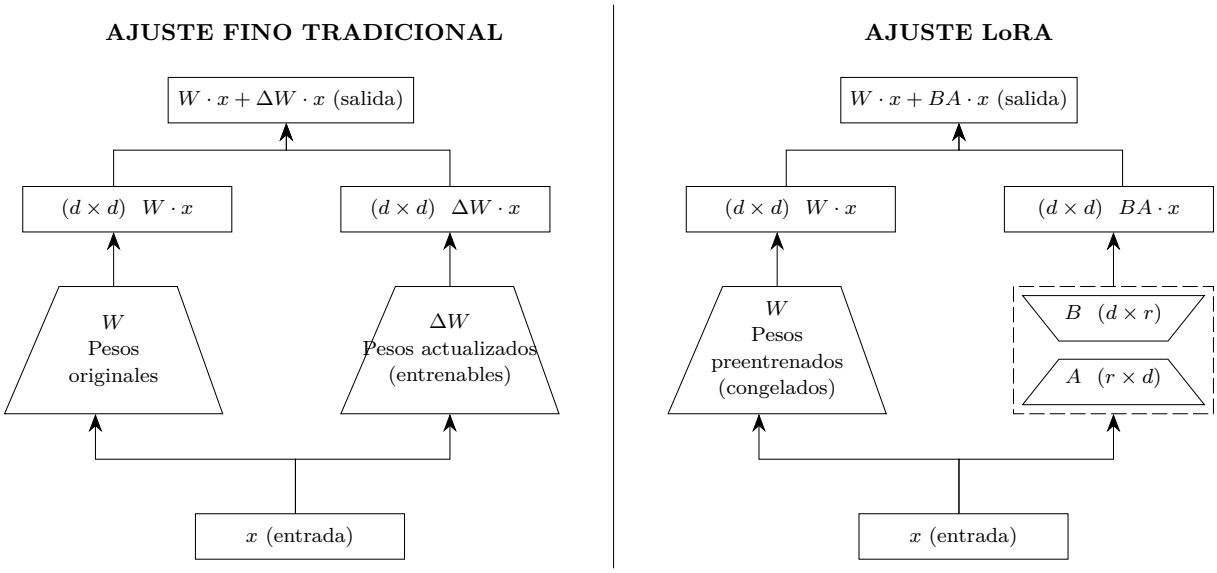


Figura 3.1: Comparación algebraica del ajuste. A la izquierda, un ajuste completo se escribe como $W \cdot x + \Delta W \cdot x$. En la práctica se actualizan los pesos originales; no se conserva una base congelada con una rama aparte. A la derecha, LoRA descompone ΔW en B y A. El dibujo muestra el caso $k = d$ y escribe BA.

En este trabajo, el procedimiento de inicialización de los adaptadores sigue la formulación estándar propuesto por Hu et al. [13]: la matriz $\mathbf{A}$ se inicializa mediante una distribución uniforme de Kaiming [**he2015delving**] (esquema que escala la varianza de los pesos según la dimensión de entrada para preservar la estabilidad de los gradientes), mientras que la matriz $\mathbf{B}$ se establece en cero ($\mathbf{B} = \mathbf{0}$). Esta configuración asegura que el producto inicial $\mathbf{BA}$ resulte nulo, de modo que el modelo comience el proceso de ajuste respondiendo de forma idéntica a la arquitectura base W_0.

Tras el ajuste, los adaptadores se fusionan con la base. Ese resultado sigue en 16 bits. Hay que distinguir tres cosas. El formato de archivo es GGUF. La configuración de cuantización es el preset `Q4_K_M` de `llama.cpp` [14]. La ecuación de abajo es una reconstrucción simplificada de un peso dentro de un bloque.

El conversor parte cada tensor en superbloques de 256 pesos y, dentro, en bloques más cortos. Cada bloque guarda enteros de 4 bits junto con una escala s y un mínimo m [15]:

$$\hat{w} = m + s\,q \tag{3.5}$$

donde q es el entero de ese peso. En la tabla de cuantización de `llama.cpp`, medida sobre Llama 3.1 8B, `Q4_K_M` queda en 4,8944 bits por peso, cerca de 5 frente a 16 [14]. Esa fila es el archivo completo dividido por los parámetros de ese modelo. No separa la cabecera, el vocabulario ni los tensores de otra precisión, y no se traslada tal cual a los modelos

de este trabajo. No es una medición de calidad sobre las oraciones. La mayor parte de los tensores queda en `Q4_K`. El convertidor reserva `Q6_K` para las proyecciones `v_proj` y `down_proj`. Las normalizaciones permanecen en 32 bits. Esa mezcla depende de la versión del convertidor; no se informa aquí el porcentaje exacto sobre los GGUF de este trabajo.

Una vez finalizada la adaptación del modelo mediante LoRA, la fusión de las matrices de bajo rango con los pesos base genera una arquitectura consolidada en precisión de punto flotante de 16 bits. Sin embargo, su despliegue directo en el microservicio demanda una optimización adicional para reducir la huella de memoria y acelerar la inferencia en CPU. En esta etapa interviene la cuantización, un proceso de compresión numérica que representa los parámetros con una menor cantidad de bits sin comprometer sustancialmente la precisión lingüística del sistema.

La configuración de cuantización utilizada es el preset `Q4_K_M` de `llama.cpp` [14]. La ecuación de abajo es una reconstrucción simplificada de un peso dentro de un bloque.

El conversor parte cada tensor en superbloques de 256 pesos y, dentro, en bloques más cortos. Cada bloque guarda enteros de 4 bits junto con una escala s y un mínimo m [15]:

$$\hat{w} = m + s\,q \tag{3.6}$$

donde q es el entero de ese peso. El formato de almacenamiento empleado es GGUF. Según las mediciones de la tabla de cuantización de `llama.cpp` sobre el modelo Llama 3.1 8B, la configuración `Q4_K_M` alcanza un promediado de 4,8944 bits por peso, valor cercano a 5 bits frente a los 16 bits originales [14]. Cabe destacar que dicha métrica representa el tamaño total del archivo dividido por el número global de parámetros. Por ende, no discrimina la cabecera, el vocabulario ni los tensores preservados en otras precisiones, de modo que el valor no se traslada de manera idéntica a los modelos desarrollados en este trabajo ni constituye una evaluación de la calidad de las oraciones generadas.

3.1.7. Estándares de captura web y arquitecturas de extensión web

La captura de flujo de video desde el navegador se fundamenta en la API *Media Capture and Streams* [16]. Mediante la interfaz `getUserMedia()`, el sistema solicita acceso a los dispositivos de entrada locales (como la cámara web) para obtener las pistas de dispositivos, por ejemplo la cámara. La misma especificación permite varias pistas asociadas a una misma fuente.

En las pruebas de captura de este trabajo, en Windows con Chrome y Google Meet, la apertura directa de la cámara mediante `getUserMedia()` devolvió la excepción `NotReadableError` cuando la videoconferencia ya tenía el dispositivo. Por ese motivo, la arquitectura de ILSA omite una segunda apertura de la cámara y reutiliza el objeto `MediaStream` de la reunión. Al desactivar la extensión, el sistema conserva los flujos activos y no invoca `stop()` sobre sus pistas.

El lienzo arma la imagen que sale a la llamada. Los fotogramas se leen de ese mismo flujo y el dibujo agrega el subtítulo o el esqueleto. `canvas.captureStream()`, de *Media Capture from DOM Elements* [17], construye un `MediaStream` a partir del lienzo donde esos dibujos ya están.

En el ámbito de las extensiones web bajo el estándar Manifest V3 [18], la Política de Seguridad de Contenido (CSP) prohíbe la ejecución de código remoto inyectado. Para ejecutar los clasificadores y estimadores de visión computacional se implementa una arquitectura desacoplada en dos entornos aislados:

- **Documento en segundo plano (*offscreen*):** Entorno habilitado por la API `chrome.offscreen` [19] que provee contexto de DOM completo para coordinar la mensajería y gestionar lienzos desacoplados.
- **Entorno aislado (*sandbox*):** Entorno habilitado por el manifiesto [20] con CSP relajada que autoriza la instanciación de trabajadores web (*Web Workers*) dedicados al cómputo intensivo de inferencia.

3.2. Estado del arte

La comparación usa cuatro ejes: lengua, tipo de entrada, sentido de la traducción e integración en una llamada. El prototipo reconoce señas aisladas de LSA, a partir de los puntos de una cámara web, dentro de Google Meet. No reconoce un discurso continuo sin corte entre señas.

Para evaluar el alcance del prototipo propuesto en relación con los desarrollos vigentes, la Tabla 3.1 sintetiza las herramientas y trabajos de investigación más representativos actuales. La comparación se realiza a partir de cuatro ejes estructurales: la lengua abarcada, la modalidad de la entrada, el sentido de la traducción y el nivel de integración en plataformas de comunicación sincrónica.

Cuadro 3.1: Sistemas de referencia según lengua, entrada, sentido e integración.

Sistema	**Lengua y entrada**	**Sentido**	**Integración**
Señario de la CAS [3]	LSA, material pedagógico	Consulta estática.	Fuera de la videollamada.
LSApp [21]	LSA, ítem de diccionario o deletreo	Hacia la animación, en un solo sentido.	Aplicación móvil, sin reconocimiento.
Blessit y Línea 144 [22, 23]	LSA, video con una persona	Interpretación humana.	Videollamada sujeta al turno del operador.
LSA64 y redes recurrentes posteriores [24, 25]	LSA, video con guantes de colores	Reconocimiento de señas aisladas.	Laboratorio. Los guantes impiden el paso directo a una cámara web.
SignAll [26]	ASL, visión	Hacia inglés escrito.	Escena de captura, fuera de la llamada.
WLASL [27]	ASL, video	Reconocimiento léxico.	*Corpus*. No traduce oraciones ni trabaja en LSA.
Hand Talk [28]	Lengua de señas brasileña (Libras) y ASL, texto	Hacia un avatar tridimensional.	Sin reconocimiento y sin LSA.

A partir del análisis del estado del arte se evidencia la falta de soluciones informáticas capaces de integrar de forma simultánea tres aspectos clave: el procesamiento específico de la LSA, la bidireccionalidad en la traducción y la integración con plataformas de

videollamada. Mientras que las herramientas existentes se limitan a la consulta estática de diccionarios, la intermediación humana o la ejecución en escenarios controlados fuera del flujo de conversación, el prototipo propuesto satisface esta vacancia mediante una arquitectura automatizada en tiempo pseudo-real que opera de manera transparente dentro de Google Meet.

3.3. Metodología de desarrollo

El proyecto se estructuró bajo un modelo de ciclo de vida iterativo e incremental [29], compuesto por 6 fases principales:

1. **Curación de datos y formalización léxica:** Delimitación de 100 señas de LSA en colaboración con un informante calificado; diseño del entorno de grabación; captura de clips en resolución 1080p; y generación del dataset de coordenadas vectoriales normalizadas.

2. **Entrenamiento y diagnóstico del clasificador gestual:** Prueba de ventanas de 8, 12, 16 y 32 fotogramas; búsqueda de hiperparámetros con Optuna [30]; y evaluación en cámara del checkpoint de 100 clases, con diez tomas por seña, dos condiciones de iluminación, y persona y encuadre fijos.

3. **Desarrollo de los módulos semánticos bidireccionales y diseño de despliegue:** Ingeniería de prompts; ajuste con LoRA sobre Qwen2.5 [31], SmolLM2 [32] y Llama 3.2 1B Instruct [33], en el orden manual de la Sección 3.1.2; cuantización GGUF [14]; y evaluación de fidelidad según la Sección 3.3.

4. **Integración y renderizado gestual:** Acoplamiento del pipeline en el motor local ILSA mediante patrones de diseño desacoplados; comunicación con el servidor semántico; e intercepción de video en Google Meet mediante *canvas* sintético y extensión Manifest V3 [18].

5. **Avatar e integración visual inversa:** Desarrollo del controlador del avatar para la representación esquelética de las glosas del canal inverso; implementación del subsistema de contingencia para deletreo manual dactilológico; e integración en la extensión.

6. **Fase de pruebas:** Medición de las etapas de la Sección 3.1.3, con el procedimiento de la Sección 7.3.1.

Las tareas del trabajo se gestionaron en YouTrack [34]. Cada tarea quedó registrada como un ticket, con su estado y las horas empleadas. Ese registro sigue el tiempo dedicado a cada tarea a lo largo de las fases.

Fidelidad semántica La fidelidad semántica, en este trabajo, es la conservación de la proposición dentro de los mensajes que el prototipo admite: quién interviene, de qué se habla, en qué tiempo, si hay negación y si hay pregunta. La evaluación automática compara la salida con una referencia. Se registran cinco métricas, cada una evalúa un fallo distinto:

- **BLEU** [35]. Precisión de n-gramas: qué porción de las secuencias de la salida ya está en la referencia. Una salida corta recibe una penalización. No premia, con el mismo peso, las palabras de la referencia que la salida omitió.
- **ROUGE-L** [36]. La subsecuencia común más larga que respeta el orden. Dos textos pueden compartir palabras y aun así bajar si esas palabras no aparecen en el mismo orden.
- **METEOR** [37]. Alinea palabras sueltas. Cuenta la coincidencia exacta, la raíz y, si hay un léxico de sinónimos, el sinónimo. Junta precisión y recall, y baja si el texto queda partido en muchos fragmentos.
- `BERT Score` [38]. Compara el vector de cada *token* de la salida con los de la referencia, mediante un modelo de lenguaje preentrenado. El F1 de esa alineación sube cuando el sentido está cerca aunque las palabras no coincidan.
- `BERT Score` [38]. Compara el vector de cada *token* de la salida con los de la referencia mediante un modelo de lenguaje preentrenado. Su valor F_1 representa la media armónica entre la precisión (fidelidad de los términos generados) y la cobertura (*recall*, porcentaje de información retenida de la referencia). De este modo, la métrica sube cuando el sentido se mantiene próximo, aun cuando no exista coincidencia exacta de palabras.
- `Comet Score` [39]. Un modelo entrenado para aproximar juicios humanos de traducción. Usa la entrada, la salida y la referencia. Estima si el sentido de la entrada llegó a la salida.

Un puntaje alto de solapamiento no equivale a comprensión de LSA ni a validación del avatar. La configuración de cada métrica —modelo de `BERT Score`, versión de `Comet Score`, normalización, agregación y tratamiento de referencias múltiples— se informa junto con los resultados, en la Sección 7.2.2.

3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial (IA)

Herramientas empleadas. Durante las fases de desarrollo y redacción se utilizaron dos herramientas auxiliares:

- **Cursor:** Entorno de desarrollo integrado asistido por inteligencia artificial, utilizado para la navegación del repositorio, autocompletado sintáctico y asistencia en la edición de código en Python y JavaScript.
- **Gemini (Google):** Modelo de lenguaje utilizado como soporte de consulta técnica, facilitador en la asimilación de conceptos teóricos y asistencia en la revisión del documento. También redactó borradores de pares español–glosas para el *corpus* semántico, a partir de un prompt que fijaba el orden manual, el vocabulario y los fenómenos a cubrir. Cada par se leyó y se corrigió para que respetara las instrucciones de sistema de cada canal (Subsecciones del canal directo y del canal inverso, Capítulo 6).

Alcance y propósito de su utilización. La intervención de dichas herramientas fue con el propósito de:

- **Consultas técnicas y conceptuales:** Asistencia en la comprensión de aspectos específicos relativos a la cuantización GGUF Q4_K_M con `llama.cpp`, fundamentos de LoRA, la gestión de memoria en entornos *serverless* y esquemas de comunicación asíncrona en extensiones Manifest V3.
- **Asistencia en el desarrollo de *software*:** Sugerencias de refactorización de código y resolución de incompatibilidades sintácticas en el entorno Chrome Extensions y en los conectores asíncronos de FastAPI.
- **Revisión de formato:** Corrección gramatical y adecuación de estilo académico del informe.

Límites y responsabilidad de autoría. Ninguna herramienta generativa reemplazó la labor intelectual, analítica ni experimental del proyecto. La concepción de la arquitectura híbrida, la recolección y curación del *corpus* de 100 señas, el diseño e iteración de las redes neuronales, el ajuste de los modelos de lenguaje y el análisis de resultados constituyen desarrollo de los autores, quienes asumen la responsabilidad técnica y académica por la totalidad del contenido.

3.4. Herramientas y tecnologías empleadas

Cuadro 3.2: Conjunto de tecnologías del sistema integrado

Categoría	**Herramienta**	**Rol en el sistema**
Entorno y Lenguajes	Python 3.11, JavaScript	Lógica de backend, motor local, servidor semántico y extensión web.
Visión Computacional	MediaPipe Holistic, OpenCV	Extracción de coordenadas articulares corporales y manipulación de video.
Aprendizaje Profundo	PyTorch 2.6	Inferencia local del clasificador gestual.
Modelos de Lenguaje	Transformers, PEFT, Unsloth	Fine-tuning eficiente con adaptadores LoRA para traducción directa e inversa.
Inferencia de Lenguaje	`llama.cpp`, GGUF	Inferencia del traductor en CPU.
Comunicación y Backend	FastAPI [40], Uvicorn	Servidor de interfaz local (ILSA) y microservicio semántico desacoplado.
Servicio en la nube	Google Cloud Run	Instancia del servicio semántico empleada en las pruebas experimentales.
Integración Web	Chrome Extensions (MV3)	Intercepción de media mediante *canvas* `captureStream()` en Google Meet.
Transcripción	Vosk	Transcripción local de audio a texto en el cliente.
Síntesis de Salida	Síntesis WAV (Windows, ILSA)	Generación del WAV en ILSA, mezcla en la pista de audio de Meet y dibujo esquelético del avatar.
Avatar	PoseFormat	Conversión de metadatos y almacenamiento en formato binario `.pose`.

Capítulo 4

Análisis de requerimientos

4.1. Requerimientos funcionales

- **RF-01 — Captura y extracción de puntos clave gestuales:** El sistema debe capturar el flujo de video del usuario en tiempo real en el cliente y extraer la información posicional de la pose corporal y de ambas manos para su posterior procesamiento.
- **RF-02 — Reconocimiento y clasificación de gestos:** El sistema debe procesar las secuencias de puntos clave extraídos y clasificar localmente los gestos realizados en su correspondiente glosa en LSA, y reportar la glosa identificada y su nivel de confianza.
- **RF-03 — Traducción semántica de LSA a Español:** El sistema debe convertir una secuencia de glosas identificadas en una oración en español rioplatense, procesar localmente las construcciones simples (deletreos o glosas únicas) y derivar las estructuras complejas al servicio de traducción semántica.
- **RF-04 — Presentación y síntesis de la salida traducida:** El sistema debe permitir la emisión del mensaje traducido al español hacia los participantes de la videollamada mediante subtítulos en pantalla, síntesis de voz inyectada en el canal de audio, o ambos modos, y respetar el estado de silencio del micrófono del usuario.
- **RF-05 — Captura del turno del interlocutor oyente:** El sistema debe capturar las intervenciones de voz o texto de los participantes oyentes en la videollamada para alimentar el canal de traducción hacia el usuario sordo.
- **RF-06 — Traducción semántica de Español a LSA:** El sistema debe transformar el texto en español capturado desde el turno del interlocutor en una secuencia ordenada de glosas equivalentes en LSA.
- **RF-07 — Interpretación visual mediante avatar:** El sistema debe representar la secuencia de glosas traducidas mediante la reproducción animada de un avatar esquelético en tiempo psudo-real. En caso de ausencia de animación para una glosa específica, el sistema debe recurrir a la representación por dactilología (deletreo manual).
- **RF-08 — Integración con la interfaz de videollamada:** El sistema debe integrarse sobre la interfaz del cliente de videollamadas (Google Meet), superponer

los paneles de control, el flujo de video procesado y las salidas de texto y avatar sin interrumpir el funcionamiento nativo de la plataforma.

- **RF-09 — Gestión de estado e inicialización del sistema:** El sistema debe verificar la disponibilidad de los servicios de procesamiento y traducción durante la etapa de inicialización, permitir el acceso al panel de control únicamente tras validar la conectividad operativa, y preveer mecanismos de reintento en caso de falla.

4.2. Requerimientos no funcionales

- **RNF-01 — Rendimiento y latencia del pipeline de procesamiento:** El sistema debe operar bajo un régimen de tiempo pseudo-real. las metas operativas se dividen por subsistemas:

  1. *Clasificación gestual local:* El módulo de inferencia corporal (TinySkeleton) debe procesar y clasificar el vector de landmarks en un tiempo inferior a 50 ms por seña.
  2. *Transcripción de voz:* La conversión de audio a texto en el extremo oyente debe ejecutarse de forma local con un tiempo de procesamiento inferior a 2000 ms.
  3. *Inferencia semántica remota:* La generación de la oración en español (canal directo) o la reestructuración a glosas (canal inverso) en el servidor remoto no debe superar los 2000 ms por solicitud con el servicio en estado activo.
  4. *Transporte de red:* La latencia de ida y vuelta (RTT) entre el cliente y las APIs remotas en la nube debe ser inferior a 500 ms bajo conexiones de banda ancha estándar.
  5. *Ensamblado cinemático del avatar:* El procesamiento geométrico y la interpolación de clips de animación en el cliente debe realizarse en un tiempo inferior a 350 ms.

- **RNF-02 — Privacidad y segregación de datos biométricos:** El procesamiento de visión computacional y extracción de características corporales debe ejecutarse exclusivamente en el dispositivo cliente. Queda estrictamente prohibida la transmisión de flujos de video o imágenes crudas hacia los servicios remotos en la nube; el canal de comunicación externo debe transmitir únicamente cargas de datos puramente textuales (glosas, letras de dactilología o cadenas en español).

- **RNF-03 — Fiabilidad y precisión del reconocimiento gestual:** El clasificador gestual local debe alcanzar un acierto Top-1 superior al 75 % en condiciones de iluminación doméstica estándar.

- **RNF-04 — Desacoplamiento y escalabilidad de la arquitectura en la nube:** Los servicios backend (traducción semántica y servidor de activos de animación) deben implementarse como microservicios sin estado (*stateless*) desacoplados, expuestos mediante interfaces REST HTTP/HTTPS, lo que permita su escalado horizontal dinámico e independiente ante incrementos en la demanda concurrente.

- **RNF-05 — Usabilidad, ergometría visual y no invasividad:** La interfaz de usuario inyectada en la plataforma de videollamada (HUD) debe posicionarse en

un recuadro no invasivo que evite la oclusión de los elementos críticos del dominio comunicativo (rostro, torso y manos del señante o interlocutor) durante el desarrollo de la videollamada.

4.3. Restricciones del sistema

- **RC-01 — Delimitación del alcance léxico:** El modelo de reconocimiento gestual está acotado estrictamente a un corpus de 100 señas aisladas de la LSA, previamente validadas con profesionales lingüísticos calificados.
- **RC-02 — Alcance del despliegue en la nube:** La evaluación experimental del prototipo está acotada al aprovisionamiento de los microservicios semánticos y de movimiento en la infraestructura *serverless* de Google Cloud Run mediante instancias de ejecución fija. Queda fuera del alcance de validación el despliegue del *clúster* elástico multinodo con auto-escalado dinámico.
- **RC-03 — Entorno de ejecución y compatibilidad de plataforma:** ILSA y el componente de extensión web deben operar exclusivamente sobre sistemas operativos Windows (64 bits) y en navegadores basados en la arquitectura Chromium compatibles con la especificación Manifest V3.
- **RC-04 — Marco tecnológico de estimación postural:** El pipeline de extracción de coordenadas articulares en el extremo cliente está supeditado a la ejecución del framework MediaPipe Holistic como motor primario de visión computacional.
- **RC-05 — Perfil de hardware objetivo en el cliente:** El subsistema cliente debe ser capaz de ejecutar la extracción de puntos y la clasificación gestual en computadoras portátiles convencionales de gama media (procesadores de 4 núcleos con soporte AVX2 y 8 GB de memoria RAM), sin requerir la presencia de unidades de procesamiento gráfico (GPU) dedicadas.

4.3.1. Especificación del entorno de ejecución cliente

El cuadro siguiente propone un equipo mínimo y uno con más margen. El nombre comercial de una familia cubre piezas distintas: el criterio operativo es el número de núcleos, el soporte AVX2 y el piso de reloj, no la etiqueta Core i3 o Ryzen 3.

El mínimo se elige para que convivan tres cargas: la videollamada, la extracción de puntos en el navegador y la clasificación de ILSA en CPU [18]. De ahí salen cuatro núcleos, 8 GB de RAM, gráficos con WebGL para el lienzo, disco de estado sólido, cámara de 720p y banda ancha. Windows 10/11 de 64 bits es el sistema del empaquetado de ILSA. Por otra parte, la fila recomendada, es una propuesta de holgura: más núcleos, más memoria y cámara de 1080p. No es una configuración medida.

Edge, Brave, Opera y Vivaldi comparten motor Chromium, y por eso se proponen. Este trabajo no informa una pasada en cada uno: las políticas y las APIs pueden diferir. La comparación entre la máquina de desarrollo y la máquina objetivo queda en la Sección 7.3.4. Esa sección describe el comportamiento observado. El cuadro de mínimos de esta sección es una propuesta de entorno, no una batería de pruebas por modelo de procesador.

Cuadro 4.1: Requisitos de har*hardware*dware y *software* del dispositivo cliente.

Componente	**Requisitos mínimos (Dispositivo base)**	**Requisitos recomendados**
Procesador (CPU)	Intel Core i7-10510U / i5 (10ª gen.) o AMD Ryzen 3 (serie 3000) (4 núcleos / 8 hilos, base $\geq$ 1,8 GHz, AVX2).	Intel Core i5-11400H / AMD Ryzen 5 5600U (6 núcleos / 12 hilos).
Memoria RAM	8 GB DDR4 ($\geq$ 2,5 GB libres para el navegador y entorno de ejecución).	16 GB DDR4 / DDR5.
Gráficos (GPU)	Gráficos integrados Intel UHD Graphics 620 / 630 o AMD Radeon Vega 3 con soporte WebGL 2.0.	Intel Iris Xe / AMD Radeon 600M+ o GPU dedicada básica.
Almacenamiento	Disco de estado sólido (SSD) con 2 GB de espacio libre disponible.	SSD NVMe con $\geq$ 5 GB libres.
Cámara web	Resolución 720p (1280×720) a 24/30 FPS.	Resolución 1080p a 30 FPS.
Conectividad	Conexión de banda ancha ($\geq$ 10 Mbps bajada / $\geq$ 3 Mbps subida).	Conexión por fibra óptica ($\geq$ 50 Mbps).
Entorno de *software*	Windows 10/11 (64 bits), Google Chrome 115+ (soporte Manifest V3).	Windows 11 (64 bits), Google Chrome última versión estable.

Condiciones de funcionamiento La CPU debe sostener a la vez la videollamada, MediaPipe en WebAssembly y el clasificador en CPU. El informe no asigna núcleos por carga ni un tiempo de clasificación en milisegundos. Las condiciones que importan, en el mínimo propuesto, son: 4 núcleos con AVX2, 8 GB de RAM con memoria libre para el proceso, WebGL, Windows 10/11 de 64 bits y Chrome 115 o posterior.

4.4. Casos de uso

4.4.1. Casos de uso esenciales en formato breve

- **CU-01 — Conversar con otra persona (Modo Sordo):** El usuario sordo realiza señas frente a la cámara web durante la videollamada. El sistema procesa localmente los gestos, consulta el servicio semántico en la nube para estructurar la oración en español y proyecta el resultado en la videollamada mediante subtítulos, síntesis de voz o ambos canales simultáneamente.
- **CU-02 — Seleccionar lateralidad (mano hábil):** El usuario accede a la configuración de lateralidad en la extensión y selecciona entre la modalidad diestra o zurda. El sistema aplica las transformaciones geométricas correspondientes sobre las coordenadas articulares para ajustar el motor de clasificación durante la sesión activa.

- **CU-03 — Configurar canal de salida:** El usuario selecciona el canal de preferencia para proyectar la traducción en la llamada de Google Meet (subtítulos, audio o ambos). El sistema actualiza la interfaz de salida de la extensión.
- **CU-04 — Conversar con otra persona (Modo Oyente):** El usuario oyente introduce el mensaje mediante voz o texto dentro de la llamada. El sistema analiza la entrada en español, deriva la secuencia de glosas hacia el motor gráfico y reproduce la animación correspondiente mediante el avatar esquelético o el recurso dactilológico.
- **CU-05 — Iniciar y verificar sesión:** El usuario inicia el motor local o cambia de modo operativo desde la interfaz de la extensión. El sistema verifica la disponibilidad de los servicios en la nube y habilita el panel principal de control tras confirmar la conectividad.

4.4.2. Diagrama de casos de uso

La Figura 4.1 ubica a los dos actores frente a las cinco funciones del prototipo. El usuario señante opera el canal directo y la configuración de la sesión. El usuario oyente opera el canal inverso. Ambos pueden abrir el panel. El recuadro punteado delimita el sistema: ILSA, la extensión y los servicios remotos que esas funciones consultan. El detalle de precondiciones, pasos y alternativas está en los cuadros de la subsección siguiente.

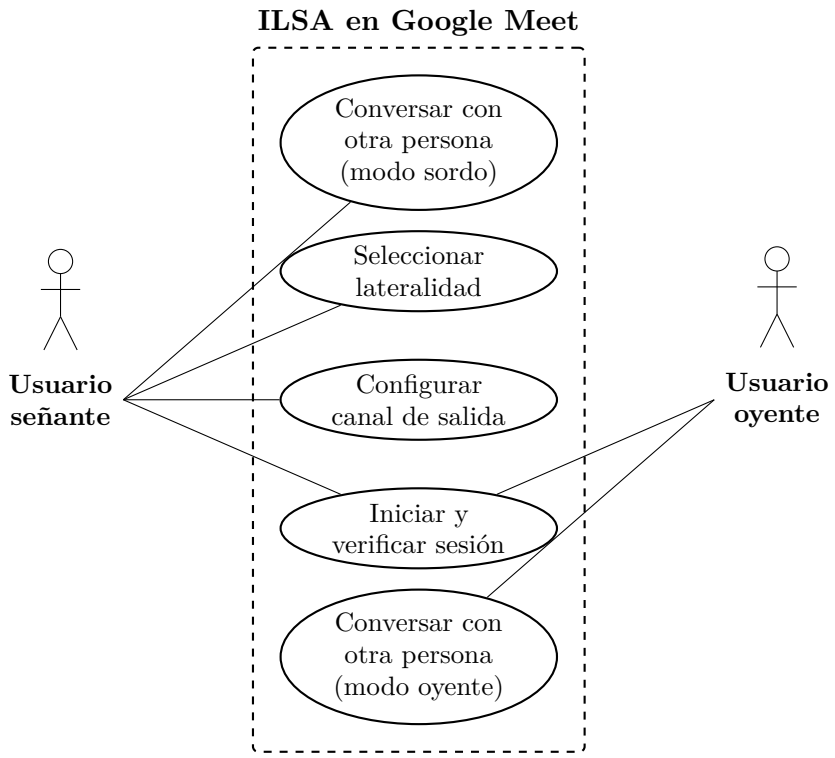


Figura 4.1: Diagrama de casos de uso del prototipo.

4.4.3. Casos de uso expandidos

Identificador	**CU-01: Conversar con otra persona (Modo Sordo)**		
Actor principal	Usuario señante		
Precondiciones	El usuario está unido a una llamada. ILSA está encendido.		
Postcondiciones	El interlocutor lee el subtítulo en el video de Meet cuando la salida es Subtítulo o Ambos. Con Audio o Ambos, ILSA sintetiza un WAV y el *script* de la página lo mezcla en la pista de audio que Meet envía. Quien señа la oye por los parlantes y el resto de la llamada, por esa pista.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Realiza las señas.	
	2		Clasifica la seña. Si la confianza es de al menos 0,75 y el filtro de repetición la admite, el HUD la muestra en cian.
	3	Si necesita otra seña, vuelve al paso 1. Si no, espera 4 segundos sin manos en el cuadro.	
	4		Arma el texto en el equipo si hay una sola glosa léxica, solo dígitos, o solo letras desde dos y sin I ni T. Van al traductor una letra sola, un deletreo con I o T, OJO, VIERNES o AÑOS solas, y las demás frases. Con Subtítulo, deja la oración en el video. Con Audio o Ambos, pide el WAV a ILSA y el script de la página lo mezcla en la pista de Meet.
Extensiones (flujos alternativos)	**1a.** Las manos no entran al cuadro: no empieza ninguna seña y no hay plazo que contar. **2a.** Confianza bajo 0,75: el HUD muestra la predicción en coral. **2b.** Confianza de al menos 0,75 y repetición no admitida: el HUD muestra gris. **2c.** Seña ajena al vocabulario con confianza de al menos 0,75: el HUD la muestra en cian y la glosa incorrecta entra. **4a.** El traductor no responde. Se conservan las glosas aceptadas, en texto, y no se rearma la oración. **4b.** ILSA deja de responder: la extensión no clasifica ni traduce. **4c.** El micrófono de Meet está silenciado: la voz no sale a la llamada y la extensión muestra un aviso.		
Requisitos especiales	RF-01, RF-02, RF-03, RF-04, RF-08, RNF-01, RNF-03.		

Identificador	**CU-02: Seleccionar lateralidad (mano hábil)**		
Actor principal	Usuario señante		
Precondiciones	ILSA está en el panel principal. El control de mano muestra diestro seleccionado.		
Postcondiciones	La sesión queda con la mano elegida. Con zurdo, el vector se espeja y se cruzan las manos.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Ve diestro ya seleccionado y selecciona ENCENDER.	
	2		Conserva la mano derecha. El vector no se espeja.
Extensiones (flujos alternativos)	**1a.** Selecciona zurdo: ILSA espeja el eje horizontal y cruza las dos manos.		
Requisitos especiales	RF-01, RF-02.		

Identificador	**CU-03: Configurar canal de salida**		
Actor principal	Usuario señante		
Precondiciones	El usuario está unido a una llamada. El panel de la extensión está disponible.		
Postcondiciones	La salida queda en Subtítulo, Audio o Ambos, según el control «Salida en Meet». El subtítulo llega en el video. La voz, cuando hay Audio o Ambos, sale de un WAV de ILSA mezclado en la pista de Meet. Si no hay elección, queda Subtítulo.		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Abre el panel de la extensión.	
	2		Muestra tres opciones bajo «Salida en Meet»: Subtítulo, Audio y Ambos.
	3	Elige una opción.	
	4		Aplica esa salida. Subtítulo deja el texto en el video y no emite voz. Audio mezcla la voz en la pista de Meet sin subtítulo. Ambos dejan el texto en el video y mezclan la voz. En Audio y en Ambos, ILSA sintetiza el WAV y el script de la página lo incorpora a la pista que Meet envía. Quien seña la oye por los parlantes.
Extensiones (flujos alternativos)	**3a.** sin elección se conserva Subtítulo y no hay voz.		
Requisitos especiales	RF-04, RF-08.		

<table>
<tr><td>Identificador</td><td colspan="3">CU-04: Conversar con otra persona (Modo Oyente)</td></tr>
<tr><td>Actor principal</td><td colspan="3">Usuario oyente</td></tr>
<tr><td>Precondiciones</td><td colspan="3">El usuario está unido a una llamada y la extensión está en modo oyente.</td></tr>
<tr><td>Postcondiciones</td><td colspan="3">El interlocutor ve el esqueleto de las glosas que tienen archivo. Lo que no se dibuja queda en texto.</td></tr>
<tr><td rowspan="5">Escenario Principal de éxito (flujo básico)</td><td></td><td>Usuario oyente</td><td>Sistema</td></tr>
<tr><td>1</td><td>Habla durante la llamada.</td><td></td></tr>
<tr><td>2</td><td></td><td>Transcribe esa voz en la misma computadora.</td></tr>
<tr><td>3</td><td></td><td>Parte el texto por espacios. Una sola palabra pasa a mayúsculas y es la glosa, sin traductor ni diccionario. Dos o más palabras van al servicio semántico.</td></tr>
<tr><td>4</td><td></td><td>Dibuja hasta 24 clips. Si falta un archivo, deletrea; eso no asegura la comprensión. Encola el bloque sin cortar el que está en curso. La cola admite 20 y, al colmarse, descarta el más antiguo.</td></tr>
<tr><td>Extensiones (flujos alternativos)</td><td colspan="3">1a. Escribe en el chat: ILSA toma ese texto, sin transcribirlo, y sigue en el paso 3. La palabra aislada también llega al paso 4.
3a. Dos o más palabras y el traductor no responde: el cliente no declara un plazo propio y espera la respuesta o el corte del servidor (hasta 300 s en Cloud Run). No hay glosas nuevas y se conserva el video. Una respuesta tardía no se aplica. La recuperación es repetir el turno.
4a. Más de 24 archivos: se dibujan los primeros 24 y el resto queda en texto.
4b. El servidor de poses no responde: misma espera que en 3a. Las glosas quedan en texto y el clip queda vacío. La recuperación es repetir el turno.
4c. ILSA deja de responder: no hay transcripción ni dibujo nuevo. La recuperación es Encender o Reintentar.
4d. Cola llena: se descarta el bloque más antiguo, sin cartel. Ese bloque no se dibuja y su subtítulo no aparece. El último espera la animación en curso y hasta 19 bloques. En el tope de 1200 fotogramas, a 30 fps, con 0,2 s de entrada y 0,2 s de salida, cada bloque dura 40,4 s y esa espera llega a 808 s. Ese valor es el techo del código, no una medición de una sesión.</td></tr>
<tr><td>Requisitos especiales</td><td colspan="3">RF-05, RF-06, RF-07, RF-08.</td></tr>
</table>

Identificador	**CU-05: Iniciar y verificar sesión**		
Actor principal	Usuario		
Precondiciones	ILSA está instalado en la computadora.		
Postcondiciones	Si responden el traductor semántico y el servidor de poses, se abre el panel principal. Al Apagar, la sesión queda detenida en ese panel, sin nueva consulta.		
Escenario Principal de éxito (flujo básico)		**Usuario**	**Sistema**
	1	Abre ILSA, o pulsa Encender o Reiniciar.	
	2		Vuelve a la pantalla de carga y consulta el traductor semántico y el servidor de poses.
	3		Ambos responden. Abre el panel principal.
Extensiones (flujos alternativos)	**2a.** El traductor o el servidor de poses no responde: el cliente no declara un plazo propio y espera la respuesta o el corte del servidor (hasta 300 s en Cloud Run, plazo que cubre también un arranque en frío). Queda el botón Reintentar y el panel principal no se abre. La recuperación es pulsar Reintentar, que repite la consulta de los dos servicios. **2b.** ILSA no está en ejecución: la consulta no llega a los servicios. La recuperación es Encender. **1a.** Pulsa Apagar: detiene la sesión, queda en el panel y el botón vuelve a Encender. No vuelve a la carga ni consulta los servicios.		
Requisitos especiales	RF-09.		

Capítulo 5

Diseño del Sistema

5.1. Arquitectura del sistema

El diseño adopta una arquitectura híbrida desacoplada. La extracción de puntos articulares y la clasificación gestual se ejecutan en el equipo del usuario, mientras que el módulo de traducción semántica y servidor de poses operan como un servicio remoto distribuido.

5.1.1. Justificación de la partición y modelo de privacidad

La extracción de puntos clave y la clasificación de señas se ejecutan de manera local en el dispositivo del usuario. Por esta razón, el servicio de traducción y el servidor de poses nunca reciben la señal de video original ni las coordenadas articulares.

La cámara transfiere la imagen directamente desde el equipo hacia Google Meet para cumplir con la exigencia RNF-02. Así, la videollamada procesa el video compuesto, el cual integra la toma original, los subtítulos y la superposición del esqueleto en el modo oyente.

Debido al consumo de memoria y al costo computacional de los modelos de lenguaje, la traducción se aloja en un servicio remoto. Hacia este componente solo se transmiten cadenas de texto, representadas por la secuencia de glosas o la oración en español. Aunque este flujo no incluye material de video, el texto puede incluir datos personales como nombres o números de documento. Por su parte, el servidor de poses procesa la solicitud a partir del nombre de una glosa o las letras de un deletreo, para luego retornar el archivo correspondiente.

El traductor y el servidor de poses no archivan ese texto. Cada réplica opera sin estado: la cadena permanece en memoria solo mientras dura la solicitud y se descarta al responder o al apagar el contenedor.

En el equipo, ILSA retiene el último texto y el búfer de glosas mientras dura la sesión. Encender, Reiniciar o Apagar vacían esa memoria. La instrumentación de latencia escribe, en `%LOCALAPPDATA%/ILSA/metrics/live-AAAAMMDD.jsonl`, la glosa y la oración de cada emisión. Ese archivo no tiene plazo de retención: queda en el disco hasta que se borra a mano. Lo ven quien usa esa computadora y, en la videollamada, quienes reciben el subtítulo o la voz. Si la reunión se graba, el subtítulo queda en esa grabación, fuera del control de ILSA. Las rutas locales `GET /state` y `GET /metrics` exponen el último texto al panel, en `127.0.0.1`.

5.1.2. Variación artificial del conjunto de señas

El *corpus* de señas reúne tres personas en seis sesiones de grabación. Una toma de ese conjunto no cubre otra velocidad, otra posición de cámara ni un fotograma que el extractor pierde. El diseño agrega variaciones artificiales solo sobre los clips de entrenamiento, para que el clasificador vea esos cambios sin una grabación nueva. Los clips apartados para la validación quedan como se grabaron. Los tipos y los rangos están en la Sección 6.2.2.

5.1.3. Flujo integral de datos entre componentes

La interacción entre el navegador, el motor local y los servicios remotos se organiza según la estructura representada en la Figura 5.1. El procesamiento local gestiona la captura y clasificación. Hacia el traductor salen mensajes JSON, glosas u oración. El servidor de poses entrega el archivo `.pose` en binario, no en JSON. Esta descripción no incluye el video ni el audio que transmite Meet.

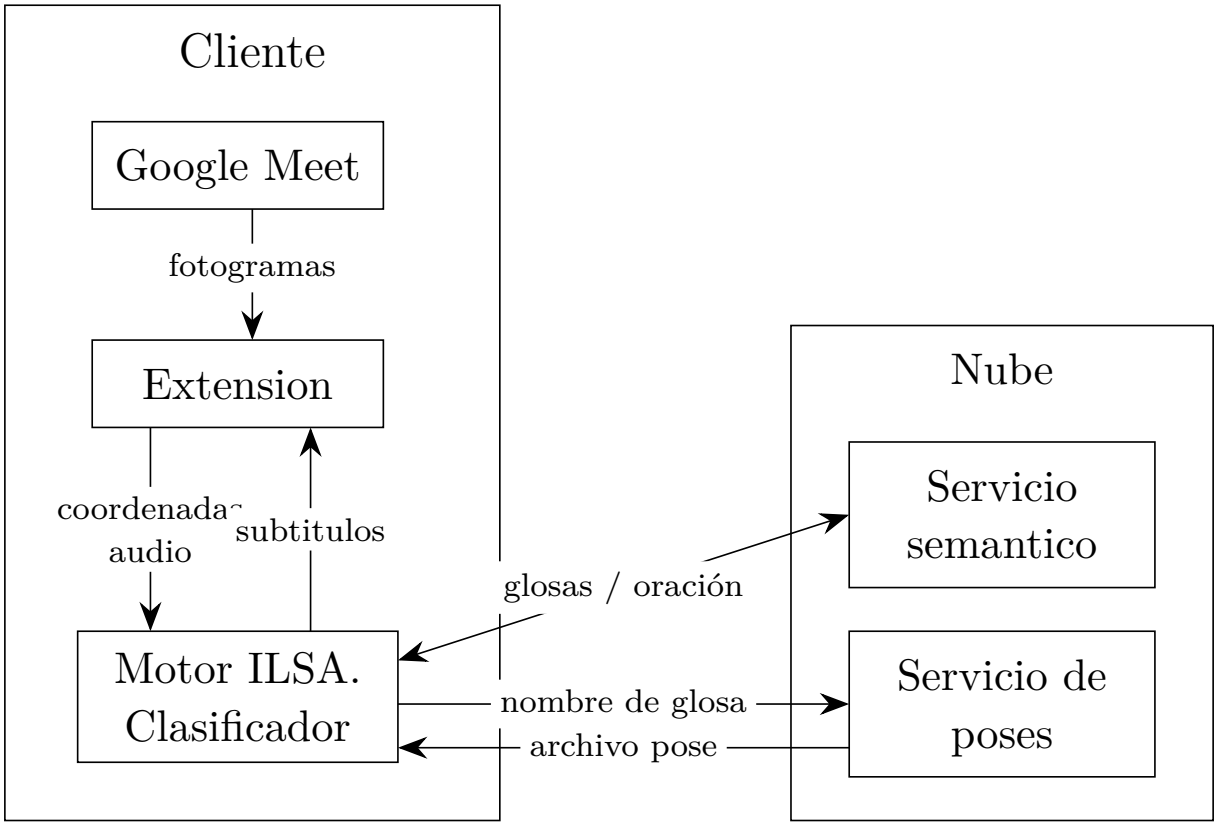


Figura 5.1: Diagrama de la arquitectura general del sistema.

Para detallar la secuencia de intercambio de mensajes, las Figuras 5.2 y 5.3 ilustran el comportamiento de los canales directo e inverso, respectivamente.

En ese diagrama, ILSA arma el texto en el equipo en tres casos: una sola glosa léxica, un enunciado de solo dígitos, o solo letras de dos en adelante y ninguna es I ni T. Una letra sola, un deletreo que incluye I o T, y las señas OJO, VIERNES o AÑOS cuando están solas van al traductor, igual que una frase mixta. Antes de ese envío, las letras seguidas se juntan y los dígitos seguidos también. Con Audio o Ambos, la extensión pide el WAV a ILSA por `POST /tts`. El script de la página mezcla ese WAV con el micrófono y

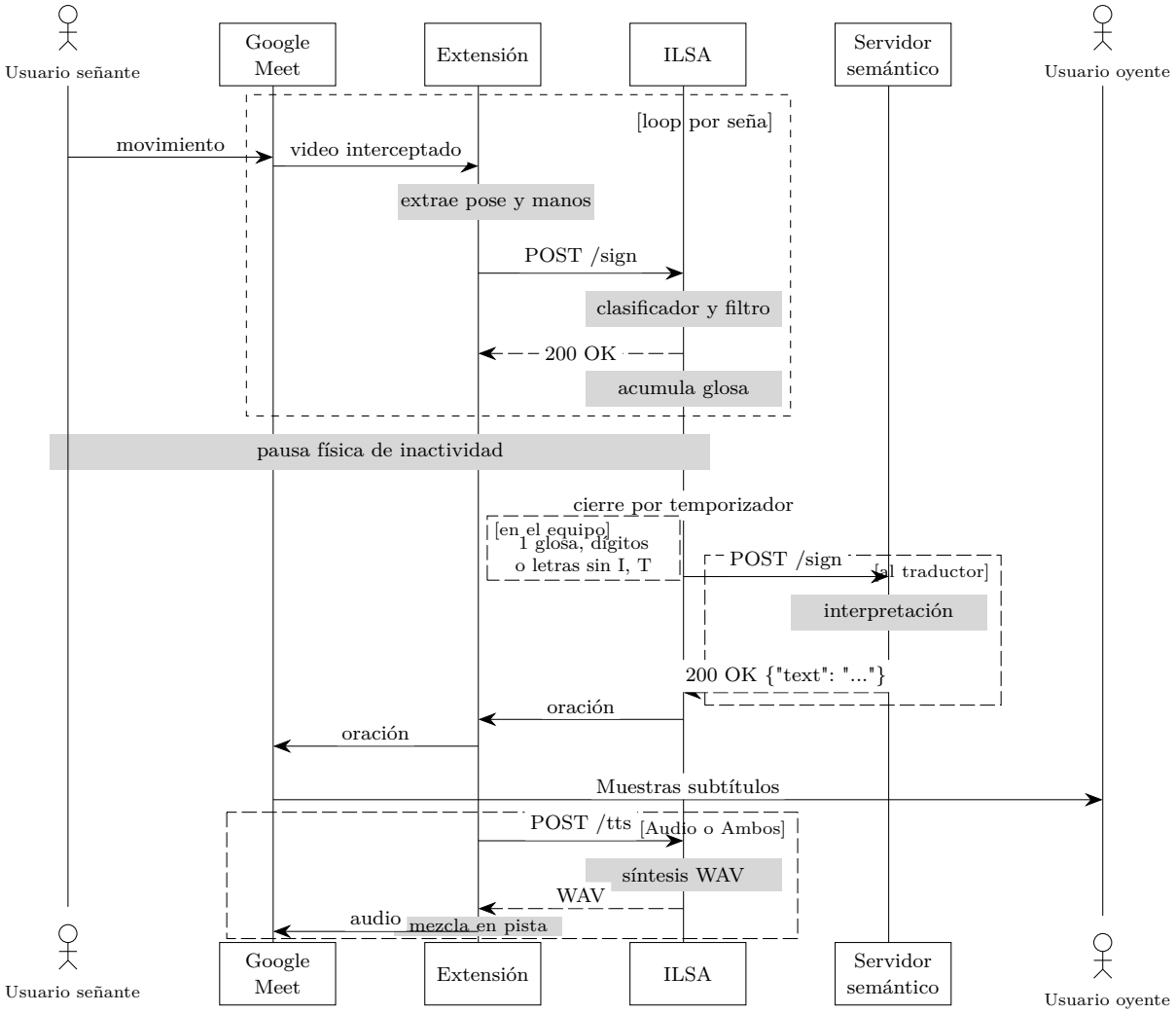


Figura 5.2: Diagrama de secuencia del flujo de traducción de Lengua de Señas a voz y subtítulos.

sustituye la pista de audio que Meet envía. Esa mezcla es la que oye el resto de la llamada. El sintetizador del navegador no entra en esa pista.

En ese diagrama, ILSA parte el texto por espacios antes de llamar al servidor semántico. Si queda una sola palabra, la pasa a mayúsculas y esa cadena es la glosa: el recuadro «1 palabra» no sale hacia el traductor. Si quedan dos o más palabras, la oración sigue por `POST /hearing`. La decisión no consulta el diccionario de poses. La pose de la glosa, venga de este atajo o del traductor, se pide después, en el recuadro de cada glosa.

5.2. Diseño de contratos de API REST

La comunicación entre el cliente local y los microservicios se rige por contratos JSON estructurados.

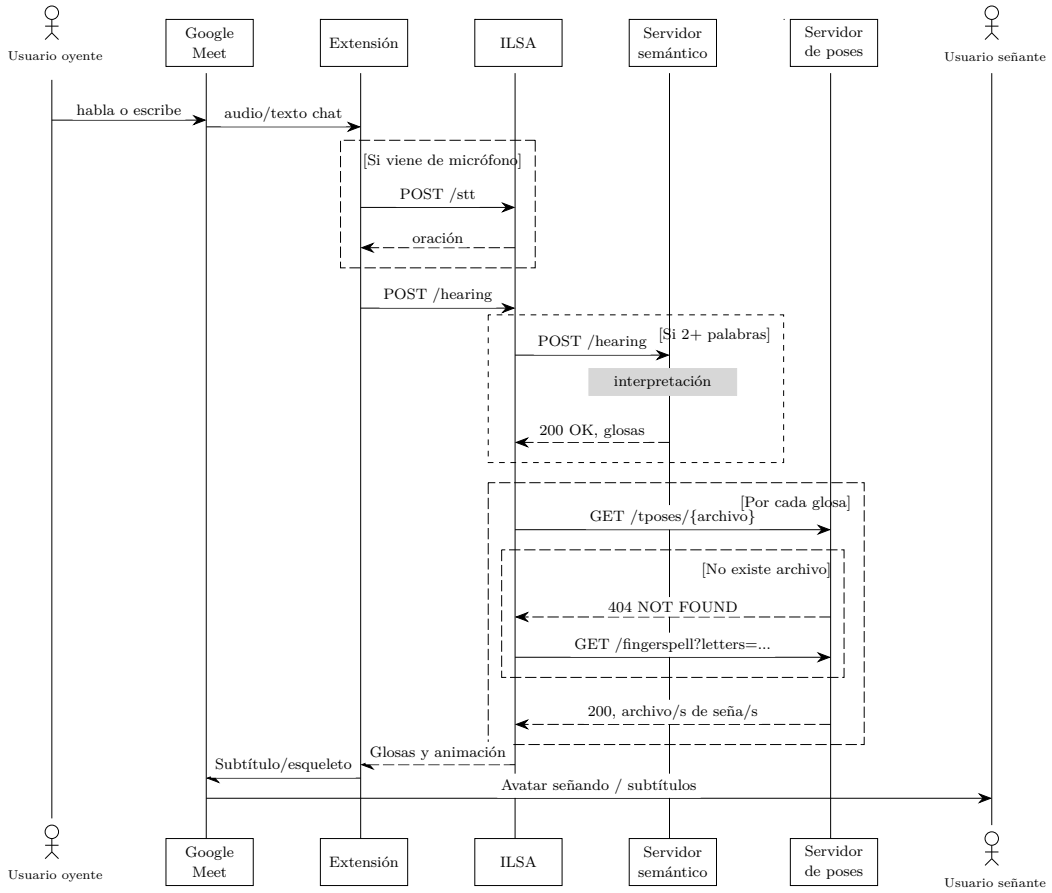


Figura 5.3: Diagrama de secuencia del canal inverso, de español a LSA.

5.2.1. API local del motor ILSA

El motor ILSA expone una interfaz HTTP en el bucle local. La extensión web actúa como cliente y el proceso local como servidor. El esquema general de componentes internos se representa en la Figura 5.4.

El Cuadro 5.1 resume las rutas expuestas por el motor local. Los cuerpos, los campos obligatorios, las unidades y los errores están en la Sección 6.7.1.

5.2.2. API del servicio semántico remoto

El microservicio remoto en la nube expone contratos REST sobre HTTPS para resolver la traducción semántica desacoplada. El Cuadro 5.2 resume los endpoints principales.

Los cuerpos, las unidades y los errores de esas rutas están en la Sección 6.7.2.

5.2.3. API del servicio de archivos del avatar

El subsistema de representación gestual de salida consulta un repositorio de archivos cinemáticos binarios. El Cuadro 5.3 detalla las rutas disponibles.

Los cuerpos, las unidades y los errores de esas rutas están en la Sección 6.7.3.

Cuadro 5.1: Endpoints de la API local del motor ILSA (`http://127.0.0.1:8765`).

Método	**Ruta**	**Contrato**	**Códigos**
GET	`/health`	Disponibilidad del proceso, del modo, del clasificador y del traductor remoto.	200
GET	`/config`	Umbrales de captura del modo en curso.	200, 503
GET	`/state`	Búfer de glosas, último texto y estado del traductor.	200, 503
GET	`/metrics`	Conteos y tiempos de la sesión en curso.	200
POST	`/session`	Apertura de sesión. El cuerpo fija la lateralidad mediante `left_handed`.	200, 503
POST	`/sign`	Ingesta de hasta 60 fotogramas para clasificación en modo señante.	200, 400, 503
POST	`/stt`	Transcripción de audio WAV en Base64 a texto en modo oyente.	200, 400, 502
POST	`/tts`	Síntesis de un WAV de la oración, para mezclarlo en el audio de Meet.	200, 400, 502
POST	`/hearing`	Envío de oración y recepción de animación en modo oyente.	200, 400, 503
POST	`/activity`	Reinicio del temporizador de pausa de inactividad ($\sim 4\,\mathrm{s}$).	200, 503
POST	`/utterance/end`	Cierre explícito del enunciado y disparo de traducción.	200, 503
GET	`/mediapipe/{name}`	Despacho de recursos estáticos del estimador visual.	200, 400, 404
GET	`/download/exe`	Entrega del ejecutable del motor local.	200, 404

Cuadro 5.2: Endpoints del servicio semántico remoto.

Método	Ruta	Función
GET	`/health`	Verificación de disponibilidad de los modelos de lenguaje en memoria.
POST	`/sign`	Canal directo: traducción de glosas LSA a oración en español rioplatense.
POST	`/hearing`	Canal inverso: traducción de oración en español a glosas en estructura *Topic-Comment*.

Cuadro 5.3: Endpoints del servicio de archivos del avatar.

Método	Ruta	Función
GET	`/health`	Disponibilidad del proceso de archivos.
GET	`/tposes/{glosa}.pose`	Entrega del flujo binario cinemático correspondiente a la glosa solicitada.
GET	`/fingerspell`	Solicitud de clips del alfabeto manual para deletreo dactilológico.

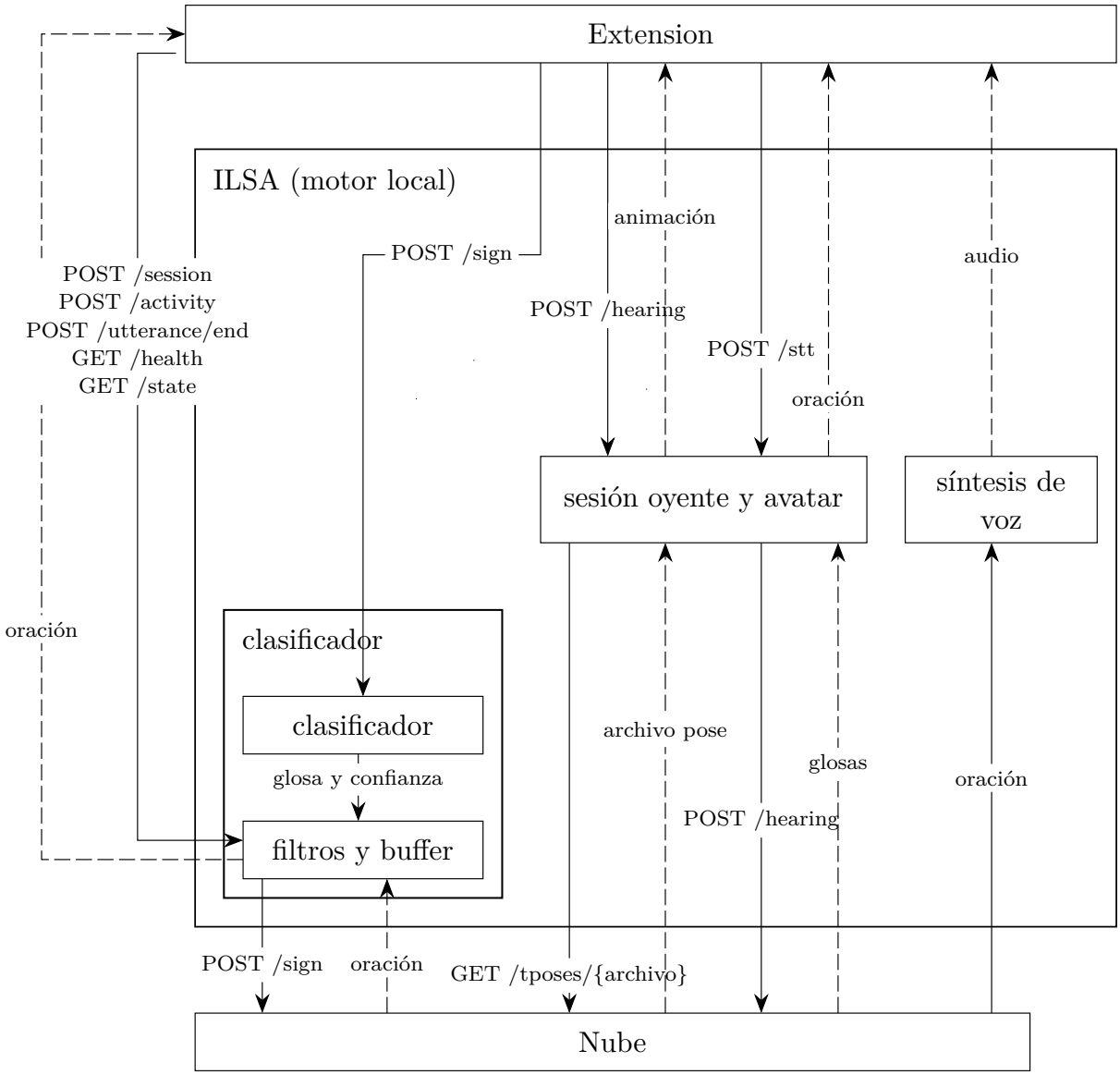


Figura 5.4: Arquitectura detallada del motor local ILSA y su interacción con los servicios en la nube.

5.3. Diseño de la representación gestual inversa

El canal inverso procesa la lista de glosas devuelta por el traductor e infiere la secuencia cinemática correspondiente. Cada glosa se asocia a un clip de referencias cinemáticas previamente parametrizado.

5.3.1. Procesamiento geométrico de coordenadas y segmentación temporal

Los clips cinemáticos registran la secuencia de puntos clave extraídos mediante la ejecución previa e independiente de *MediaPipe Holistic* sobre el video de un informante calificado. Las coordenadas planares (x, y) se representan en el espacio discreto de píxeles de la cámara de captura ($x \in [0, W]$, $y \in [0, H]$) y preservan las proporciones nativas. La

componente de profundidad z posee carácter relativo con origen en la muñeca y actúa como métrica para resolver oclusiones mutuas. Por consiguiente, la proyección gráfica reproyecta x e y directamente sobre el plano bidimensional.

El archivo binario `.pose` conserva las tres fases fonológicas estándar (preparación, núcleo, retracción). Para aislar el tramo con valor morfosintáctico distintivo sin alterar el archivo original, el sistema registra los marcadores temporales de inicio (t_{inicio}) y fin (t_{fin}) del núcleo en un índice de anotaciones externo. Durante la renderización, la lectura consulta este índice para aislar el segmento relevante.

5.3.2. Preparación, acondicionamiento y ensamblado cinemático

El servicio de poses entrega el archivo `.pose` que corresponde a la glosa pedida. Esa preparación corre en ILSA, después de la descarga. La extensión recibe la lista de fotogramas ya armada y la dibuja; no rehace el empalme.

ILSA aplica entonces estas operaciones sobre cada archivo recibido:

1. **Normalización temporal:** Conversión de los clips a una cadencia común de 30 fotogramas por segundo.
2. **Aislamiento del núcleo:** Recorte del tramo delimitado por t_{inicio} y t_{fin}.
3. **Interpolación de faltantes:** Relleno de ausencias puntuales mediante interpolación entre fotogramas válidos adyacentes.
4. **Ajuste de extremidad inactiva:** Reposicionamiento de la mano no participante hacia el costado del torso.
5. **Filtro de suavizado:** Aplicación de un promedio exponencial para atenuar la oscilación entre fotogramas.
6. **Ajuste de escala:** Normalización del tamaño corporal en función de una distancia inter-hombros fija.

Antes de la reproducción de los clips, para evitar discontinuidades espaciales abruptas al empalmar el núcleo del gesto k con el núcleo $k+1$, ILSA interpola un *puente sintético* de fotogramas de transición:

$$S(x) = 3x^2 - 2x^3 \quad \text{para } x \in [0, 1] \tag{5.1}$$

Este puente ejecuta una traslación suave sobre las coordenadas planares. Se explicita que este procedimiento constituye un ajuste gráfico e interpolatorio, distinto del fenómeno biomecánico natural de la coarticulación gestual. El Cuadro 5.4 sintetiza las previsiones de diseño ante contingencias en la representación.

Cuadro 5.4: Previsiones del diseño frente a fallos de estimación o léxico.

Situación	Previsión
Puntos nulos puntuales en el estimador.	Interpolación lineal a partir de fotogramas válidos vecinos.
Inexistencia de archivo para una glosa.	Activación del subsistema de deletreo manual dactilológico.
Secuencias extensas (más de 24 clips o más de 1200 cuadros).	El 24 cuenta los clips que se dibujan, no las glosas de entrada. Si una glosa no tiene archivo, cada letra del deletreo es un clip: un nombre de cinco letras ocupa cinco. Al llegar a 24, el resto queda en texto. Los 1200 cuadros cortan el bloque ya empalmado.

5.3.3. Estrategia de contingencia léxica

Ante palabras ausentes en el vocabulario grabado (nombres propios o términos técnicos), el sistema aplica un mecanismo de descomposición dactilológica. La palabra se segmenta en sus letras constituyentes y se sintetiza mediante la secuencia correspondiente de caracteres del alfabeto dactilológico al consultar el endpoint específico `/fingerspell`. Esos clips de letra entran al tope de 24 de la fila anterior: una sola glosa puede consumir varios de esos 24.

5.4. Diseño de la extensión de videollamada

La integración con la plataforma Google Meet se resuelve mediante una extensión de navegador que intercepta la señal de video y gestiona la superposición de capas gráficas. La Figura 5.5 detalla la distribución funcional de sus componentes.

5.5. Patrones de diseño de *software* aplicados

El diseño incorpora patrones de *software* consolidados para organizar las responsabilidades del sistema:

- **Estrategia (*Strategy*):** Aplicado en el módulo de filtrado de repeticiones. `RepeatGate` define políticas diferenciadas según la clase léxica: los dígitos admiten repetición continua, las letras aceptan hasta dos instancias consecutivas y el vocabulario general descarta duplicados inmediatos.
- **Fachada (*Facade*):** Aplicado en la API HTTP del motor ILSA local. Expone una interfaz unificada que oculta la complejidad interna de extracción, clasificación y comunicación con la nube.
- **Adaptador (*Adapter*):** Implementado en la extensión web y en los conectores de red para transformar los formatos nativos del navegador a los contratos JSON requeridos por los servicios REST.
- **Estado (*State*):** Gestiona la modalidad operativa de la sesión (Señante u Oyente) y altera dinámicamente el comportamiento de los endpoints sin requerir instancias de servicio separadas.

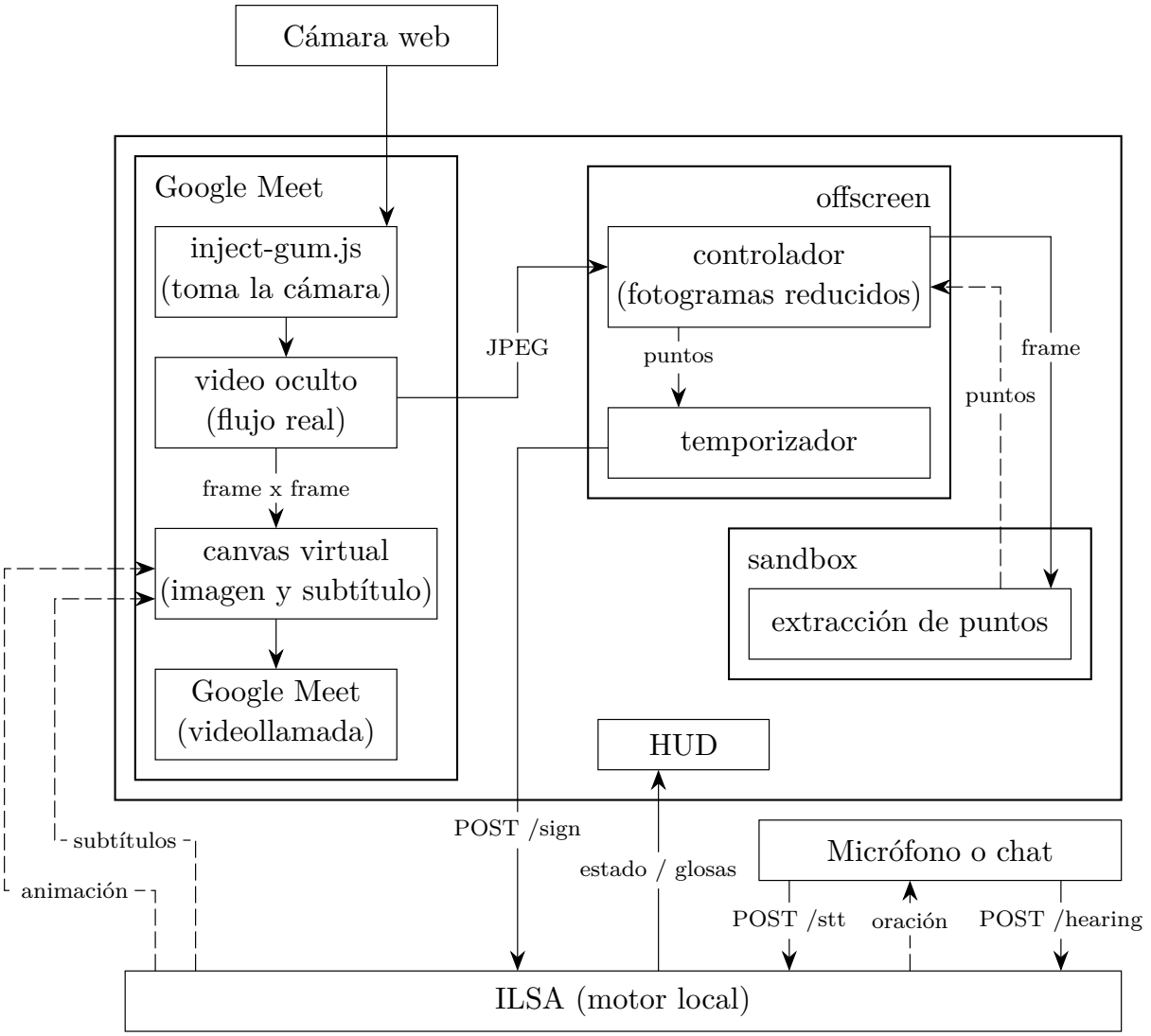


Figura 5.5: Diagrama de arquitectura del flujo de video, procesamiento offscreen e integración con el motor ILSA.

5.6. Diseño conceptual de la arquitectura escalable de producción en la nube

Las pruebas de este trabajo usan una instancia de Cloud Run por servicio. Esta sección no describe esa instancia. Describe la proyección para varias videollamadas a la vez, en coincidencia con la restricción RC-02.

El diagrama, en hoja horizontal, está en el Apéndice C (Figura C.1).

La proyección tiene dos servicios detrás de un mismo ingreso. El traductor genera texto. El servicio de archivos entrega un clip de seña. No comparten réplica ni cola.

5.6.1. Ingreso

Las computadoras cliente salen por HTTPS. El ingreso resuelve el nombre, presenta el certificado TLS y filtra tráfico elemental. Detrás opera un balanceador.

- `POST /sign` y `POST /hearing` van al traductor. El criterio es el menor número de conexiones activas, para no cargar una réplica que ya genera un enunciado largo.
- `GET /tposes/{glosa}.pose`, `GET /fingerspell` y la salud de ese servicio van a las réplicas de archivos.

5.6.2. Traductor

El traductor viaja en una imagen de contenedor. Cada réplica monta los binarios GGUF Q4_K_M desde un volumen de solo lectura, así un arranque nuevo no vuelve a bajar los modelos. Un escalador horizontal mira la cola y el tiempo de generación. Si los turnos simultáneos superan a las réplicas activas, el orquestador abre contenedores nuevos. Cada réplica corre en procesador, sin placa de video. Carga el modelo principal de los dos canales, Qwen2.5 1,5B, con un contexto de 2048 *tokens*, y deja el respaldo Llama 3.2 1B en disco hasta que ese principal no carga. La estimación de menos de 2 GiB por réplica es la suma de esos dos archivos y de sus cachés de claves y valores, calculada en la Sección 7.3.3. El buffer de activaciones, definido junto a la Ecuación 6.6, queda fuera de esa suma. Los 8 GiB de la Sección 6.6 son la memoria asignada a la instancia de validación, no una medición de este consumo.

5.6.3. Archivos de seña

Una petición de pose es una lectura, no una generación. El contrato es el de la Sección 5.2.3. Las réplicas no guardan estado y leen la misma carpeta. El escalador de este grupo mira la cantidad de lecturas, no la cola del traductor. La ruta de salud queda aparte de la ruta del archivo. La caché del cliente sigue como primer recurso.

5.6.4. Monitoreo

Un recolector de métricas observa la latencia por oración, la memoria de cada contenedor del traductor, los errores de red y los errores de lectura del servicio de archivos. Esas series ajustan los dos escaladores.

5.6.5. Qué queda fuera de las pruebas

El prototipo no crea el balanceador, ni los escaladores, ni el volumen compartido. ILSA consulta una instancia del traductor, con los dos canales en un solo proceso, y una instancia del servicio de archivos. Esas dos publicaciones se describen en la Sección 6.6. La carga concurrente y el auto-escalado no entran en la validación.

Capítulo 6

Implementación y Desarrollo

6.1. Estructura del código

La arquitectura del sistema se organiza de forma modular para facilitar la escalabilidad, el mantenimiento y la separación de responsabilidades entre los componentes. El proyecto utiliza una arquitectura distribuida basada en microservicios y clientes livianos. La lógica se divide en dos bloques principales: un backend central encargado del procesamiento con inteligencia artificial, renderizado y comunicación en tiempo real, e interfaces de cliente compuestas por una extensión de navegador web y un empaquetador nativo.

6.1.1. Organización del Repositorio

El repositorio fuente contiene la totalidad de los módulos del sistema, recursos gráficos, archivos de configuración de entorno e instrucciones de despliegue. A continuación, se presenta la distribución lógica de directorios dentro de la rama principal del proyecto, la rama `main`. El commit es https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/5974ac79d41ec6738e554354aad44ba963e5f412:

```
.
|-- assets/              # Gráficos de la aplicación
|-- extension/           # Extensión web para Chromium
|   |-- content/         # Scripts inyectados en Google Meet
|   |-- lib/             # Captura de video y MediaPipe
|   |-- vendor/          # MediaPipe Wasm de terceros
|   |-- background.js    # Service Worker de mensajería
|   |-- manifest.json    # Permisos y manifiesto V3
|   `-- offscreen.js     # Proceso fuera de la pestaña
|-- packaging/           # Compilación del ejecutable
|-- scripts/             # Evaluación y latencia
|-- src/                 # Backend en Python
|   |-- avatar/          # Animación del avatar
|   |-- backend/         # API FastAPI y Uvicorn
|   `-- classifier/      # Inferencia de señas
|-- LSABackend.bat       # Arranque en Windows
|-- requirements.txt     # Dependencias de Python
`-- run_backend.py       # Entrada del servidor
```

6.1.2. Descripción de Componentes y Responsabilidades

Cada directorio y paquete del sistema cumple una función específica dentro del flujo de inferencia y visualización:

- **Núcleo del Backend (`src/backend/`)**: Implementa la API REST mediante FastAPI y Uvicorn. Administra las sesiones concurrentes para usuarios oyentes y señantes y coordina el flujo continuo de datos de landmarks y la devolución de traducciones o poses.
- **Módulo Clasificador de Señas (`src/classifier/`)**: Contiene la arquitectura de la red neuronal en PyTorch (`arch.py`) y los archivos de pesos entrenados (`tinyskeleton_best.pth`).
- **Motor de Avatar esquelético (`src/avatar/`)**: Administra la lógica de animación y comunicación remota con servicios de representación gráfica. Traduce texto o *tokens* interpretados en secuencias de animación esquelética para su renderizado.
- **Extensión de Navegador (`extension/`)**: Actúa como la interfaz de usuario en tiempo real sobre Google Meet. Utiliza un *Service Worker* (`background.js`) para la gestión del estado, un entorno *offscreen* (`offscreen.js`) para procesar el flujo de video sin ralentizar la pestaña activa, y scripts de contenido (`content/meet.js`) para modificar la interfaz del usuario e integrar la ventana de interpretación.
- **Empaquetado y Distribución (`packaging/`)**: Contiene las configuraciones para PyInstaller (`LSABackend.spec`) y scripts automatizados en sintaxis Batch y Python. Permite generar la versión ejecutable del servidor sin requerir la instalación previa de un entorno Python en la máquina del usuario final.
- **Herramientas de Benchmarking (`scripts/`)**: Proporciona utilidades como `bench_latency.py` para medir la latencia punto a punto de la red neuronal y del canal de transmisión de datos.
- **Instrumentación de latencia:** El motor ILSA registra, por emisión, los cinco relojes de la Sección 3.1.3 (pausa, cola, inferencia, transporte y entrega) en CSV y en JSONL. Esa traza alimenta la verificación de RNF-01 en la Sección 7.3.1.

6.1.3. Convenciones de Nomenclatura y Estándares

El desarrollo adopta convenciones estándares según el lenguaje y entorno de ejecución:

- **Python**: Se aplican las directrices de estilo PEP 8 [41].. Los nombres de módulos, paquetes y funciones utilizan `snake_case`, mientras que las clases se definen en `PascalCase`. Las constantes globales se expresan en `UPPER_SNAKE_CASE`.
- **JavaScript/HTML/CSS**: Los scripts de la extensión siguen el estándar ECMAScript 6+ [42] con sintaxis `camelCase` para variables y funciones. Los archivos de hoja de estilo e interfaces DOM emplean la convención `kebab-case`.

6.1.4. Archivos de Configuración y Entorno de Pruebas

El proyecto integra múltiples mecanismos de configuración para asegurar su reproducibilidad:

- **`manifest.json`**: Define los permisos del navegador, scripts de fondo y recursos accesibles desde la Extensión Web bajo la especificación Manifest V3.
- **`requirements.txt`** y **`requirements_exe.txt`**: Especifican las librerías necesarias con sus versiones fijadas tanto para el entorno de desarrollo como para la compilación final.
- **`mapeo_clases.json`**: Archivo de configuración alojado en `src/classifier/weights/` que vincula los índices numéricos de salida de la red neuronal con las etiquetas textuales del vocabulario LSA.

El entrenamiento del clasificador, el servicio semántico en la nube, el entrenamiento de cada canal y el servicio de poses del avatar viven en ramas propias. Su función y su disposición están en el Apéndice B.

6.2. Implementación del subsistema de reconocimiento gestual

6.2.1. Preprocesamiento de datos y pipeline de extracción

El pipeline procesa los fotogramas de entrada mediante *MediaPipe Holistic* para la estructuración de un vector instantáneo de 225 coordenadas (33×3 de pose, 21×3 de mano izquierda y 21×3 de mano derecha). Como paso previo a la extracción, el sistema aplica un reencuadre con bandas de relleno a una resolución uniforme de 853×480 píxeles, con el objetivo de corregir las discrepancias en la relación de aspecto de las cámaras ($4:3$ frente a $16:9$).

A continuación, cada vector se traslada al punto medio de los hombros y se divide por la distancia existente entre ellos. En la topología de MediaPipe, el índice 11 corresponde al hombro izquierdo y el índice 12 al hombro derecho. Dichas coordenadas no se expresan en metros ni en píxeles: MediaPipe define x como la fracción del ancho de la imagen, y como la fracción del alto, y z como la profundidad asociada a cada punto. El punto medio $\mathbf{a}$ utiliza las tres componentes, mientras que la escala s contempla únicamente el plano (x, y):

$$\mathbf{a} = \frac{\mathbf{p}_{11} + \mathbf{p}_{12}}{2}, \qquad s = \sqrt{(x_{11} - x_{12})^2 + (y_{11} - y_{12})^2}. \tag{6.1}$$

$$\mathbf{p}' = \frac{\mathbf{p} - \mathbf{a}}{s}. \tag{6.2}$$

Tras esta transformación, la coordenada $\mathbf{p}'$ se expresa en unidades de dicha distancia interhumeral, donde el valor 1 equivale a un ancho de hombros en el plano (x, y) ya normalizado respecto a la imagen. La componente de profundidad z también se traslada con el punto medio y se divide por s.

El algoritmo contempla diversos casos límite durante la ejecución:

- Si la lista de puntos de pose no está disponible, el punto medio se fija en el origen y la escala s adopta el valor 1.
- Durante el armado del vector de entrenamiento, cualquier lista que contenga menos de 13 puntos recibe este mismo tratamiento.
- Los bloques compuestos íntegramente por ceros se conservan sin cambios y no se someten a la división.
- Si la escala s resulta menor que 10^{-5}, el sistema la reemplaza por 1; de este modo, los puntos se trasladan pero no se dividen por un valor indeterminado o cercano a cero.
- El cálculo se ejecuta de forma directa sin consultar la visibilidad de los hombros: basta con la presencia de los índices 11 y 12 en la lista para su inclusión en la fórmula.

6.2.2. Arquitectura neuronal TinySkeletonClassifier

La clasificación de la secuencia temporal de 16 fotogramas se ejecuta mediante el modelo *TinySkeletonClassifier*, cuya implementación comprende:

1. Dos capas de convolución unidimensional temporal (núcleo de tamaño 3, 128 canales, normalización por lotes, activación ReLU y regularización Dropout de 0,2).
2. Un módulo de codificación posicional sinusoidal sumado a la representación de 128 dimensiones.
3. Un codificador Transformer de 2 capas con 4 cabezas de atención.
4. Una capa de agregación por atención (*Attention Pooling*). Una capa lineal aprendida proyecta cada vector $\mathbf{h}_t$, de 128 componentes, a un escalar. El vector de pesos $\mathbf{w}$ y el sesgo b salen del entrenamiento:

   $$s_t = \mathbf{w}^\top \mathbf{h}_t + b. \tag{6.3}$$

   Una softmax sobre los T fotogramas de esa secuencia convierte los escalares en pesos. Esos pesos son positivos y suman 1:

   $$a_t = \frac{e^{s_t}}{\sum_{k=1}^{T} e^{s_k}}. \tag{6.4}$$

   El vector de 128 dimensiones es la suma de los fotogramas, cada uno multiplicado por su peso:

   $$\mathbf{v} = \sum_{t=1}^{T} a_t \mathbf{h}_t. \tag{6.5}$$

5. Una capa densa lineal final hacia las 100 clases del vocabulario gestual.

La Figura 6.1 resume esa sucesión y la forma del tensor en cada etapa.

El *corpus* se conforma por las grabaciones de tres personas (dos hombres y una mujer) distribuidas en seis sesiones. El conjunto total abarca 100 clases con un aporte de 100 clips por clase. Para cada clase, los clips se aleatorizan mediante una semilla fija (42) y se dividen en una proporción de 80 clips para entrenamiento y 20 para validación. De este



Figura 6.1: Forma de los tensores en *TinySkeletonClassifier*.

modo, sobre el total de 100 clases, el conjunto de entrenamiento reúne 8 000 clips y el de validación 2 000 clips. Esta partición se realiza a nivel de clip individual, por lo que las tres personas pueden estar presentes en ambos conjuntos. El punto de control (*checkpoint*) seleccionado alcanzó una exactitud de validación fuera de línea del 99,0 %. Esta métrica evalúa clips excluidos del ajuste que pertenecen a los mismos informantes del conjunto de entrenamiento. La evaluación con cámara en vivo para este mismo archivo se detalla en la Sección 7.2.1.

Las técnicas de aumento de datos (*data augmentation*) se aplican de manera exclusiva durante la lectura de los clips de entrenamiento. Cada clip entra 25 veces en cada época y recibe transformaciones con parámetros estocásticos nuevos en cada iteración. El conjunto de validación no experimenta ningún tipo de alteración.

En la dimensión temporal, la velocidad del gesto se selecciona de manera aleatoria en un rango de 0,80 a 1,25. Para factores superiores a 1, la duración de la seña se reduce y el espacio sobrante se rellena con la repetición de los fotogramas extremos. Para factores inferiores a 1, la seña se extiende y el sistema conserva una ventana de observación. Asimismo, se puede descartar hasta un 15 % de los fotogramas en los extremos inicial y final de la secuencia, debido a que el comienzo y la conclusión de la seña presentan variaciones en la ventana. Posteriormente, la secuencia se reajusta a una longitud fija de 16 fotogramas. Con el objetivo de simular las pérdidas accidentales en la detección de *MediaPipe*, se anulan hasta 3 fotogramas (se fijan en cero) y se reconstruyen mediante interpolación lineal.

En la dimensión espacial, cada secuencia recibe una rotación aleatoria compuesta por un giro lateral de hasta 15°, un ajuste de perspectiva vertical (cámara alta o baja) de hasta 8° e una inclinación de hasta 8°. Se aplica un factor de escala comprendido entre 0,85 y 1,15 para modificar la amplitud del esqueleto con respecto al origen centrado en los hombros. Finalmente, se añade ruido gaussiano con una desviación estándar de 0,012 para los puntos de la pose y de 0,027 para los puntos de las manos, expresado en las unidades del vector previamente normalizado. El nivel de ruido introducido en las manos es superior para compensar la menor estabilidad en el seguimiento de estas articulaciones.

El modelo se optimizó mediante el algoritmo AdamW, con una tasa de aprendizaje inicial de $3{,}24 \times 10^{-5}$ y un decaimiento de pesos (*weight decay*) de 0,0145. Las muestras

se agruparon en lotes de 32 secuencias. La función de pérdida empleada fue la entropía cruzada con un suavizado de etiquetas (*label smoothing*) de $2{,}54 \times 10^{-4}$. Se configuró un esquema de reducción de la tasa de aprendizaje a la mitad tras 5 épocas consecutivas sin mejoras en la pérdida de validación. El límite máximo de entrenamiento se fijó en 200 épocas, con una estrategia de parada temprana (*early stopping*) que interrumpe el proceso si la pérdida de validación no disminuye durante 15 épocas. El sistema conserva los pesos correspondientes a la época con el menor valor de pérdida de validación.

La corrida de entrenamiento finalizó tras 35 épocas. La menor pérdida de validación registrada fue de 0,028, con una exactitud del 99,0 % sobre los 2 000 clips de validación. La Figura 6.2 ilustra el comportamiento de las curvas de entrenamiento y validación. Por su parte, la Figura 6.3 presenta la matriz de confusión correspondiente sobre esos mismos clips, donde cada clase cuenta con un aporte de 20 muestras y la escala alcanza un límite de 20.

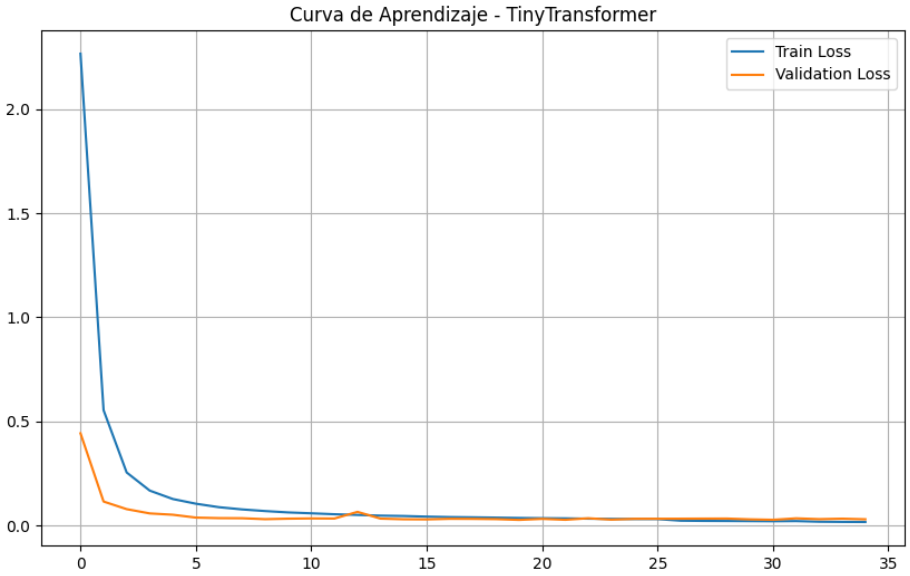


Figura 6.2: Pérdida de entrenamiento y de validación. El eje horizontal es la época.

6.3. Implementación del subsistema de traducción semántico bidireccional

6.3.1. Selección y especificación de la suite de modelos de lenguaje

Para la traducción semántica bidireccional se adaptaron modelos de lenguaje pequeños de entre 0,5B y 3B parámetros:

- **Familia Qwen2.5:** Variantes de instrucciones de 0,5B, 1,5B y 3B parámetros.
- **Familia Llama 3.2:** Variante de instrucciones de 1B parámetros.
- **Familia SmolLM2:** Variante de instrucciones de 1,7B parámetros.

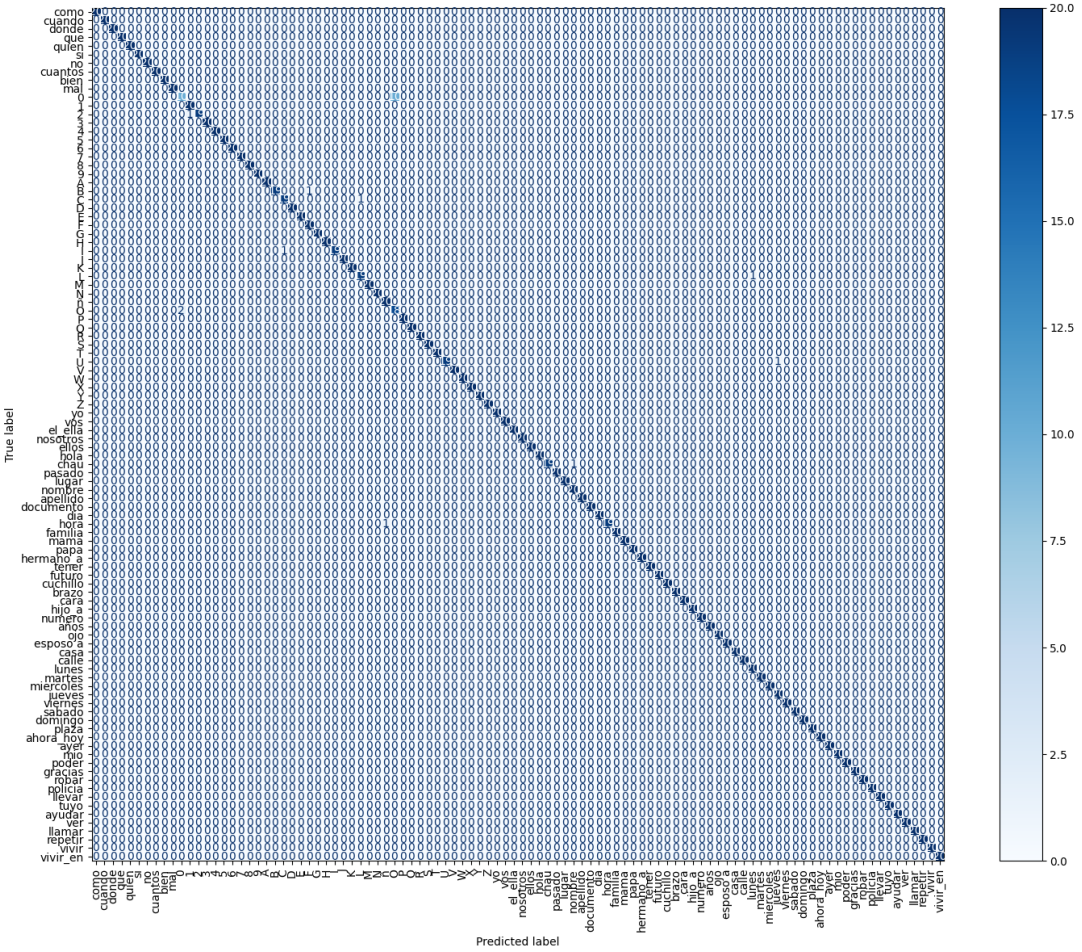


Figura 6.3: Matriz de confusión de los 2 000 clips de validación. Cada clase tiene 20 clips.

En el servidor, cada canal arranca con Qwen2.5 de 1,5B y, si ese archivo no está disponible, con Llama 3.2 de 1B. El inventario de esas rutas está en la Sección 5.2.2. Los cuerpos, las unidades y los errores están en la Sección 6.7.2.

6.3.2. Régimen de entrenamiento y ajuste LoRA

El canal directo y el canal inverso comparten el mismo procedimiento de entrenamiento. El *corpus* se divide en un 85 % para entrenamiento y un 15 % para evaluación. Esta separación utiliza la semilla 3407, la misma del entrenador, para que cualquier prueba se repita en las mismas condiciones. Cada ejemplo del conjunto de datos representa una interacción de tres partes: las instrucciones iniciales del sistema (*prompt*), el mensaje del usuario y la respuesta esperada. La ventana de procesamiento admite hasta 2048 *tokens*.

Para adaptar el modelo a la lengua de señas sin modificar sus miles de millones de parámetros originales, se utiliza la técnica LoRA. Se eligió un rango de adaptación $r = 16$ y un factor de escala $\alpha = 32$. La relación $\alpha/r = 2$. Estos valores responden a la configuración recomendada para modelos de este volumen.

Las matrices que reciben entrenamiento son las cuatro capas de proyección de atención y las tres capas de la red de avance. En cambio, la tabla de *tokens* de entrada y la capa de salida permanecen fijas. Esta elección se debe a que el vocabulario base del modelo no

cambia; la tarea solo necesita reorganizar y conectar palabras que la red ya conoce.

El tamaño de lote efectivo se fijó en 16 ejemplos por cada paso del optimizador. Dado que el modelo de 3B parámetros consume más memoria, su micro-lote se redujo a 2 ejemplos y los gradientes se acumulan durante 8 pasos. Los modelos más pequeños utilizan micro-lotes de 4 ejemplos y acumulan durante 4 pasos. Ese producto deja en 16 la cantidad de ejemplos de cada paso. La actualización de los pesos depende de la arquitectura de cada modelo y de su tasa de aprendizaje.

La tasa de aprendizaje (*learning rate*) se fijó en 3×10^{-4} para el modelo de 3B y en 5×10^{-4} para las versiones de menor tamaño. Se asignó un valor inferior al modelo de mayor escala debido a que las actualizaciones abruptas en redes con una cantidad superior de capas introducen inestabilidad durante el proceso de aprendizaje.

El optimizador seleccionado es AdamW con cuantización a 8 bits. Esta configuración optimiza el consumo de memoria VRAM en la GPU, en tanto que los estados del optimizador se almacenan en 8 bits y coexisten de manera eficiente con el modelo base cuantizado a 4 bits. Asimismo, el decaimiento de pesos (*weight decay*) de 0,01 previene el crecimiento excesivo de los adaptadores para evitar el sobreajuste (*overfitting*) de los datos.

El esquema contempla un período de calentamiento (*warmup*) de 10 pasos, intervalo en el cual la tasa de aprendizaje se incrementa de forma progresiva desde cero. En cuanto a los adaptadores de bajo rango, la combinación entre la inicialización uniforme de la matriz $\mathbf{A}$ y la matriz $\mathbf{B}$ en cero asegura que el producto $\mathbf{BA}$ expresado en la Ecuación 3.4 resulte nulo en el paso inicial, lo que garantiza que la salida coincida con la del modelo base congelado. Posteriormente, un planificador de tipo coseno (*cosine scheduler*) atenúa de forma gradual la tasa de aprendizaje hacia la fase final del entrenamiento con el propósito de refinar los ajustes residuales.

Para acelerar el entrenamiento y no perder capacidad de cómputo, la función de pérdida no evalúa todo el diálogo. Una máscara excluye del cálculo directo de la pérdida las posiciones del *prompt* y de la entrada, y deja el error sobre la respuesta deseada. La respuesta sigue condicionada por esos *tokens* durante el entrenamiento. De este modo, la red concentra su aprendizaje en la traducción.

Por último, antes de ingresar al entrenamiento, los datos reciben una limpieza específica. Las letras sueltas y consecutivas del dataset se unen en un solo *token* con mayúscula inicial, y los dígitos sueltos forman un número consolidado. Tres letras idénticas seguidas se reducen a dos para corregir errores de tipeo o captura. Las secuencias de palabras sin separación previa, como *Mar del Plata*, se agrupan en un único *token Mardelplata* para que la falta de espacios no confunda al modelo durante el aprendizaje.

Procedimiento de evaluación y post-entrenamiento

Cada canal entrena los cinco modelos durante 5 épocas. El argumento `max_steps` queda en -1, y el entrenamiento corre esas épocas. El canal directo ejecuta 210 pasos del optimizador y el inverso ejecuta 180. La semilla del corte y la del entrenador es 3407. Las versiones cargadas son:

- `unsloth/Qwen2.5-0.5B-Instruct`
- `unsloth/Qwen2.5-1.5B-Instruct`
- `unsloth/Qwen2.5-3B-Instruct`
- `unsloth/Llama-3.2-1B-Instruct`

- `unsloth/SmolLM2-1.7B-Instruct`

El 85 % de las filas entrena cada modelo. El 15 % restante es el único corte reservado: 119 filas en el canal directo y 102 en el inverso. La comparación entre los cinco modelos y los resultados de los Cuadros 7.9 y 7.10 salen de esas mismas filas. La elección de Qwen2.5 1,5B y el informe de ese modelo comparten ese corte.

Al cierre de cada entrenamiento, las filas reservadas se evalúan sobre el binario GGUF cuantizado a 4 bits. Esta prueba mide el archivo que después sirve el modelo.

La evaluación configura los siguientes parámetros generales de generación y medición:

- **Determinismo:** Una temperatura de valor 0 anula la aleatoriedad en la generación de texto y fuerza al modelo a elegir siempre el *token* de mayor probabilidad matemática.
- **Control de repetición y parada:** Se aplica una penalización de repetición de 1,05 para desincentivar que el modelo entre en bucles de palabras o glosas idénticas durante la inferencia. El proceso se detiene ante la aparición de un salto de línea o de la marca de fin de turno, lo que evita el consumo innecesario de cómputo.
- **Múltiples referencias:** Cada entrada acepta las respuestas que el *corpus* completo asocia a esa entrada. En el canal directo la clave es la glosa. De las 119 filas de evaluación, 89 tienen más de una oración española en el *corpus*, y 85 usan una glosa que el corte también dejó en el entrenamiento. En el canal inverso la clave es la oración. En este archivo cada oración aparece una sola vez, y las 102 filas reservadas quedan fuera del entrenamiento. Aceptar varias respuestas del *corpus* completo deja en el entrenamiento una entrada que el corte reparte entre los dos lados.
- **Métricas de rendimiento:** La prueba registra el acierto exacto, el acierto tras normalizar puntuación y mayúsculas, y cinco métricas de texto: BLEU, ROUGE-L, METEOR, `BERT Score` y `Comet Score`. BLEU cuenta n-gramas compartidos con la referencia. ROUGE-L cuenta la subsecuencia común más larga. METEOR alinea palabras, raíces y sinónimos. `BERT Score` mide la cercanía de los vectores de los *tokens*. `Comet Score` estima la calidad de la traducción a partir de la entrada, la salida y la referencia. Van juntas porque el solapamiento de palabras no alcanza para decidir si se conservó el sentido. El detalle de cada una está en la Sección 3.3. La latencia se registra aparte, como tiempo de respuesta.

6.3.3. Canal directo (LSA → Español)

El par de este canal es glosas hacia una oración en español rioplatense. La entrada del usuario antepone «Glosas:» a la secuencia ya compacta. La consigna del ajuste pide una sola oración, sin notas ni datos ajenos a la entrada:

- El español es rioplatense y usa voseo. La oración incorpora artículos, preposiciones y concordancia verbal. La traducción no es palabra por palabra.
- Los pronombres de entrada son `YO`, `VOS`, `EL`, `NOSOTROS` y `ELLOS`. Sin una marca de género al lado, el sujeto queda tácito.
- La letra `A` inmediata al sustantivo o al pronombre marca femenino, y la letra `O` marca masculino. Un parentesco sin esa letra se considera masculino.
- `MIO` y `TUYO` expresan posesivos del español.

- `PASADO` o `AYER` exigen pretérito. `FUTURO` exige tiempo futuro. Sin marca temporal, el verbo queda en presente.
- `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` y `CUANTOS` producen una pregunta directa, con signos de apertura y de cierre.
- `NO` junto al concepto o al verbo produce una negación.
- Una palabra de la secuencia en capitalización de nombre, o una cifra, se copia tal cual en la oración. La consigna prohíbe sustituirla o alterarla.
- La entrada sigue el orden *Topic-Comment* (tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y pregunta). La oración de salida reordena esa secuencia según la sintaxis del español.

Para la evaluación post-entrenamiento de este canal, se establece un límite máximo de 24 *tokens* del modelo para la salida (Sección 3.1.6). Esta restricción responde a que las oraciones traducidas del vocabulario base son breves.

6.3.4. Canal inverso (Español → LSA)

El par invierte al canal directo: la entrada es la oración y la respuesta son las glosas. La entrada antepone «Español:». La máscara deja el error sobre las glosas. Pide una sola secuencia, separada por espacios, sin notas, sin el prefijo «Glosas:» y sin artículos ni preposiciones del español:

- Las glosas léxicas van en mayúsculas sostenidas. Un nombre, un apellido o una calle va en capitalización de nombre, como `Patricia` o `Moreno`, y es una sola palabra de la secuencia, nunca letra por letra. Una cifra, como un documento o una cantidad, también es una sola palabra, no un dígito por espacio.
- El orden es tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y, al cierre, la partícula interrogativa o la negación.
- Los pronombres se explicitan aunque el español los omita, se asume masculino en caso de que no esté especificado. 'Tengo un cuchillo.' → `YO CUCHILLO TENER`. El mapeo es yo → `YO`, vos o tú → `VOS`, él → `EL O`, ella → `EL A`, nosotros → `NOSOTROS O`, nosotras → `NOSOTROS A`, ellos → `ELLOS O` y ellas → `ELLOS A`.
- El género del parentesco usa el sustantivo en masculino más `O` o `A`: hermana → `HERMANO A`, hija → `HIJO A`, esposa → `ESPOSO A`. Un sustantivo de persona sin marca de género queda en masculino y suma `O` si el español lo explicita.
- `MIO` y `TUYO` cubren los posesivos. 'Me llamo X' o 'Mi nombre es X' → `NOMBRE MIO` más el nombre.
- El pretérito o el copretérito colocan `PASADO` al inicio. El futuro, o la perífrasis 'voy a', coloca `FUTURO`. El presente sin marca no agrega una glosa temporal.
- Las preguntas de dónde, quién, cómo, qué, cuándo y cuántos dejan `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` o `CUANTOS` al final. La negación deja `NO` al final, junto al verbo.

Mediante estas reglas, el modelo aprende a emitir un nombre propio o una cifra como una sola palabra de la secuencia, y no como una cadena de letras. En la evaluación, el límite de generación de este canal es de 48 *tokens* del modelo, frente a 24 en el canal directo (Sección 3.1.6). Ese número cuenta piezas del tokenizador. Un nombre o una cifra larga es una palabra de la secuencia y, aun así, el tokenizador puede partirla en varias de esas piezas. El deletreo letra por letra ocurre después: si el avatar no encuentra el archivo de esa palabra, pide un clip por letra, con otro tope, de 24 clips.

6.3.5. Cuantización GGUF Q4_K_M y estimación de memoria

Los modelos ajustados se fusionaron y convirtieron a binarios GGUF con cuantización `Q4_K_M`. La memoria RAM requerida durante la inferencia se calcula mediante la relación:

$$\text{RAM} \approx \mathbf{W} + KV_{\text{fp16}}(ctx = 2048) + \text{Buffer} \tag{6.6}$$

donde $\mathbf{W}$ representa el tamaño de los tensores cuantizados y KV_{fp16} la caché de claves y valores para 2048 *tokens* de contexto [43]. El buffer de activaciones es la memoria temporal de un paso de generación [44]. Ahí entran el estado oculto de cada capa, las puntuaciones de atención de ese paso y el resultado intermedio de la red de avance. El paso siguiente reutiliza ese mismo espacio. La caché KV conserva, para el contexto, las claves y los valores ya calculados. La ecuación deja $\mathbf{W}$ en 4 bits por parámetro, esto es $\sim 0{,}5$ bytes, y cuenta solo el entero de cada peso. En la práctica el archivo GGUF pesa más: cada bloque guarda también la escala y el mínimo [15], `v_proj` y `down_proj` quedan en `Q6_K`, las normalizaciones siguen en 32 bits, y el archivo incluye la cabecera y el vocabulario. La tabla de `llama.cpp` no separa esas partes. Publica un solo promedio, el tamaño del GGUF dividido por la cantidad de parámetros, medido sobre Llama 3.1 8B: para `Q4_K_M` da 4,8944 bits por peso y un archivo de 4,58 GiB. En un modelo de ese tamaño la cabecera y el vocabulario casi no mueven el promedio. En los cinco modelos de este trabajo el cociente es otro: los archivos y la memoria que resulta de sumarles la caché están en los Cuadros 7.15 y 7.16.

6.4. Integración en Google Meet y arquitectura de la extensión

6.4.1. Secuencia completa de inferencia y comunicación multidireccional

El motor local de la plataforma determina el modo operativo antes de cualquier interacción con la extensión de navegador. A través del panel de control, el usuario configura la lateralidad (diestro o zurdo) y el canal de comunicación (Sordo u Oyente). Al iniciar la aplicación, al Encender un modo o al Reiniciar, ILSA vuelve a la pantalla de carga y solo habilita el panel si el traductor semántico y el servidor de poses responden. Al Apagar, detiene la sesión y queda en el panel: no vuelve a la carga ni consulta esos servicios.

El proceso principal escucha peticiones en una dirección de red local. Dado que las políticas de seguridad de Google Chrome identifican esa dirección como una red interna restringida, la página de la llamada no puede comunicarse con ella de forma directa. Por este motivo, el módulo en segundo plano de la extensión opera como intermediario,

mientras que el documento secundario sin pantalla y la ventana emergente —al pertenecer al mismo origen de la extensión— entran en contacto directo con el motor local.

Durante la navegación en una llamada activa, el módulo en segundo plano efectúa una consulta inicial sobre el estado del sistema. Si el motor local se encuentra encendido, la extensión inyecta herramientas en dos entornos aislados del navegador:

- **Entorno de soporte:** Gestiona la interfaz flotante, las preferencias del usuario y el envío de solicitudes de red.
- **Entorno de la página principal:** Encargado de la renderización en pantalla y de tomar el control de la transmisión de la cámara web.

El sistema reitera la validación de estado cada dos segundos. Si el traductor no se encuentra listo, la interfaz solicita el reinicio del servicio y detiene el flujo. La modalidad seleccionada determina el canal activo, con la restricción de que el modo Sordo y el modo Oyente nunca se ejecutan en simultáneo.

El ciclo continuo de inferencia, procesamiento y comunicación se articula a través de las siguientes etapas:

1. **Gestión de la señal de captura en cámara única:** La extensión intercepta y sustituye la función nativa de acceso a la cámara. El dispositivo físico realiza una única apertura y deriva su señal de video hacia un elemento oculto dentro de la página. La llamada de la plataforma no recibe este flujo directo; en su lugar, recibe la señal sintética de un lienzo virtual a 24 cuadros por segundo. El transmisor de video de la llamada sustituye su fuente original por la de este lienzo. Al apagar la extensión, la página devuelve la señal nativa a la llamada sin detener abruptamente el hardware. Esta estrategia evita errores de dispositivo ocupado o no disponible, los cuales ocurren cuando la llamada exige el control exclusivo de la cámara física.

2. **Transmisión de fotogramas hacia el clasificador (Modo Sordo):** En la modalidad para personas sordas, la extensión extrae las imágenes de la cámara oculta y las comprime en un formato liviano con resolución y calidad reducidas. El sistema envía fotogramas y aguarda la confirmación antes de despachar el siguiente. El mensaje viaja internamente a través del módulo en segundo plano sin abandonar la computadora ni ingresar de forma directa al motor principal ILSA. Si la cámara de la llamada se cierra, la página emite un aviso para reiniciar el proceso de captura y recorte.

3. **Estimación de puntos articulares en el entorno aislado:** Las restricciones de seguridad del navegador impiden ejecutar la herramienta de visión artificial directamente en el módulo en segundo plano. Por esta razón, el sistema decodifica la imagen comprimida sobre un lienzo, extrae los píxeles y los transfiere hacia un documento aislado y protegido. Con esta información, el sistema estructura un vector cinemático. Si el usuario seleccionó la opción de lateralidad zurda, el servidor activa la inversión automática de ejes y el cruce de manos en el vector lo que ajusta los datos al modelo original que fue entrenado con muestras de mano derecha. La imagen comprimida no viaja en este envío.

4. **Criterio de corte y segmentación gestual:** El algoritmo de recorte opera dentro de la extensión en modo automático, sin evaluar el movimiento de los píxeles para evitar interferencias si hay personas caminando en el fondo. El inicio de la seña

se registra al detectar una mano utilizable en la imagen. El cierre ocurre cuando las manos abandonan el encuadre, con un margen de tolerancia frente a eventuales pérdidas temporales de detección. El sistema no interrumpe una seña estática por falta de movimiento. El espacio de memoria guarda hasta 60 cuadros, al completarse o retirar las manos del *frame*, se cierra la captura y realiza un muestreo de 16 posiciones antes de la transmisión.

5. **Clasificación local y filtrado mediante compuertas:** Cada seña segmentada se envía mediante un paquete de datos que contiene únicamente el vector de coordenadas. El clasificador procesa la información dentro del motor local. Si el nivel de confianza es inferior al umbral mínimo del 75 %, la predicción se descarta y la interfaz la presenta en color coral con la etiqueta «Poca confianza». Si alcanza el umbral pero la compuerta de validación frena la entrada, la interfaz la despliega en tono gris como «No se agregó». Si la seña aprueba las validaciones, ingresa a la frase, el panel la destaca en cian con la indicación «En el mensaje» y el listado de la frase se actualiza.

6. **Finalización de enunciado y traducción semántica:** Tras un intervalo de 4 segundos sin actividad manual en la cámara, el sistema transmite la orden de cierre de frase. ILSA arma el español en el equipo si el enunciado es una sola glosa léxica, solo dígitos, o solo letras de dos en adelante y ninguna es I ni T. Una letra sola, un deletreo con I o T, OJO, VIERNES o AÑOS solas, y el resto de las frases van al servidor semántico. Si el módulo de traducción no emite respuesta, el motor conserva las señas aceptadas en formato de texto original sin reestructurar la oración.

7. **Emisión de salidas en el Canal Directo (Modo Sordo):** El sistema recibe la traducción en español y la publica en la página de la llamada. El módulo de video graba este texto sobre una franja oscura en la zona inferior del lienzo virtual, el cual conforma la imagen que los demás participantes observan. La capa visual del panel mantiene el texto visible durante 8 segundos. En la modalidad con salida por Subtítulo no se genera audio. En Audio o Ambos, la extensión pide a ILSA un WAV por `POST /tts`. ILSA lo sintetiza con la voz de Windows. El script inyectado en la página de Meet decodifica ese WAV, lo mezcla con el micrófono en un *AudioContext* y sustituye la pista de audio que Meet envía a la llamada. Esa mezcla es el componente que incorpora la voz al audio de Meet: el sintetizador del navegador no entra en esa pista. Quien seña oye la locución por los parlantes locales. Si el micrófono de Meet está silenciado, la voz no sale a la llamada y la extensión muestra un aviso.

8. **Procesamiento de entrada por voz en el Canal Inverso (Modo Oyente):** En la modalidad para personas oyentes, la extensión arma un WAV de un canal, en PCM lineal de 16 bits y a 16 kHz, y lo envía en `POST /stt`. El cuerpo lleva ese archivo en Base64. Base64 no comprime el audio: reescribe los mismos bytes como texto, y el cuerpo queda más largo que el WAV. El motor local decodifica ese texto y transcribe con un modelo de voz en español, en el propio equipo. Si la conversión no produce texto, la interfaz solicita repetir la locución o recurrir al canal de texto.

9. **Procesamiento de entrada por texto en el Canal Inverso (Modo Oyente):** El sistema captura el contenido escrito sin pasar por el módulo de voz. La entrada de voz y la de texto pueden permanecer activas de forma concurrente; ambas representan distintas vías de ingreso para el mismo caso de uso y no constituyen un modo separado.

10. **Generación de glosas y renderización de animación sintética:** La cadena de texto en español ingresa al motor local. ILSA la parte por espacios. Si queda una sola palabra, la pasa a mayúsculas y esa cadena es la glosa: no consulta al traductor ni a un diccionario para tomar esa decisión. Si quedan dos o más palabras, la oración se envía al servicio en la nube y retorna estructurada como señas en el orden gramatical del sistema. El motor local solicita los archivos de animación cinemática correspondientes a cada seña, con un límite de hasta 24 elementos por enunciado. En ausencia del archivo para una seña específica, el sistema solicita los archivos de cada letra para realizar un deletreo dactilológico. Si el servidor de animaciones no responde, las señas se conservan en texto y el bloque visual permanece vacío. La extensión recibe los datos de movimiento ya unificados por el servidor. En la superficie del lienzo, la cámara del usuario ocupa la mitad izquierda y el modelo de esqueleto ocupa la mitad derecha. La llegada de una nueva animación no interrumpe la secuencia en ejecución, y la cola de reproducción admite hasta 20 bloques animados, descarta el más antiguo al colmar su capacidad. Las señas procesadas también se graban sobre la franja inferior del mismo video. En el modo oyente, la persona oyente ve su cámara y el avatar en la miniatura local, y ahí los subtítulos le salen espejados. La persona sorda, que es quien tiene que comprenderlos, los recibe al derecho.

6.4.2. Motor local ILSA (FastAPI)

El servidor local se implementó en Python mediante el *framework* asíncrono FastAPI y se ejecuta sobre un servidor Uvicorn. El servicio escucha en `127.0.0.1:8765`, únicamente en la interfaz de bucle local. Esa escucha impide las conexiones directas que llegan por las demás interfaces de red. Los procesos de la misma máquina pueden abrir el puerto, y el enlace no comprueba su identidad ni opera como autorización.

Para cumplir con las políticas de seguridad del navegador, el middleware CORS compara el encabezado `Origin` con tres valores literales y con una expresión regular. Los literales son `https://meet.google.com`, `http://127.0.0.1:8765` y `http://localhost:8765`. El script de contenido corre dentro de la página de Meet, de modo que sus peticiones presentan el primer origen. Los documentos propios de la extensión, cuyo origen comienza por `chrome-extension://`, se aceptan con la expresión regular `^chrome-extension://[a-p]{32}$`. Esa expresión describe el formato de un identificador de extensión de Chrome: 32 letras minúsculas de la `a` a la `p`. El navegador permite entonces leer la respuesta a cualquier extensión cuyo identificador tenga esa forma. La configuración no reserva el identificador de este prototipo.

La expresión regular se usa porque la extensión todavía no está publicada en la tienda del navegador y, por lo tanto, no tiene un identificador fijo. Al publicarla, la tienda asigna un identificador de ese mismo formato y lo conserva en las actualizaciones. En ese momento la expresión regular puede retirarse y el servidor puede admitir solo el origen literal `chrome-extension://` seguido de ese identificador.

Ese control no autentica al llamador. La API local no exige credencial, y un programa de la misma computadora puede consultar el puerto sin presentar el origen de una extensión. El servidor responde además con la cabecera de acceso a red privada (`Access-Control-Allow-Private-Network: true`), requerida cuando una página HTTPS realiza peticiones hacia el dominio local.

La voz de salida se sintetiza en ILSA, en `POST /tts`, con la API de Windows. El WAV vuelve a la extensión y el script de la página lo suma a la pista de audio de Meet.

El motor local se empaquetó como un ejecutable independiente de Windows (`ILSA.exe`) mediante PyInstaller, e incluye una interfaz gráfica desarrollada en Tkinter para permitir la selección de modo (Sordo u Oyente), la configuración de la mano dominante y el control de encendido del servicio.

Los cuerpos, las unidades y los errores de cada ruta local están en la Sección 6.7.1.

6.5. Implementación del avatar

6.5.1. Vía descartada: retarget hacia un avatar VRM

Antes del diccionario de poses se probó un personaje VRM, con malla y huesos de longitud fija, animado por rotaciones. El fundamento del ensayo es el *retarget*: el paso de posiciones de *landmarks* a rotaciones de otro esqueleto. MediaPipe Holistic entrega posiciones, y el avatar exige rotaciones sobre su propio esqueleto. El visor se armó con Three.js. Para la mano se probó Kalidokit, una biblioteca de JavaScript para ese *retarget*. Toma los *landmarks* de MediaPipe y calcula rotaciones de Euler del esqueleto y pesos de *blendshape* del rostro. En la mano resume cada dedo en un único ángulo de rizo. También se probó una cinemática inversa que lleva el extremo del hueso hasta el punto medido.

La prueba no se retuvo. El desarrollo no contaba con práctica suficiente de animación tridimensional, y las bibliotecas de *retarget* no sostuvieron de forma estable el paso de la pose al esqueleto VRM. La pose conserva una profundidad relativa en el eje Z. La componente z de la mano es relativa a la muñeca y no es una distancia en metros. Una sola cámara limita esa medida: una toma frontal no alcanza para reconstruir el eje. El emparejamiento fallaba con frecuencia por la escala y por el ajuste al esqueleto. Esa coordenada y el desplazamiento en el plano no comparten escala, de modo que el hueso no recibe una distancia utilizable. Si se respeta la longitud del hueso, la yema no cae sobre el punto medido. Si la cinemática inversa alarga el miembro hasta ese punto, la mano pierde la configuración y, en el alfabeto manual, la letra cambia. Kalidokit, además, confunde configuraciones próximas.

El diccionario de `.pose` evita ese paso: dibuja los puntos medidos y no los convierte en rotaciones de otro cuerpo. Si más adelante se retoma el avatar tridimensional, una línea de mejora es filmar cada seña desde tres ángulos, para reconstruir la profundidad que una sola toma frontal no conserva.

6.5.2. Decisiones técnicas de la reproducción

La Sección 5.3 fija qué se muestra y por qué. Aquí quedan los valores con los que esas reglas se programaron. El servicio de poses entrega el archivo `.pose` pedido, sin recortarlo, sin llevarlo a una cadencia común y sin armar el puente. La preparación del clip y el empalme entre glosas corren en ILSA, una vez que los archivos ya llegaron. La extensión recibe la lista de fotogramas ya armada y la dibuja en la mitad derecha del lienzo. Elige, según el tiempo transcurrido, cuál de esos fotogramas le toca mostrar.

Valores del compositor. Antes del recorte, todos los clips se llevan a 30 fotogramas por segundo. Con esa tasa común, una espera de seis fotogramas dura lo mismo en cualquier seña. Si se dejara cada video a la cadencia con la que fue filmado, seis fotogramas de un clip a 24 por segundo durarían más que seis fotogramas de un clip a 30, y el empalme perdería la medida del tiempo.

El suavizado es un promedio exponencial. Si x_t es la posición recién medida y p_{t-1} es la pose ya aceptada en el fotograma anterior, la pose que se conserva es

$$p_t = \alpha\, x_t + (1 - \alpha)\, p_{t-1}, \tag{6.7}$$

con $\alpha = 0{,}50$. La medición nueva y la pose anterior pesan lo mismo. Ese peso es único para todo el diccionario.

El recuadro común mide 720×780 píxeles. El cuerpo se escala para que el ancho de hombros quede en 250 píxeles. Así, dos tomas de distinto encuadre ocupan el mismo espacio cuando se pegan una detrás de la otra.

El puente dura entre 4 y 8 fotogramas. Si las muñecas del final de una seña y del inicio de la siguiente ya están cerca, se usan 4. Si están lejos, se llega a 8, para que la mano no salte. Un punto cuya confianza queda por debajo de 0,15 sale en la lista como ausente, y el trazo del lienzo no lo une con el punto anterior.

La entrada, desde la postura de espera hasta el primer fotograma de la seña, y la salida, de vuelta a esa espera, duran cerca de seis fotogramas de la tasa de 30. A esa cadencia, cada uno de esos tramos ocupa cerca de un quinto de segundo. La secuencia se corta en el techo fijado en el diseño: 24 glosas y 1200 fotogramas. Esos 1200 fotogramas, a 30 por segundo, son 40 s de animación. Con la entrada y la salida, el bloque que alcanza ese techo dura 40,4 s. El bloque en curso sigue hasta el final. El mensaje nuevo entra en la cola, que admite hasta 20 bloques. El turno siguiente espera el resto del bloque en pantalla y los que quedaron delante. En ese techo, la espera es de 40,4 s por bloque. Con el bloque en pantalla y 19 delante, todos en el tope, la espera del último llega a 808 s. Ese valor es el techo del código, no una medición de una sesión.

Postura de espera. No hay un archivo aparte llamado reposo. El cuerpo y las manos de la espera se copian de los primeros fotogramas de una seña de referencia, el tramo en el que la persona todavía no empezó el gesto, y se escalan al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea.

Reflejo de la miniatura local. La videollamada muestra la miniatura de quien usa el sistema como un reflejo. La extensión deshace ese reflejo solo en ese video local para la cámara y el esqueleto. El esqueleto que viaja hacia la otra persona no se invierte. Esas dos vistas quedan iguales: la cámara en la mitad izquierda y el esqueleto, sin darlo vuelta, en la mitad derecha. En el modo oyente, los subtítulos de la miniatura local le salen espejados a la persona oyente. La persona sorda los recibe al derecho.

6.6. Instancia de validación del prototipo mediante microservicios en la nube

Para la validación funcional del prototipo en un entorno de producción representativo, se completó el desacoplamiento del backend semántico y del servidor de poses del avatar hacia la infraestructura serverless de Google Cloud Run.

La arquitectura se dividió en dos microservicios independientes, empaquetados en contenedores Docker:

- **Servidor Semántico:** Este microservicio gestiona los modelos cuantizados a 4 bits (GGUF `Q4_K_M`) para la traducción directa e inversa. Se desplegó en la región *us-central1* con una asignación de 8 GiB de memoria RAM y 4 vCPU.
- **Servidor de Poses y Activos del Avatar:** Este microservicio entrega el diccionario de glosas como archivos `.pose`. La carpeta que viaja en la imagen reúne 551 archivos y ocupa cerca de 637 MiB. El proceso no los carga todos en memoria: responde el archivo que pide cada solicitud. Se desplegó en la región *southamerica-east1* con una asignación de 512 MiB de memoria RAM y 1 vCPU.

El proceso de despliegue e integración en la nube se llevó a cabo bajo los siguientes criterios técnicos:

1. **Proceso de despliegue:** Mediante la herramienta `gcloud builds`, el código fuente y las dependencias de cada rama se enviaron a Google Cloud Build para construir las imágenes de contenedor, las cuales quedaron almacenadas en Google Artifact Registry (`ilsa-repo`).
2. **Gestión dinámica de puertos y comprobación de estado:** Cada contenedor escucha en `0.0.0.0` y en el puerto de la variable `$PORT`, que Cloud Run inyecta al crear la instancia. Esa apertura habilita la sonda de arranque y queda fuera de los plazos. El tiempo máximo de una solicitud, la comprobación de arranque y la espera del cliente son tres plazos distintos. Configurar uno deja los otros en su valor. El de la solicitud es el de Cloud Run: 300 segundos por defecto. Si la respuesta no sale dentro de ese plazo, la plataforma cierra la conexión y responde HTTP 504. El reloj de esa solicitud incluye el arranque del contenedor desde que la petición llega. La comprobación de arranque usa la sonda TCP por defecto, de 240 segundos (`timeoutSeconds` 240, `periodSeconds` 240 y `failureThreshold` 1). Esa sonda abre una conexión al puerto. La ruta `/health` queda para las consultas de la aplicación. Si el proceso no escucha dentro de esos 240 segundos, la plataforma apaga el contenedor. El cliente es independiente: ILSA no declara un plazo propio y espera la respuesta o el corte del servidor. Un cambio del plazo de la solicitud, o del de la sonda, deja intacta esa espera.
3. **Política de asignación de instancias y control presupuestario:** Con el propósito de optimizar el consumo de créditos y acotar los costos operativos dentro del marco del Trabajo Final de Grado, se configuró una política de instancias delimitada entre cero y una réplica (`-min-instances 0`, `-max-instances 1`). El escalado a cero libera los recursos cuando no existen llamadas activas.

El motor local ILSA y la extensión de navegador se enlazaron con las URLs públicas HTTPS de ambas instancias en Cloud Run. Esta integración confirmó la correcta transmisión de cargas textuales y la recuperación de activos para la sintaxis del avatar en tiempo pseudo-real y mantiene liberados los núcleos de cómputo y la memoria del dispositivo cliente.

6.7. Contratos HTTP de las APIs

Los Cuadros 5.1, 5.2 y 5.3 del Capítulo 5 nombran las rutas. Esta sección fija lo que se envía, lo que se recibe, qué campo es obligatorio, en qué unidad se expresa y

qué ocurre si el pedido falla. El contrato local coincide con `src/backend/server.py` del commit https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/5974ac79d41ec6738e554354aad44ba963e5f412. El semántico coincide con la rama `semantic-cloud`. El de poses, con `avatar-cloud`.

Las tres APIs hablan JSON UTF-8, salvo las que entregan un archivo. El encabezado de un `POST` con cuerpo es `Content-Type: application/json`. Los tiempos van en milisegundos. La confianza del clasificador es un número en $[0, 1]$. Un cuerpo que no cumple el esquema de Pydantic responde HTTP 422; el detalle lista el campo y el motivo. Ese código no figura en los cuadros del diseño: lo agrega FastAPI al validar.

6.7.1. Motor local ILSA

La base es `http://127.0.0.1:8765`. No hay *token*. Encender en la ventana de ILSA crea la sesión. Mientras esa sesión no existe, `GET /health` responde 200 con `ok` en falso y el resto de las rutas de sesión responde 503, con `detail` igual a «El backend todavía no inicializó el pipeline.». El modo `signer` es el canal sordo. El modo `hearing` es el canal oyente.

Instantánea compartida

`GET /state`, `POST /session` y las respuestas de `POST /sign` y `POST /utterance/end` reutilizan el mismo objeto. El Cuadro 6.1 lista sus campos. En modo oyente el mismo objeto se reduce a `mode`, `classifier_ready` (siempre falso), `semantic_ready`, `semantic_error`, `spanish`, `glosses` y `device`.

Cuadro 6.1: Campos de la instantánea del modo sordo (`GET /state`).

Campo	Obligatorio	Unidad	Significado
`left_handed`	sí	adimensional	`true` si la sesión es zurda.
`classifier_ready`	sí	adimensional	El `.pth` está en memoria.
`num_classes`	sí	cantidad	Clases del mapeo; el prototipo usa 100.
`semantic_ready`	sí	adimensional	El `GET /health` remoto respondió 2xx.
`semantic_error`	sí	texto	Vacío si no hay fallo.
`semantic_model`	sí	texto	En este prototipo, `remote`.
`semantic_busy`	sí	adimensional	Traducción en curso.
`top3`	sí	lista	Hasta tres pares `gloss` y `confidence` $\in [0, 1]$.
`glosses`	sí	lista de textos	Búfer del enunciado aún no cerrado.
`pending_text`	sí	texto	Esas glosas unidas por espacio.
`spanish`	sí	texto	Última oración emitida.
`last_utterance`	sí	texto	Última secuencia enviada al traductor.
`device`	sí	texto	Dispositivo del clasificador; aquí `cpu`.

`GET /state`

Sin cuerpo. HTTP 200 con el objeto del Cuadro 6.1, o 503 si aún no hay sesión. No agrega campos extra.

GET /health

Sin cuerpo. Siempre HTTP 200. Suma a la instantánea el modo, las etiquetas del traductor y una línea de métricas. `ok` es verdadero solo si ya hay sesión.

```
{
  "ok": true,
  "mode": "signer",
  "classifier_ready": true,
  "num_classes": 100,
  "semantic_ready": true,
  "semantic_error": "",
  "semantic_model": "remote",
  "semantic_label": "Remote translator",
  "semantic_load": "cloud",
  "device": "cpu",
  "left_handed": false,
  "metrics": "signs 12/15 - conf 91% - clf 40 ms"
}
```

Si el traductor remoto no responde, `semantic_ready` queda en falso, `semantic_label` vale «Translator unavailable» o «No translator» y `semantic_load` queda vacío. Eso no cambia el código HTTP.

GET /config

Sin cuerpo. HTTP 200 o 503. En modo oyente la respuesta es `{"mode": "hearing"}`. En modo sordo entrega los umbrales que usa la extensión para recortar. El Cuadro 6.2 fija las unidades. El JSON de respuesta es el siguiente.

```
{
  "max_frames": 16,
  "min_capture_frames": 5,
  "capture_buffer_size": 60,
  "confidence_threshold": 0.75,
  "inference_cooldown_sec": 1.0,
  "utterance_pause_sec": 4.0,
  "mode": "signer",
  "left_handed": false,
  "num_classes": 100
}
```

GET /metrics

Sin cuerpo. HTTP 200. Conteos de la sesión en curso y percentiles de tiempo en milisegundos. `file` es la ruta del JSONL diario en `%LOCALAPPDATA%/ILSA/metrics/`. `n_sign`, `n_accepted` y `n_added` son cantidades de señas. `mean_conf1` es la media de la primera confianza, en $[0, 1]$. Los percentiles dentro de `ms` están en milisegundos. `process.rss_mb` está en mebibytes. `process.cpu_percent` está en por ciento del proceso.

```
{
  "file": "C:/Users/x/AppData/Local/ILSA/metrics/live-20261030.
   jsonl",
```

Cuadro 6.2: Campos de `GET /config` en modo sordo.

Campo	Unidad	Valor del prototipo
`max_frames`	fotogramas	16, ventana del clasificador.
`min_capture_frames`	fotogramas	5, piso para no rechazar la seña.
`capture_buffer_size`	fotogramas	60, techo de lo que llega en `POST /sign`.
`confidence_threshold`	adimensional	0,75 sobre la softmax.
`inference_cooldown_sec`	segundos	1,0 entre dos clasificaciones.
`utterance_pause_sec`	segundos	4,0 de inactividad para cerrar el enunciado.
`landmark_motion_threshold`	dist. interhumeral	0,008.
`motion_pixel_threshold`	píxeles	500. El recorte en vivo no usa este valor.
`hands_frames_to_start`	fotogramas	2 con mano visible para abrir la captura.
`missing_hands_limit`	fotogramas	12 sin mano para cerrar la captura.
`still_frames_limit`	fotogramas	10.
`pose_dim`	coordenadas	$33 \times 3 = 99$.
`mode`	texto	`signer`.
`left_handed`	adimensional	Copia de la sesión.

```
  "n_sign": 15,
  "n_accepted": 12,
  "n_added": 9,
  "n_utterance": 3,
  "n_hearing": 0,
  "mean_conf1": 0.91,
  "via": {"literal": 1, "llm": 2, "fallback": 0},
  "ms": {"classifier_p50": 40.0, "inference_p50": 410.0, "
   transport_p50": 95.0}
}
```

`POST /session`

El cuerpo admite `left_handed` (booleano, por omisión `false`). La extensión lo envía. El servidor no lo aplica: la lateralidad queda fijada al Encender en la ventana de ILSA, para que una reconexión de Meet no cambie la mano ni borre el enunciado en curso. La respuesta es la instantánea, HTTP 200, o 503 si aún no hay sesión.

```
{"left_handed": false}
```

`POST /sign`

Solo modo sordo. El campo obligatorio es `frames`: una lista de 1 a 60 fotogramas. Cada fotograma es un objeto con `pose`, `left_hand` y `right_hand`. Cada lista de puntos acepta como máximo 33 elementos. Un punto es `{"x", "y", "z"}` o una terna. `x` e `y` llegan como fracción del ancho y del alto de la imagen, según MediaPipe. `z` es la profundidad relativa de ese paquete, no un metro ni un píxel. El servidor ancla esos valores al punto medio de los hombros y los divide por la distancia interhumeral (Ecuaciones 6.1 y 6.2). Una lista ausente o nula se rellena con ceros. `client` es opcional: `capture_ms` y `holistic_ms` van en milisegundos; `n_raw` es la cantidad de fotogramas antes del muestreo a 16.

El ejemplo siguiente muestra un fotograma abreviado. El envío real lleva 33 puntos de pose y 21 de cada mano, y hasta 60 fotogramas.

```
{
  "frames": [
    {
      "pose": [{"x": 0.51, "y": 0.42, "z": -0.08}],
      "left_hand": null,
      "right_hand": [{"x": 0.62, "y": 0.55, "z": 0.01}]
    }
  ],
  "client": {"capture_ms": 180.0, "holistic_ms": 45.0, "n_raw":
   24}
}
```

HTTP 200 no significa que la glosa haya entrado al búfer. El servidor distingue el rechazo de aplicación del error de protocolo. El Cuadro 6.3 junta ambos. El JSON siguiente es una seña aceptada y agregada.

```
{
  "accepted": true,
  "added": true,
  "activity": true,
  "gloss": "HOLA",
  "confidence": 0.91,
  "reject_reason": "",
  "ms": 42.0,
  "glosses": ["HOLA"],
  "pending_text": "HOLA",
  "spanish": "",
  "left_handed": false,
  "classifier_ready": true,
  "num_classes": 100,
  "semantic_ready": true,
  "top3": [{"gloss": "hola", "confidence": 0.91}]
}
```

`ms` es el tiempo total de esa petición, en milisegundos. `confidence` es la primera softmax, en $[0, 1]$. Si `added` es falso y la confianza queda bajo 0,75, `reject_reason` vale `confidence`. Si la compuerta de repetición la frena, vale `repeat`.

`POST /stt`

Solo modo oyente. El campo obligatorio es `wav_b64`: el WAV en Base64. `lang` se acepta y se ignora; el modelo es español. El archivo es PCM lineal de 16 bits. Un canal se usa tal cual. Varios canales se reducen al primero. La tasa se remuestrea a 16 kHz si hace falta. Menos de 0,35 s de audio produce HTTP 200 con `spanish` vacío: no es un error. La extensión arma exactamente ese formato, un canal a 16 kHz.

```
{"wav_b64": "UklGRiQAAABXQVZFZm10IBAAAAABAAE...", "lang": "es-ES"
   }

{"spanish": "hola me llamo maite", "ok": true}
```

Cuadro 6.3: Errores y rechazos de `POST /sign`.

HTTP	Campo o `detail`	Cuándo
400	`Modo oyente: no se clasifican señas.`	La ventana está en modo oyente.
400	`Faltan frames de la seña.`	`frames` llega vacío.
422	lista de validación	Más de 60 fotogramas, un ítem que no es objeto, o más de 33 puntos en una lista.
503	pipeline no inicializado	ILSA todavía no Encendió.
200	`accepted: false`, `reason: cooldown`	Menos de 1,0 s desde la seña anterior.
200	`accepted: false`, `reason: too_short`	Menos de 5 fotogramas.
200	`accepted: false`, `reason: bad_tensor`	Tras el muestreo no quedan 16 filas.

Errores: 400 «Solo modo oyente.» si el modo es sordo; 400 «Falta el audio.» si `wav_b64` viene vacío; 400 «WAV inválido» si Base64 no decodifica; 502 con el texto de la excepción si el WAV no es PCM de 16 bits o si Vosk no carga.

`POST /hearing`

Solo modo oyente. El campo de trabajo es `spanish` (texto). `final` es un booleano; por omisión `true`. Una cadena vacía no es error: HTTP 200 devuelve la instantánea sin traducir. Con `final` en falso tampoco traduce. Una sola palabra se pasa a mayúsculas y no sale a la nube. Dos o más palabras van al `POST /hearing` remoto. Después ILSA pide los archivos de pose y arma el clip.

```
{"spanish": "Me llamo Maite.", "final": true}

{
  "closed": true,
  "spanish": "Me llamo Maite.",
  "glosses": "NOMBRE MIO Maite",
  "caption": "NOMBRE MIO Maite",
  "pause_ms": 0.0,
  "queue_ms": 12.0,
  "inference_ms": 380.0,
  "transport_ms": 90.0,
  "delivery_ms": 210.0,
  "avatar": {
    "ok": true,
    "fps": 30.0,
    "w": 720,
    "h": 780,
    "shoulder": 250.0,
    "played": ["NOMBRE", "MIO", "M", "A", "I", "T", "E"],
    "missing": [],
    "frames": [{"p": [210, 180, 460, 180], "f": [], "l": [], "r":
    []}]
  }
}
```

`pause_ms`, `queue_ms`, `inference_ms`, `transport_ms` y `delivery_ms` están en milisegundos. `fps` está en fotogramas por segundo. `w` y `h` están en píxeles del recuadro 720×780. `shoulder` es el ancho de hombros en píxeles. Cada fotograma de `avatar.frames` lleva `p`, `f`, `l` y `r`: enteros x, y intercalados. El valor -1 marca un punto con confianza menor que 0,15. El ejemplo recorta esas listas.

Errores: 400 «Solo modo oyente.»; 400 «Esta sesión no acepta voz.» si la sesión no es la oyente; 503 si no hay sesión. Si el traductor o el servidor de poses fallan, HTTP sigue en 200: `glosses` puede quedar vacío y `avatar.ok` en falso, con `avatar.error` en texto y `frames` vacío.

`POST /activity` y `POST /utterance/end`

`POST /activity` no exige cuerpo. Reinicia el reloj de inactividad de ~ 4 s. Responde `{"ok": true}`, o 503.

`POST /utterance/end` cierra el búfer y traduce. El cuerpo es opcional. `pause_ms` va en milisegundos; si falta, se usa 4000. Sin glosas pendientes, HTTP 200 con `closed: false`. Con glosas, `closed: true`, la lista en `glosses`, la oración en `spanish` y `via` igual a `literal`, `lexical`, `llm` o `fallback`. `ms`, `pause_ms`, `queue_ms`, `inference_ms` y `transport_ms` están en milisegundos. 503 si no hay sesión.

```
{"pause_ms": 4000.0}

{
  "closed": true,
  "glosses": ["NOMBRE", "MIO", "Maite"],
  "spanish": "Me llamo Maite.",
  "via": "llm",
  "ms": 520.0,
  "pause_ms": 4000.0,
  "queue_ms": 8.0,
  "inference_ms": 410.0,
  "transport_ms": 95.0,
  "delivery_ms": 0.0
}
```

`POST /tts` y `POST /metrics/client`

`POST /tts` es del modo sordo. El campo es `text`, como máximo 500 caracteres. Responde `wav_b64`: un WAV PCM de la voz de Windows, en Base64. Errores: 400 «La voz en off es del modo sordo.»; 502 si Windows no sintetiza o si el texto llega vacío.

```
{"text": "Me llamo Maite."}

{"ok": true, "wav_b64": "UklGRiQAAABXQVZFZm10IBAAAAABAAE..."}
```

`POST /metrics/client` registra la entrega en el cliente. No figura en el Cuadro 5.1. Campos opcionales: `channel` (`direct` u otro), `delivery_ms` y `pause_ms` en milisegundos, `fps` y `holistic_fps` en fotogramas por segundo, `rss_mb` y `js_heap_mb` en mebibytes. Responde `{"ok": true}` y el snapshot del proceso.

```
{"channel": "direct", "delivery_ms": 18.0, "fps": 24.0, "
   js_heap_mb": 85.3}
```

GET /mediapipe/{name} y GET /download/exe

GET /mediapipe/{name} entrega un archivo estático de Holistic. name es el nombre de archivo, sin barras. Extensiones admitidas: .wasm, .js, .data, .tflite, .binarypb. HTTP 200 con el binario; 400 «Nombre inválido» o «Tipo de archivo no permitido»; 404 con detail «Falta» seguido del nombre, por ejemplo «Falta holistic.wasm».

GET /download/exe entrega ILSA.exe. HTTP 200 con el binario; 404 «No hay .exe. En desarrollo usá: python run_backend.py».

6.7.2. Servicio semántico remoto

ILSA llama a este servicio por HTTPS. El cuerpo es JSON. Si la variable LSA_SEMANTIC_TOKEN está vacía, no hay autenticación. Si tiene valor, el pedido debe repetirlo en Authorization: Bearer ... o en X-LSA-Token. Un *token* distinto responde 401, con detail igual a Invalid token. El alias POST /translate ejecuta lo mismo que POST /sign.

GET /health

Sin cuerpo. HTTP 200. ready es verdadero cuando los dos canales tienen un GGUF en memoria. rss_mb está en mebibytes. cpu_percent está en por ciento. Un 200 con ready en falso ocurre durante la carga: el ping de ILSA solo mira el código HTTP, no este campo.

```
{
  "ok": true,
  "ready": true,
  "loading": false,
  "load_error": null,
  "primary": "qwen2.5-1.5b.gguf",
  "backup": "llama-3.2-1b.gguf",
  "sign_gguf": "src/semantic/models/sign/qwen2.5-1.5b.gguf",
  "hearing_gguf": "src/semantic/models/hearing/qwen2.5-1.5b.gguf"
   ,
  "model_used_sign": "qwen2.5-1.5b.gguf",
  "model_used_hearing": "qwen2.5-1.5b.gguf",
  "rss_mb": 2100.0,
  "cpu_percent": 3.5
}
```

POST /sign y POST /hearing

El canal directo exige glosses. El inverso exige spanish. Ambos son texto de 1 a 2000 caracteres. ILSA envía la secuencia ya compacta, o la oración, sin el prefijo «Glosas:» ni «Español:»: eso lo antepone el servidor.

```
{"glosses": "HOLA YO NOMBRE Maite"}

{
  "spanish": "Me llamo Maite.",
  "glosses": "NOMBRE MIO Maite",
  "model_used": "qwen2.5-1.5b.gguf",
```

```
  "ms": 410.0,
  "inference_ms": 410.0,
  "queued_ms": 8.0,
  "queue_ms": 8.0
}
```

```
{"spanish": "Me llamo Maite."}

{
  "glosses": "NOMBRE MIO Maite",
  "spanish": "Me llamo Maite.",
  "model_used": "qwen2.5-1.5b.gguf",
  "ms": 390.0,
  "inference_ms": 390.0,
  "queued_ms": 5.0,
  "queue_ms": 5.0
}
```

`ms` e `inference_ms` son el mismo reloj de inferencia, en milisegundos. `queued_ms` y `queue_ms` son la espera por el candado de `llama.cpp`, también en milisegundos. `model_used` es el nombre del archivo GGUF que respondió.

Cuadro 6.4: Errores del servicio semántico remoto.

HTTP	`detail`	**Cuándo**
401	`Invalid token`	Hay *token* configurado y el pedido no lo trae o no coincide.
422	validación de Pydantic	Falta `glosses` o `spanish`, llega vacío o supera 2000 caracteres.
503	texto de la excepción	No hay GGUF usable, el archivo no está o la inferencia lanza `RuntimeError`.
504	corte de Cloud Run	La respuesta no salió en 300 s (arranca el contenedor incluido).

6.7.3. Servicio de poses del avatar

Este servicio no genera animación: lee un archivo. La base es la URL pública de Cloud Run. El nombre pedido va en mayúsculas ASCII, sin tilde y sin ñ. ILSA convierte «ñ» en «n» (`MAÑANA` busca `MANANA.pose`). La letra Ñ, cuando es la glosa entera, se pide como `ENIE`. Los guiones bajos se borran: `VIVIR_EN` busca `VIVIREN.pose`. Cada respuesta lleva el encabezado `X-ILSA-Ms` con el tiempo de esa lectura, en milisegundos.

`GET /health`

Sin cuerpo. HTTP 200 y `{"status": "ok"}`. No lista los archivos.

`GET /tposes/{glosa}.pose`

Sin cuerpo. El segmento `{glosa}` es el nombre sin extensión y sin espacios. HTTP 200 entrega el binario `.pose`, tipo `application/octet-stream`. HTTP 404 si ese nombre no

está en la carpeta. No hay JSON de error en el 404 de archivos estáticos. ILSA interpreta el 404 como «no hay clip» y pasa al deletreo.

```
GET /tposes/HOLA.pose
```

GET /fingerspell

La query `letters` es obligatoria. Acepta repeticiones (`letters=M&letters=A`) o una lista separada por comas (`letters=M,A,I,T,E`). Como máximo 20 letras por pedido. El alfabeto admitido es A–Z, `CH` y `ENIE` (la ñ). La respuesta 200 es `multipart/mixed`: cada parte es un `.pose`, con `X-Index` (entero desde 0) y `X-Letter` (la letra canónica). El encabezado `X-Count` es la cantidad de partes.

```
GET /fingerspell?letters=M,A,I,T,E
```

Cuadro 6.5: Errores de `GET /fingerspell`.

HTTP	**Cuerpo JSON**	**Cuándo**
400	`The request does not include any letters.`	`letters` llega vacío.
400	tope de 20, con `received` y `limit`	Más de 20 *tokens*.
400	`{token} is not a fingerspelling letter.`	El *token* no está en A–Z, `CH` ni `ENIE`.
404	`No pose file exists for letter {letter}.`	La letra es válida y falta el archivo.

Capítulo 7

Pruebas y validación

En el presente capítulo se expone la estrategia de verificación y validación del sistema de traducción en tiempo pseudo-real de LSA. Se detallan el plan de pruebas adoptado, la ejecución de los experimentos sobre los distintos componentes y el análisis cuantitativo y cualitativo de los resultados obtenidos frente a los requerimientos del proyecto. El objetivo central consiste en evidenciar la aplicación de criterios técnicos para evaluar el funcionamiento, el desempeño y la calidad del *software*, con una distinción clara entre planificación, ejecución y análisis de resultados.

7.1. Plan de pruebas

El plan de pruebas establece la estrategia sistemática para comprobar el correcto funcionamiento, el desempeño en tiempo real y la calidad del *software* desarrollado.

7.1.1. Alcance y tipos de pruebas

La evaluación del sistema abarca desde la recolección de datos y el entrenamiento de modelos de aprendizaje profundo hasta la integración completa de los componentes en una videollamada web. Se aplicaron los siguientes tipos de pruebas:

- **Pruebas unitarias y de componentes:** Verificación del extractor de puntos clave (*landmarks*), la lógica de filtrado gestual (`RepeatGate`), los clasificadores de señas y la cuantización de los modelos de lenguaje.
- **Pruebas de integración:** Evaluación de las interfaces entre la extensión de navegador (Manifest V3), el motor de inferencia local, las llamadas a API y el cliente de videoconferencia.
- **Pruebas de rendimiento y latencia:** Medición de tiempos de respuesta por etapa del canal de procesamiento, consumo de memoria en cliente y servidor, y tasa de fotogramas por segundo.
- **Pruebas de sistema de extremo a extremo (*End-to-End*):** Validación del circuito completo de traducción (LSA a español y español a LSA) en entornos de llamada reales.
- **Pruebas de robustez y casos límite:** Evaluación del clasificador frente a cambios de iluminación, distorsiones ópticas de cámara web, presencia de múltiples personas en el fondo y variabilidad en la velocidad de articulación gestual.

7.1.2. Entornos de evaluación

Las pruebas se ejecutaron en tres entornos representativos:

1. **Estación de desarrollo (GPU):** Utilizada para la evaluación estática fuera de línea, el entrenamiento de modelos y las pruebas de velocidad máxima de inferencia.

2. **Equipo objetivo de cliente (Laptop sin GPU):** Configuración de gama media estándar orientada a verificar el comportamiento en tiempo real, el consumo de CPU/RAM y la latencia perceptible por el usuario final. Sus componentes están en la Sección 7.1.3.

3. **Infraestructura en la nube:** Instancia de Google Cloud Run (configuración de 4 vCPU y 8 GiB de memoria RAM) conectada mediante HTTPS para los servicios remotos de traducción semántica y reproducción de avatar.

7.1.3. Equipo portátil de las pruebas de cliente

La sesión de la Sección 7.3.2 corrió en una laptop con Windows 11 Home. El Cuadro 7.1 reúne los componentes de ese equipo. El nombre comercial del equipo no quedó registrado. La toma del 30 de octubre de 2026 se midió en esa laptop y en la estación de escritorio: cada máquina guardó el lado de quien seña y el de quien escucha. La PC está en la Sección 7.3.4.

Cuadro 7.1: Laptop de las pruebas de cliente.

Componente	**Valor**
Procesador	Intel Core i7-10510U a 1,80 GHz
Memoria RAM	8 GB a 2667 MT/s
Gráficos	Intel UHD Graphics, 128 MB
Almacenamiento	477 GB
Sistema operativo	Windows 11 Home

7.1.4. Criterios de aceptación y trazabilidad

Cada caso de prueba se vinculó de manera directa con los requerimientos funcionales y no funcionales del sistema. En el Cuadro 7.2 se sintetizan los criterios de aceptación principales definidos para la validación.

Cuadro 7.2: Matriz de trazabilidad entre requerimientos y criterios de aceptación para pruebas.

Módulo	**Requerimiento asociado**	**Criterio de aceptación**
Clasificador Gestual	RF-02	Exactitud Top-1 en cámara real $> 75\,\%$ y Top-3 $> 90\,\%$. Latencia de inferencia $< 10\,\text{ms}$.
Traductor Semántico	RF-03, RF-06	Exactitud sintáctica/semántica $> 85\,\%$, BLEU > 90, BERTScore > 98.
Motor Local	RF-02	Supresión de falsos positivos y rebotes sin descartar repeticiones intencionales.
Integración Meet	RF-04, RF-08	Sin interrupción de la pista de video principal; renderizado de subtítulos sin pérdidas.

7.2. Resultados de la verificación

Esta sección presenta la evidencia empírica obtenida a partir de la ejecución del plan de pruebas sobre cada componente del sistema.

7.2.1. Evaluación del clasificador gestual (TinySkeletonClassifier)

El modelo de clasificación gestual se evaluó tanto en entornos estáticos fuera de línea como en pruebas en vivo frente a cámara web en la estación local.

Evolución del *corpus* y vocabulario

La incorporación de clases al modelo avanzó de manera paulatina conforme se consolidaron las muestras del dataset propio de LSA. En el Cuadro 7.3 se detalla la progresión del vocabulario y las muestras recopiladas.

Cuadro 7.3: Progresión cronológica del conjunto de datos y vocabulario de LSA.

Nro	**Clases**	**Muestras/Clase**	**Muestras Entrenamiento**	**Muestras Validación**
1	91	50	3 640	910
2	94	60	4 512	1 128
3	97	60	4 656	1 164
4	100	60	4 800	1 200
5	100	100	8 000	2 000

Comparativa entre evaluación fuera de línea y cámara en vivo

En el Cuadro 7.4 se expone la diferencia observada entre la exactitud de validación sobre el dataset estático y el rendimiento real obtenido frente a la cámara web.

Cuadro 7.4: Exactitud de validación fuera de línea frente al acierto en cámara. Las filas de 91 a 97 clases corresponden a una toma por seña. Las de 100 clases corresponden a diez tomas por seña.

Checkpoint	**Dataset**	**Val. Top-1**	**Cámara Top-1**	**Cámara Top-3**
16 fotogramas, 91 clases	1	97,36 %	61,5 %	83,5 %
32 fotogramas, 128/4/2	1	96,48 %	71,4 %	87,9 %
32 fotogramas, 64/2/1	1	97,36 %	67,0 %	90,1 %
Optuna v1, 32 fotogramas	1	97,69 %	66,0 %	84,6 %
Optuna v2, 12 fotogramas, 94 clases	2	97,78 %	80,9 %	96,8 %
Baseline 16f, luz ambiental, 94 clases	2	98,40 %	67,0 %	89,4 %
Baseline 16f, iluminación dirigida, 94 clases	2	98,40 %	86,2 %	95,7 %
Baseline 16f, 97 clases	3	98,28 %	77,3 %	94,8 %
100 clases, 10 tomas, luz adecuada	4	99,00 %	99,6 %	100,0 %
100 clases, 10 tomas, luz reducida	5	99,00 %	94,6 %	98,7 %

En las iteraciones de 94 clases, el control de la iluminación del entorno elevó el acierto en cámara del 67,0 % al 86,2 % en Top-1 sobre la misma arquitectura, sin modificar los pesos del modelo.

Resolución temporal: 16 frente a 32 fotogramas

El Cuadro 7.5 sintetiza los experimentos realizados para determinar la longitud óptima de la ventana temporal de entrada.

Cuadro 7.5: Evaluación de la resolución temporal (91 clases, 50 videos por clase).

Iteración	**Arquitectura**	**Val. Top-1**	**Cám. Top-1**	**Cám. Top-3**
A, 16 frames	128 dim / 4 cabezas / 2 capas	97,36 %	61,5 %	83,5 %
B, 32 frames	128 dim / 4 cabezas / 2 capas	96,48 %	71,4 %	87,9 %
C, 32 frames	64 dim / 2 cabezas / 1 capa	97,36 %	67,0 %	90,1 %

A pesar de las ganancias iniciales observadas con 32 fotogramas, la incorporación de aumentaciones avanzadas de datos confirmó que la ventana de 16 fotogramas ofreció mayor capacidad de generalización sobre señas estáticas y redujo la redundancia temporal.

Búsqueda de hiperparámetros con Optuna

La exploración del espacio de hiperparámetros se estructuró en dos estudios compuestos por 40 ensayos cada uno, evaluados mediante validación cruzada de tres pliegues con poda de ensayos no prometedores. La función objetivo priorizó la minimización de la pérdida media de validación fuera de línea, sin incluir la exactitud directa frente a la cámara web.

El primer estudio abarcó un vocabulario de 91 clases con una ventana fija de 32 fotogramas. La configuración seleccionada por la búsqueda conservó la dimensión de 128 y

2 capas, al tiempo que duplicó las cabezas de atención a 8, redujo la tasa de *dropout* a 0,30, fijó el tamaño de lote en 16 e introdujo una tasa de aprendizaje de $3{,}77 \times 10^{-5}$. Tras el reentrenamiento final, este modelo obtuvo una exactitud de validación del 97,69 % y una pérdida de 0,087. En las pruebas frente a cámara, las métricas registraron un 66,0 % en Top-1 y un 84,6 % en Top-3. Dichos valores se situaron 5,4 puntos porcentuales por debajo de la iteración manual B en Top-1 y 5,5 puntos por debajo de la iteración C en Top-3.

El segundo estudio amplió el catálogo a 94 clases e incorporó la resolución temporal (ventanas de 8, 12 y 16 fotogramas) dentro del espacio de búsqueda. El ensayo de mayor rendimiento seleccionó una ventana de 12 fotogramas, 256 dimensiones, 2 cabezas de atención, 2 capas y un *dropout* de 0,57. En la evaluación fuera de línea, la pérdida descendió a 0,080 y la exactitud alcanzó el 97,78 %, lo que resultó 0,62 puntos porcentuales inferior al *baseline* de 16 fotogramas (98,40 %). Bajo condiciones controladas de iluminación en cámara, la exactitud Top-1 se registró en 80,9 %, frente al 86,2 % obtenido por el *baseline*. Esta degradación afectó de forma directa al alfabeto dactilológico, donde la exactitud en las letras cayó 14,8 puntos porcentuales (con fallos específicos en `A`, `N`, `V` e `Y`). Si bien la métrica Top-3 mostró un leve incremento en sesiones independientes, una evaluación apareada sobre tomas idénticas descartó la existencia de una ventaja estadística significativa.

Cuadro 7.6: Comparativa entre los ensayos de Optuna y las arquitecturas de referencia. Las cifras en cámara del segundo estudio corresponden a idénticas condiciones de iluminación que el *baseline* de 16 fotogramas.

Modelo	**Arquitectura**	**Val. Top-1**	**Cám. Top-1**	**Cám. Top-3**
Optuna v1, 32 fotogramas	128 / 8 / 2	97,69 %	66,0 %	84,6 %
Iteración B, 32 fotogramas	128 / 4 / 2	96,48 %	71,4 %	87,9 %
Optuna v2, 12 fotogramas	256 / 2 / 2	97,78 %	80,9 %	96,8 %
Baseline 16 frames, misma luz	128 / 4 / 2	98,40 %	86,2 %	95,7 %

El descarte de las configuraciones producidas por Optuna obedeció a tres razones fundamentales:

1. **Disparidad de dominio:** La pérdida de validación sobre secuencias previamente curadas no predice el comportamiento real frente a la cámara, puesto que las fluctuaciones de iluminación, los errores de encuadre y la inestabilidad en la extracción de puntos articulares permanecen ajenos al proceso de validación cruzada.

2. **Aislamiento de variables en la captura:** Las comparaciones iniciales solaparon los cambios arquitectónicos con variaciones en las condiciones de luz. Una vez estabilizada la iluminación ambiental, se constató que el incremento en el rendimiento provino de la calidad de la captura y no de los hiperparámetros.

3. **Criterio de desempeño del producto:** El diseño del sistema prioriza la exactitud Top-1 en vivo, con especial énfasis en el deletreo dactilológico. La arquitectura base (16 fotogramas, 128 dimensiones, 4 cabezas de atención y 2 capas) superó a los dos modelos generados por la optimización automática y constituyó la solución integrada en el prototipo final.

Impacto de la corrección de relación de aspecto

La estandarización del marco de captura desde proporciones 4 : 3 hacia 16 : 9 con bandas de relleno generó un impacto positivo directo en la clasificación, según se detalla en el Cuadro 7.7.

Cuadro 7.7: Efecto de la corrección de relación de aspecto sobre 36 tomas de letras y dígitos.

Configuración de entrada	**Exactitud Top-1**
Captura original en relación 4:3	66,7 %
Corrección analítica en eje y ($\times$1,333)	88,9 %
Corrección simulada en ejes y y z	75,0 %

La transformación del eje y aportó un incremento de 22,2 puntos porcentuales en exactitud. Este ajuste corrigió confusiones sistemáticas, como el caso de la letra `I` clasificada previamente como `ojo`.

Evaluación en cámara del modelo de 100 clases

El prototipo integrado carga el modelo de 100 clases, 16 fotogramas, 128 dimensiones, 4 cabezas y 2 capas. El entrenamiento usó 100 clips por clase (8 000 secuencias de entrenamiento y 2 000 de validación) y alcanzó una exactitud de validación del 99,0 %, con pérdida de 0,028. Ese corte es por clip dentro de cada clase, como se describe en la Sección 6.2.2. Las mismas tres personas pueden estar en el entrenamiento y en la validación.

La prueba en cámara consistió en dos sesiones sobre ese mismo archivo. La mano, el encuadre y el extractor se mantuvieron fijos; solo varió la iluminación de la habitación. Quien ejecuta el script anota la etiqueta de luz adecuada o reducida, dado que el programa no mide la iluminancia. Cada sesión recorrió las 100 señas con diez tomas consecutivas (1 000 inferencias por condición), en modo automático y con mano derecha, sin omitir tomas.

La cámara se solicita a 1280×720 y entrega 640×480. El script rellena bandas hasta 853×480, el mismo marco 16:9 de los videos de entrenamiento. El extractor corresponde a MediaPipe Holistic en navegador (`@mediapipe/holistic@0.5.1675471629`): pose y dos manos, sin malla facial, vector de 225 coordenadas y resampleo a 16 fotogramas. Entre inferencias se estableció un segundo de espera.

El Cuadro 7.8 resume los resultados. Las dos condiciones de iluminación cumplen el criterio de aceptación del plan (Top-1 mayor al 75 % y Top-3 mayor al 90 %).

Cuadro 7.8: Evaluación en cámara del modelo (`model_new_2026_10_20`). Diez tomas por seña, misma persona y mismo encuadre.

Métrica	**Luz adecuada**	**Luz reducida**
Top-1	99,6 % (996/1000)	94,6 % (946/1000)
Top-3	100,0 % (1000/1000)	98,7 % (987/1000)
Señas con 10 aciertos sobre 10	97	77
Errores que restan	4	54
Confianza media en aciertos	0,979	0,967
Confianza media en errores	0,632	0,771

Con luz adecuada, 97 señas registran diez aciertos en diez tomas. Restan cuatro errores y en todos ellos la seña solicitada ocupa el segundo lugar: `A` se confunde dos veces con `6`; `ahora_hoy` una vez con `tuyo`; `repetir` una vez con `esposo_a`. El subconjunto de 37 letras, ñ y dígitos alcanza un 99,5 % de Top-1.

Con luz reducida el Top-1 desciende 5,0 puntos y 20 señas dejan de completar diez aciertos sobre diez. El Top-3 permanece por encima del 98 %. Las caídas más marcadas en señas con diez o nueve aciertos con luz adecuada corresponden a `T` (3/10, hacia `I`), `como` (3/10, hacia `numero`), `años` (5/10, hacia `G`), `hola` (6/10, hacia `chau`) y `ahora_hoy` (6/10, con destinos diversos). En trece tomas la seña solicitada no entra al Top-3. En luz reducida la confianza media de los errores asciende a 0,771, indicador de que la red se equivoca con mayor certeza que con luz adecuada.

7.2.2. Resultados de los módulos de traducción semántica

La evaluación de los SLM cuantizados en formato GGUF (`Q4_K_M`) se ejecutó sobre la partición final del *corpus* (119 oraciones para el canal directo LSA → Español y 102 oraciones para el canal inverso Español → LSA).

Configuración de las métricas. Los índices de los Cuadros 7.9 y 7.10 salen de `src/semantic/metrics.py`. Ese archivo lo llama `eval_trial.py`, en las ramas de entrenamiento del Apéndice B. El corte de 119 y 102 filas, y el uso de varias respuestas por entrada, están en la Sección 6.3.2.

La preparación del texto no lematiza ni borra puntuación. BERTScore y COMET recortan espacios en los extremos y reemplazan una cadena vacía por un espacio, para no pasarle una cadena nula al modelo. BLEU, ROUGE-L y METEOR reciben la cadena tal cual, sin ese reemplazo.

Con varias referencias, BLEU es el BLEU de *corpus* de la biblioteca `evaluate`: usa la lista completa de respuestas de cada entrada. ROUGE-L, METEOR, BERTScore y COMET toman, en cada oración, el máximo entre esas referencias. El número de la tabla es la media de las 119 o de las 102 oraciones, en escala 0–100.

BERTScore usa la biblioteca `bert-score` con `lang=es` [38]. Ese parámetro carga `bert-base-multilingual-cased`. No se aplica ponderación IDF ni el reescalado con la línea de base del idioma. El valor informado es el F1 medio.

COMET usa el checkpoint `Unbabel/wmt22-comet-da` mediante `unbabel-comet` 2.2.7 [39]. Recibe la entrada del ejemplo, la salida y la referencia. En el canal directo la entrada son las glosas; en el inverso, el español. Ese checkpoint se entrenó con pares de texto escrito, no con glosas de LSA. Un puntaje alto no demuestra, por sí solo, que las glosas sean correctas en LSA.

Evaluación del canal directo (LSA a Español)

En el Cuadro 7.9 se presentan las métricas cualitativas y cuantitativas para el canal directo.

Cuadro 7.9: Canal directo (LSA → Español), resultados finales (119 oraciones de evaluación). Índices en escala 0–100.

Modelo	Exacto	BLEU	ROUGE-L	METEOR	BERTScore	COMET	ms
Qwen2.5 0,5B	92,44	95,58	95,78	95,51	99,16	97,47	141,05
Qwen2.5 1,5B	94,96	96,32	98,02	97,35	99,20	97,36	256,30
Qwen2.5 3B	95,80	97,88	98,39	97,84	99,20	97,56	514,38
Llama 3.2 1B	89,92	92,64	95,94	96,13	98,58	96,83	231,28
SmolLM2 1,7B	90,76	93,53	96,48	95,97	98,85	97,12	438,31

Evaluación del canal inverso (Español a LSA)

En el Cuadro 7.10 se detallan las métricas correspondientes al módulo de estructuración en glosas para el canal inverso.

Cuadro 7.10: Canal inverso (Español → LSA), resultados finales (102 oraciones de evaluación). Índices en escala 0–100.

Modelo	Exacto	BLEU	ROUGE-L	METEOR	BERTScore	COMET	ms
Qwen2.5 0,5B	87,25	91,29	96,20	91,25	98,86	94,28	155,97
Qwen2.5 1,5B	91,18	94,14	97,17	92,75	99,22	95,98	290,82
Qwen2.5 3B	89,22	92,51	96,85	92,71	99,21	95,45	537,58
Llama 3.2 1B	89,22	92,66	96,36	92,89	99,13	95,29	245,70
SmolLM2 1,7B	77,45	82,90	91,54	86,31	97,68	92,37	458,39

El modelo Qwen2.5 1,5B alcanzó el equilibrio óptimo en ambas direcciones, con un acierto exacto del 94,96 % en canal directo y del 91,18 % en canal inverso. Esos cuadros y la elección del modelo usan el mismo 15 % reservado, descrito en la Sección 6.3.2. Las medias de 256,30 ms y 290,82 ms salen de la evaluación local con la GPU de la estación de desarrollo. El equipo, el programa y la longitud de cada pasada están en la Sección 7.3.4.

7.2.3. Pruebas de la lógica de control del motor local

La batería de pruebas diseñada para validar la lógica del filtro `RepeatGate` y los tiempos de corte de enunciado produjo los resultados consignados en el Cuadro 7.11.

Cuadro 7.11: Resultados de las pruebas funcionales sobre la lógica de control del motor local.

Caso de prueba	**Entrada ingresada**	**Resultado emitido**
Dígitos repetidos	`1 1 1 2 2`	Aceptación total: `1 1 1 2 2`
Letra repetida tres veces	`A A A B`	Filtrado a máximo de dos: `A A B`
Rebote del clasificador	`HOLA HOLA`	Descarte de duplicado inmediato: `HOLA`
Repetición no consecutiva	`HOLA VER HOLA`	Conservación de la secuencia completa
Glosa repetida descartada a los 3 s	Glosa repetida	El cierre se desplaza a los 7 s
Movimiento continuo hasta 5 s	Sin glosa nueva	Cierre de enunciado a los 9 s
Deletreo dactilológico	`J U A N`	Conversión directa a «Juan.» sin consulta a nube
Deletreo con ambigüedad	`T I N A`	Derivación al traductor por contener `T` e `I`

7.2.4. Tratamiento de fallas de integración en Google Meet

Durante la evaluación sobre la interfaz de usuario en llamadas activas se identificaron inconvenientes técnicos que se solucionaron mediante intervenciones específicas de arquitectura, según se detalla en el Cuadro 7.12.

Cuadro 7.12: Anomalías detectadas en la integración con Google Meet y soluciones aplicadas.

Falla detectada	**Intervención de ingeniería aplicada**
Bloqueo del inicio de video en Meet por intercepción permanente de cámara.	Activación del reemplazo de la pista únicamente con el motor ILSA encendido.
Pérdida de enlace al recargar la extensión con la pestaña de la reunión abierta.	Recarga automática de la pestaña de Meet ante el reinicio del contexto de la extensión.
Detección de falsas manos provocada por movimiento del torso.	Exigencia de un número mínimo de cuadros consecutivos con mano detectada antes de abrir el gesto.
Interferencia en la captura por movimiento de personas en el fondo.	Delimitación del área de actividad basada exclusivamente en los puntos articulares de las manos.
Sobrecarga de CPU en la transferencia de imágenes hacia la ventana *offscreen*.	Reducción de la resolución de los cuadros transferidos a 240 píxeles de ancho con compresión ajustada.

7.3. Métricas de calidad

La evaluación objetiva del sistema se consolidó mediante métricas cuantificables agrupadas en cuatro dimensiones principales: desempeño en tiempo real, eficiencia en el uso de recursos, calidad semántica y confiabilidad.

7.3.1. Pruebas de latencia y end-to-end en llamada real

Se ejecutó una sesión de pruebas end-to-end sobre la extensión integrada en videollamada web activa, en la laptop de la Sección 7.1.3. El registro de quien seña y el de quien escucha cubren los dos canales. Quedan la latencia por etapa, `RAM_mb` y `CPU_percent` de ILSA, `Browser_RAM_mb` y `Browser_CPU_percent` de Edge, el heap de JavaScript, las glosas por oración, la vía (`llm` o léxica), el STT y las poses del inverso, y la tasa de fotogramas (FPS).

En el Cuadro 7.13 se sintetizan los valores promedios, mínimos y máximos recolectados durante las ejecuciones de prueba.

Cuadro 7.13: Resumen de métricas de perfilado y latencia registradas.

Métrica de ejecución	**Canal Directo (LSA → Esp)**	**Canal Inverso (Esp → LSA)**	**Unidad**
Eventos registrados	16	11	oraciones
Vía	15 LLM, 1 léxica	10 LLM, 1 léxica	oraciones
Glosas por oración	3,6 (1 – 10)	3,1 (1 – 6)	glosas
Caracteres de entrada	17,3 (4 – 36)	17,6 (4 – 25)	car.
Caracteres de salida	17,2 (5 – 27)	15,2 (4 – 25)	car.
Pausa de cierre (configurada)	$4126{,}5 \pm 68{,}0$	0,0	ms
Cola de inferencia	0,0	0,0	ms
Tiempo de inferencia remota	1527,9 (0,0 – 2525,5)	1407,3 (0,0 – 2224,1)	ms
Inferencia del modelo $\leq 2\,\mathrm{s}$	14/15	9/10	oraciones
Tiempo de transporte de red	299,1 (0,0 – 350,2)	278,5 (0,0 – 376,1)	ms
Tiempo de entrega / renderizado	0,4 (0,2 – 1,0)	988,9 (175,3 – 2945,1)	ms
Latencia total de procesamiento	1827,5 (0,3 – 2853,2)	2674,7 (175,3 – 4707,7)	ms
Espera hasta el subtítulo	5830,9 (3958,0 – 6866,0)	2674,7 (175,3 – 4707,7)	ms
Transcripción local	N/A	1658,0 (1015,7 – 3762,8)	ms
WAV de cada locución	N/A	64,7 (42,8 – 84,8)	KB
Descarga de poses	N/A	82,1 (0,0 – 479,0); 9/11 en caché	ms
Armado del clip	N/A	203,8 (58,7 – 499,0)	ms
Consultas de pose / aciertos de caché	N/A	116 / 68	n
Glosas sin archivo	N/A	3 oraciones	n
Ñ restaurada	N/A	2 oraciones	n
RAM de ILSA	114,4 (82,3 – 222,1)	247,7 (200,3 – 286,7)	MB
RAM de Edge	2062,9 (1793,9 – 2640,6)	2324,6 (2049,5 – 2487,6)	MB
CPU de Edge	199,0 (143,1 – 347,2)	174,4 (133,6 – 219,3)	%
Procesos de Edge	19 – 20	19	n
Heap JS de la página	105,5 (78,9 – 209,8)	164,1 (92,1 – 260,4)	MB
Heap JS de Holistic	3,9 (0,8 – 7,3)	N/A	MB
Tasa de procesamiento (Holistic)	0,1 – 12,4	N/A	FPS
Tasa de renderizado cliente	10,8 – 40,9	32,9 – 40,5	FPS
Cumplimiento RNF-01 ($< 2{,}0\,\mathrm{s}$)	93,8 % (15/16)	90,9 % (10/11)	aciertos

7.3.2. Evaluación del uso de recursos de hardware en el dispositivo cliente

A partir de los datos recopilados en cliente, se analizó el impacto del motor de inferencia local sobre los recursos de cómputo de la laptop objetivo sin GPU dedicada. `RAM_mb` y `CPU_percent` pertenecen al proceso de Python de ILSA (`os.getpid()`). `RAM_mb` es el working set de ese proceso. `CPU_percent` es el porcentaje de CPU del mismo proceso y supera el 100 % cuando el proceso usa varios núcleos. `Browser_RAM_mb` y `Browser_CPU_percent` suman los procesos `msedge.exe`: 19 o 20 en el directo, 19 en el inverso. La extensión envía, aparte, `js_heap_mb`: el heap de JavaScript de la página. Ese heap no es la RAM del proceso del navegador.

En el Cuadro 7.14 se comparan los requerimientos de memoria y CPU observados durante la ejecución continua de la extensión.

Cuadro 7.14: RAM y CPU de ILSA y de Edge en la laptop durante la prueba activa.

Canal	ILSA media	ILSA máx.	ILSA CPU	Edge media	Edge máx.	Edge CPU
Directo	114,4 MB	222,1 MB	9,33 % (60,1 %)	2062,9 MB	2640,6 MB	199,0 % (347,2 %)
Inverso	247,7 MB	286,7 MB	40,02 % (98,4 %)	2324,6 MB	2487,6 MB	174,4 % (219,3 %)

El análisis de perfilado muestra los siguientes comportamientos:

- **Eficiencia en memoria:** La RAM de ILSA en el canal directo descendió desde 222,1 MB hasta un régimen de 82,3 MB. En el canal inverso se estabilizó cerca de 247,7 MB. Edge quedó cerca de 2,1 GB en el directo y de 2,3 GB en el inverso.
- **Carga del procesador:** Durante el procesamiento del canal directo la CPU de ILSA permaneció en 9,33 % de promedio, con un pico de 60,1 % en la oración y de 176,3 % en el muestreo continuo. En el canal inverso, Vosk y el armado de las secuencias llevaron la media a 40,02 %, con un pico de 98,40 % en la oración y de 184,8 % en el muestreo continuo. La CPU de Edge tiene media 199,0 % en el directo y 174,4 % en el inverso.
- **Heap de la página:** En el directo el heap de Meet queda en 105,5 MB (78,9 – 209,8). El documento de Holistic queda en 3,9 MB (0,8 – 7,3). En el inverso el heap de la página sube a 164,1 MB (92,1 – 260,4).

7.3.3. Consumo de memoria y huella del sistema en servidor

La cuantización en formato GGUF de 4 bits permitió reducir la huella de memoria del modelo de lenguaje a valores compatibles con despliegues eficientes. En el Cuadro 7.15 y el Cuadro 7.16 se sintetizan el tamaño de los archivos y el consumo estimado de memoria RAM en inferencia con un contexto de 2048 *tokens*.

Cuadro 7.15: Dimensiones de los archivos cuantizados y compresión alcanzada.

Modelo	**16 bits (GiB)**	**GGUF (MiB)**	**GGUF (GiB)**	**bits/parámetro**
Qwen2.5 0,5B	0,913	379,4	0,370	6,5
Qwen2.5 1,5B	2,869	940,4	0,918	5,1
Qwen2.5 3B	5,756	1,840,5	1,797	5,0
Llama 3.2 1B	2,291	770,3	0,752	5,3
SmolLM2 1,7B	3,185	1,006,7	0,983	4,9

Cuadro 7.16: Estimación del consumo de memoria RAM en inferencia (contexto de 2048 *tokens*).

Modelo	**Caché KV (GiB)**	**16 bits + caché (GiB)**	**GGUF + caché (GiB)**
Qwen2.5 0,5B	0,023	0,936	0,394
Qwen2.5 1,5B	0,055	2,923	0,973
Qwen2.5 3B	0,070	5,826	1,867
Llama 3.2 1B	0,062	2,354	0,815
SmolLM2 1,7B	0,375	3,560	1,358

La columna de caché KV sale de un contexto de 2048 *tokens* en fp16, dos bytes por elemento [45]. El primer 2 separa la clave del valor:

$$KV = 2, n_{\text{layers}}, n_{\text{kv}}, d_{\text{head}}, 2048 \times 2. \tag{7.1}$$

n_{layers} es el número de capas, n_{kv} el de cabezas de clave y valor, y d_{head} el largo de cada vector. El Cuadro 7.17 aplica esa cuenta. El cociente en GiB es el número de bytes dividido por 1024^3. El Cuadro 7.16 guarda ese cociente con tres decimales.

Cuadro 7.17: Datos de la caché KV para 2048 *tokens* en fp16.

Modelo	**Capas**	**Cabezas KV**	d_{head}	**Bytes**	**GiB**
Qwen2.5 0,5B	24	2	64	25,165,824	0,023
Qwen2.5 1,5B	28	2	128	58,720,256	0,055
Qwen2.5 3B	36	2	128	75,497,472	0,070
Llama 3.2 1B	16	8	64	67,108,864	0,062
SmolLM2 1,7B	24	32	64	402,653,184	0,375

Qwen2.5 0,5B da 0,0234375,GiB y el cuadro lo deja en 0,023. Qwen2.5 1,5B da 0,0546875,GiB y el cuadro lo deja en 0,055. Qwen2.5 3B da 0,0703125,GiB y el cuadro lo deja en 0,070. Llama 3.2 1B da 0,0625,GiB y el cuadro lo deja en 0,062. SmolLM2 1,7B da 0,375,GiB exactos. En Qwen2.5 0,5B la cabeza mide 64. En Qwen2.5 1,5B y 3B mide 128. En Llama 3.2 1B, $2048/32 = 64$. En SmolLM2 1,7B, $2048/32 = 64$, y las 32 cabezas de atención son también las de KV.En la réplica de producción desplegada en Cloud Run, ambos canales cargan Qwen2.5 1,5B con ese contexto de 2048. La columna GGUF más caché da 0,973,GiB para ese modelo, y los dos canales suman 1,946,GiB. Esa suma es el archivo medido más la caché KV en fp16. El buffer de activaciones de la Ecuación 6.6 queda fuera, y la cifra no es el RSS del proceso. Llama 3.2 1B queda en disco, a 0,815,GiB con su caché por canal, y entra en memoria solo si el modelo principal no carga.

7.3.4. Comparativa de rendimiento según arquitectura de hardware

La toma del 30 de octubre de 2026 corrió en la estación de desarrollo. Esa PC reúne un procesador AMD Ryzen 7 8700F de ocho núcleos a 4,10 GHz, 16 GB de memoria RAM a 4800 MT/s, una GPU NVIDIA GeForce RTX 5070 de 12 GB y un almacenamiento de 1,37 TB. El navegador de esa toma es Microsoft Edge. `RAM_mb` y `CPU_percent` son el proceso de ILSA. `Browser_RAM_mb` y `Browser_CPU_percent` suman los procesos `msedge.exe`: 31 en el canal directo, y 30 o 31 en el inverso. El heap de JavaScript de la página es otro número, y el del documento donde corre Holistic es otro.

El Cuadro 7.18 usa el tramo del registro que trae esas columnas del navegador. Hay 25 oraciones en el canal directo (20 pasan por el modelo, 3 quedan en forma literal y 2 en forma léxica) y 18 en el inverso (15 por el modelo y 3 léxicas). La cola de inferencia queda en 0 ms en las dos. La pausa del directo mide de 4000 a 4050 ms, con media 4025 ms.

Cuadro 7.18: Toma del 30 de octubre de 2026 en la estación de escritorio. ILSA es el proceso de Python. El navegador es la suma de los procesos de Edge.

Métrica	**Canal directo**	**Canal inverso**
Glosas por oración, media (mín.–máx.)	3,3 (1 – 9)	3,1 (1 – 5)
Caracteres de entrada, media (mín.–máx.)	12,9 (1 – 28)	15,0 (4 – 30)
Caracteres de salida, media (mín.–máx.)	12,6 (2 – 21)	15,2 (4 – 22)
Inferencia del modelo, media (mín.–máx.)	1521,3 ms (942,0 – 2178,1)	1545,9 ms (1004,4 – 2088,8)
Inferencia del modelo ≤ 2 s	17/20	14/15
Transporte del modelo, media (mín.–máx.)	291,5 ms (267,0 – 361,1)	301,1 ms (272,2 – 403,1)
Entrega, media (mín.–máx.)	0,1 ms (0,0 – 0,3)	854,3 ms (61,4 – 1532,6)
Latencia sin la pausa, media (mín.–máx.)	1450,3 ms (0,1 – 2445,3)	2393,4 ms (61,4 – 3759,9)
Espera hasta el subtítulo, media (mín.–máx.)	5475,4 ms (4007,2 – 6487,3)	2393,4 ms (61,4 – 3759,9)
RAM de ILSA, media (mín.–máx.)	551,1 MB (536,8 – 554,8)	664,7 MB (549,0 – 686,9)
CPU de ILSA, mediana (máximo)	2,4 % (746,4 %)	3,5 % (83,1 %)
RAM de Edge, media (mín.–máx.)	3685,0 MB (3525,0 – 3953,5)	3987,2 MB (3787,1 – 4082,7)
CPU de Edge, media (mín.–máx.)	187,7 % (48,8 – 390,8)	126,1 % (77,0 – 222,3)
Heap JS de la página, media (mín.–máx.)	95,4 MB (76,1 – 124,1)	125,8 MB (84,3 – 174,4)
Heap JS de Holistic, media (mín.–máx.)	4,8 MB (1,1 – 8,7)	N/A
FPS del lienzo	179,7 – 201,3	192,0 – 200,1
FPS de Holistic	22,4 – 28,9; una muestra en 0	N/A
Clasificador, mediana (máximo)	1,4 ms (80,4 ms)	N/A
Transcripción local, media (mín.–máx.)	N/A	417,4 ms (261,6 – 1040,2)
WAV de cada locución, media (mín.–máx.)	N/A	53,0 KB (34,8 – 75,3)
Armado del clip, media (mín.–máx.)	N/A	37,2 ms (4,1 – 116,5)
Consultas de pose / aciertos de caché	N/A	143 / 68
Ñ restaurada	N/A	1 oración

La meta de 2 s cubre la inferencia del modelo. En esas filas quedan 17 de 20 y 14 de 15. Las 5 oraciones locales del directo y las 3 del inverso tienen inferencia 0, y el registro las cuenta dentro del techo: 22 de 25 y 17 de 18. En 10 de 18 oraciones del inverso la descarga de poses queda en 0 ms, con el archivo ya en caché. Las otras 8 tardan de 320,9 a 1254,4 ms. Cuatro oraciones tienen una glosa sin archivo y una restaura la Ñ. De 14 locuciones, tres no devuelven texto. En la laptop las 11 locuciones del inverso devolvieron texto (4 a 25 caracteres).

El Cuadro 7.14 incluye Edge en la laptop. En la PC, ILSA queda cerca de 551 MB en el directo y de 665 MB en el inverso. Edge queda cerca de 3,7 GB y de 4,0 GB. La CPU de ILSA en la PC tiene mediana 2,4 % en el directo, con un muestreo de 746,4 %, y mediana

3,5 % en el inverso, con máximo 83,1 %. La CPU de Edge tiene media 187,7 % y 126,1 %. El lienzo de la PC va de 179,7 a 201,3 FPS en el directo y de 192,0 a 200,1 FPS en el inverso. Holistic en la PC, canal directo, va de 22,4 a 28,9 FPS. En la laptop el mismo extractor va de 0,1 a 12,4 FPS. Esas tasas describen el dibujo y el extractor.

7.3.5. Señas aceptadas y rechazadas en llamada

El lado de quien señа guarda una fila por cada toma: glosa, confianza Top-1 a Top-3, margen, tiempos del clasificador y de Holistic, fotogramas y payload. El umbral de 0,75 deja la seña fuera de la frase. El Cuadro 7.19 compara las 83 tomas de la laptop con las 98 de la PC.

Cuadro 7.19: Señas del canal directo en la toma del 30 de octubre de 2026.

Métrica	**Laptop**	**PC**
Señas registradas	83	98
Entran a la frase	58	83
Quedan fuera (confianza $< 0{,}75$)	25	15
Clases distintas (todas / aceptadas)	41 / 37	37 / 35
Confianza media en aciertos	0,981 (0,806 – 1,000)	0,956 (0,756 – 1,000)
Confianza media en rechazos	0,594 (0,345 – 0,741)	0,587 (0,298 – 0,741)
Margen Top-1 − Top-2, media	0,770	0,830
Clasificador, mediana (máximo)	7,2 ms (88,2)	1,4 ms (80,4)
Preparación del tensor, media	8,3 ms	2,4 ms
Holistic por seña, media (mín.–máx.)	209,1 ms (176 – 271)	27,2 ms (18 – 40)
JPEG por seña, media	31,2 ms (1 – 81)	N/A
Captura, media (mín.–máx.)	1616,7 ms (845 – 2738)	1461,1 ms (776 – 2705)
Fotogramas enviados, media (mín.–máx.)	8,5 (5 – 13)	16 (fijo)
Fotogramas crudos, media (mín.–máx.)	8,5 (5 – 13)	38,6 (21 – 72)
Payload JPEG, media (mín.–máx.)	29,4 KB (16,3 – 54,1)	N/A
Mano dominante	derecha en las 83	derecha en las 98

En la laptop el rechazo es 25 de 83. Las glosas que más caen por confianza son `mio` (5), `3` (4) y `6` (3). En la PC el rechazo es 15 de 98, con más caídas en `O` (3), `6` (2) y `hermano_a` (2). La laptop envía de 5 a 13 fotogramas; la PC recorta 16 de 21 a 72 crudos. Holistic por seña pasa de 209 ms en la laptop a 27 ms en la PC. El clasificador queda en 7,2 ms de mediana en la laptop y en 1,4 ms en la PC. La primera seña de cada máquina es la más lenta: 88,2 ms y 80,4 ms.

Capítulo 8

Evaluación

El presente capítulo expone la evaluación técnica, crítica e integradora del sistema desarrollado. Se analiza el nivel de cumplimiento de los objetivos planteados al inicio del proyecto sobre la base de las evidencias obtenidas en las etapas de experimentación, prueba y validación. Asimismo, se detallan las limitaciones metodológicas y tecnológicas observadas durante el desarrollo, y se examinan los riesgos técnicos y operativos inherentes a la solución con sus respectivas estrategias de mitigación.

8.1. Cumplimiento de objetivos

La valoración de los resultados se realiza a través de la trazabilidad directa entre los objetivos definidos en el Capítulo 2, los requerimientos especificados en el Capítulo 4 y las evidencias empíricas recolectadas en los ensayos de validación.

8.1.1. Evaluación del objetivo general

El objetivo general consistió en desarrollar un prototipo funcional de traducción bidireccional entre la LSA y el español rioplatense en tiempo pseudo-real, integrado en una plataforma de videoconferencia mediante una arquitectura híbrida cliente-nube.

El cumplimiento de este objetivo se alcanzó mediante la construcción exitosa de la extensión para navegadores Chromium conectada al motor local ILSA y a los servicios en la nube. El sistema procesa la interacción en dos sentidos. En el canal directo, la sesión integrada puso la oración en el video compuesto; con Audio o Ambos, ILSA sintetiza un WAV y el script de la página lo mezcla en la pista de audio de Meet. En el canal inverso traduce voz o texto a la animación de un avatar esquelético. La visión queda en el cliente y la inferencia de lenguaje, en la nube, sin enviar video a los servicios de traducción y de poses (Sección 5.1.1).

8.1.2. Evaluación de los objetivos específicos

A continuación se analiza el grado de logro de cada uno de los objetivos específicos:

1. **Implementación del módulo local de visión y clasificación gestual:** *Estado: Alcanzado.* Se desarrolló el pipeline basado en MediaPipe Holistic para la extracción de 225 coordenadas articulares normalizadas y la red neuronal *TinySkeletonClassifier*. Las 2 000 secuencias de la validación fuera de línea alcanzaron un 99,0 % de exactitud

y sirvieron para validar y elegir el modelo. La medición de cámara, hecha después de esa elección, dio un 99,6 % de acierto con luz adecuada y un 94,6 % con luz reducida. Las dos condiciones superan el 75 % del requerimiento RNF-03.

2. **Diseño e integración del servicio semántico para el canal directo (LSA → Español):** *Estado: Alcanzado.* Se ajustó y desplegó el modelo Qwen2.5 1,5B mediante la técnica LoRA y cuantización GGUF `Q4_K_M`. En el Cuadro 7.9, sobre oraciones fuera del conjunto de entrenamiento, el acierto exacto es 94,96 %, el BERTScore es 99,20 y el COMET es 97,36. La inferencia remota promedió 1444,4 ms en la sesión integrada. Ese tiempo se juzga con RNF-01, aparte de RF-03.

3. **Diseño e integración del servicio semántico para el canal inverso (Español → LSA):** *Estado: Alcanzado.* Se estructuró el canal de retorno para reorganizar oraciones en español hacia el orden manual simplificado del prototipo. El modelo ajustado alcanzó un acierto exacto del 91,18 % en las pruebas del laboratorio semántico, en el requerimiento RF-06. La media de inferencia remota, 1368,2 ms, se juzga con RNF-01.

4. **Implementación del módulo de visualización gestual mediante avatar:** *Estado: Alcanzado.* Se diseñó el motor de animación cinemática basado en archivos binarios `.pose`. El sistema reproduce hasta 24 clips por oración con suavizado exponencial y puentes de transición interpolados. Ante la ausencia de un archivo de seña, deletrea o conserva el texto, como fija RF-07. La cola de reproducción admite 20 animaciones.

5. **Diseño de una arquitectura escalable en la nube:** *Estado: Alcanzado.* Se diseñó la arquitectura elástica, con balanceo de carga, contenedores sin estado y escalado horizontal automático (Sección 5.6 y Apéndice C). Ese diseño está documentado y queda realizado. Por separado, la prueba con carga concurrente sigue pendiente: la validación usó una instancia única por servicio en Google Cloud Run (`-max-instances 1`), según la restricción RC-02.

6. **Resguardo de la privacidad de los datos biométricos:** *Estado: Alcanzado.* La extracción de puntos y la clasificación corren en el cliente. El límite de enviar solo texto rige para el traductor y para el servidor de poses: hacia el traductor sale texto y hacia el servidor de poses, un nombre de glosa o las letras de un deletreo. El video compuesto sale del equipo hacia Meet, como fija RNF-02.

7. **Desarrollo de la extensión web e integración con Google Meet:** *Estado: Alcanzado.* Se construyó la extensión bajo el estándar Manifest V3 articulada con el motor local ILSA mediante la arquitectura desacoplada de entornos *offscreen* y *sandbox*. La sesión integrada inyectó el lienzo en Google Meet (`captureStream()`), en cumplimiento de RF-08. RF-09 se comprobó al iniciar la aplicación, al Encender un modo y al Reiniciar: con el traductor y el servidor de poses en marcha, vuelve a la pantalla de carga y después abre el panel principal. Sin red, Encender mantiene la carga, muestra Reintentar y deja el panel principal cerrado. Al volver la red, Reintentar repite la consulta y abre el panel. Al Apagar, la sesión se detiene y el panel queda a la vista, sin esa consulta. RNF-05 se comprobó en una llamada de dos personas, con el popup de la extensión cerrado y la tarjeta del HUD en su posición inicial. En una captura de 1920×1080 esa tarjeta mide 382×77 píxeles, 15 px desde

el borde izquierdo y 156 px desde el borde superior; queda 49 px por encima del recuadro de video. En modo seña y en modo oyente, en la pantalla de quien usa el sistema y en la de la otra persona, la cara y las manos del interlocutor quedaron libres.

8. **Evaluación integral del sistema y medición de desempeños:** *Estado: Parcial.* Se midió el acierto en cámara, la fidelidad semántica de ambos canales y la latencia por etapas. La evaluación queda incompleta: faltan los tiempos de espera en el servidor, registros que permitan repetir las mediciones y una revisión lingüística.

El Cuadro 8.1 sintetiza la trazabilidad entre los objetivos, los requerimientos asociados y los resultados obtenidos.

Cuadro 8.1: Matriz de trazabilidad entre objetivos, requerimientos y resultados.

Objetivo específico	**Requerimientos**	**Evidencia / Resultado obtenido**	**Estado**
1. Clasificador gestual	RF-02, RNF-03	Las 2 000 secuencias (99,0 %) sirvieron para elegir el modelo. Cámara, después de esa elección: Top-1 de 99,6 % con luz adecuada y 94,6 % con luz reducida. RNF-03 pide más del 75 % en luz doméstica.	Alcanzado
2. Canal directo	RF-03	Cuadro 7.9, Qwen2.5 1,5B: exacto 94,96 %; BERTScore 99,20; COMET 97,36. Latencia media de 1444,4 ms.	Alcanzado
3. Emisión al oyente	RF-04	Subtítulo: la oración queda en la franja del video compuesto de la sesión integrada. Audio y Ambos: ILSA sintetiza un WAV; el script de la página lo mezcla en la pista de audio de Meet.	Alcanzado
4. Canal inverso	RF-06	Cuadro 7.10: Exactitud exacta de 91,18 %. Latencia media de 1368,2 ms.	Alcanzado
5. Avatar e integración	RF-07	Hasta 24 clips por oración, deletreo si falta el archivo y cola de 20. En la sesión integrada, el renderizado del cliente en el canal inverso fue de 58,2 a 60,5 FPS.	Alcanzado
6. Arquitectura nube	RNF-04, RC-02	Diseño elástico documentado y realizado. La prueba con carga concurrente queda pendiente: Cloud Run con una réplica.	Alcanzado
7. Privacidad biométrica	RNF-02	Puntos clave en el cliente. Solo el traductor y el servidor de poses reciben texto, un nombre de glosa o letras. El video compuesto sale hacia Meet.	Alcanzado
8. Extensión web	RF-08, RF-09	RF-08: la sesión integrada inyectó el lienzo en Meet. RF-09: al iniciar, al Encender o al Reiniciar, con los dos servicios, vuelve a la carga y abre el panel. Sin red, Encender deja Reintentar y el panel cerrado; al volver la red, Reintentar abre el panel.	Alcanzada
9. Usabilidad	RNF-05	Llamada de dos personas, popup cerrado. Tarjeta inicial 382×77 px, arriba a la izquierda (15 px, 156 px) en 1920×1080, 49 px sobre el video. En modo seña y en modo oyente, en las dos pantallas, la cara y las manos quedaron libres.	Alcanzado
10. Evaluación integral	RNF-01, RC-05	RNF-01 parcial: solo la inferencia. Directo: media 1444,4 ms, 7 de 9 bajo 2 s. Inverso: media 1368,2 ms, 6 de 7 bajo 2 s. Faltan la espera en el servidor, registros para repetir las mediciones, la revisión lingüística y el vínculo de algunos requisitos con sus pruebas. RC-05: mínimo propuesto. La laptop medida está en la Sección 7.1.3.	Parcial

8.2. Limitaciones

El desarrollo y el desempeño del prototipo se encuentran condicionados por delimitaciones técnicas, metodológicas y tecnológicas acordadas para acotar el alcance del proyecto de grado. La explicitación de estos factores establece los bordes reales del sistema:

- **Recorte léxico y ausencia de gramática no manual:** El vocabulario soportado por el clasificador se acota estrictamente a 100 señas aisladas (Apéndice A). Asimismo, el pipeline excluye la gesticulación facial, los movimientos de cejas y la postura del torso como rasgos gramaticales. Por lo tanto, el sistema no recupera la modulación afectiva o interrogativa que reside exclusivamente en los canales no manuales de la LSA.

- **Requisito de separación de gestos en encuadre:** El algoritmo de detección de pausas exige retirar la mano del encuadre de la cámara entre la ejecución de cada seña. Esta condición operativa resulta indispensable para segmentar correctamente los gestos aislados, aunque impone una mecánica de señado discontinua que difiere de la fluidez natural de la LSA.

- **Condiciones de captura y dependencia de MediaPipe:** La calidad del reconocimiento gestual depende de la correcta extracción de *landmarks* por parte de MediaPipe Holistic. Situaciones de oclusión severa entre las manos, desenfoque por movimiento rápido, iluminación deficiente o encuadres fuera del plano del torso incrementan la tasa de falsos positivos o la degradación de las coordenadas articulares.

- **Delimitación estricta de límites temporales:** El prototipo exige una pausa fija de inactividad manual de 4 segundos para determinar la finalización de una frase en el canal directo. Esta restricción introduce un tiempo de espera estructural previo al envío de la solicitud hacia la nube, lo que impide un flujo conversacional verdaderamente fluido o ininterrumpido.

- **Representación cinemática simplificada del avatar:** El ensamblado del avatar opera mediante la interpolación geométrica entre núcleos de gestos previamente grabados. Esta aproximación atenúa la discontinuidad espacial entre señas, pero no replica los fenómenos biomecánicos complejos de la coarticulación gestual presentes en el movimiento humano natural.

- **Subtítulos espejados en la miniatura del modo oyente:** En el modo oyente, la persona oyente ve su cámara y el avatar en la miniatura local. En esa miniatura, los subtítulos le salen espejados. La persona sorda, que es quien tiene que comprenderlos, los recibe al derecho (Sección 6.5.2). La limitación queda en la vista propia de quien usa el sistema.

- **Dependencia de conectividad de red y penalización de arranque en el prototipo:** El empaquetado del motor local ILSA se optimizó para el sistema operativo Windows y navegadores basados en Chromium. La ejecución concurrente de la videollamada, la extracción de puntos clave mediante WebAssembly y la clasificación en CPU se validaron experimentalmente en la laptop de la Sección 7.1.3. A partir de estas mediciones, se estimó como perfil operativo mínimo lo propuesto en la Sección 4.3.1.

8.3. Riesgos detectados y estrategias de mitigación

El análisis del prototipo en un escenario proyectado hacia la producción evidencia la presencia de riesgos técnicos, operativos y evolutivos. En el Cuadro 8.2 se sistematizan los principales factores identificados junto con sus correspondientes líneas de mitigación y propuestas de mejora ingenieril.

Cuadro 8.2: Análisis de riesgos técnicos, operativos y evolutivos con estrategias de mitigación.

Categoría	**Riesgo identificado**	**Descripción e impacto**	**Estrategia de mitigación / Línea de mejora**
Seguridad y Privacidad	Vulnerabilidad en la API local no autenticada	La escucha del servidor FastAPI en la red local sin *tokens* de autenticación expone endpoints a otros procesos del mismo sistema cliente.	Implementación de autenticación mediante *tokens* efímeros (JWT) intercambiados de forma segura durante el arranque de la extensión.
Evolución y Obsolescencia	Cambios en el DOM de Google Meet y APIs web	Actualizaciones en la plataforma de videollamada o restricciones adicionales en Manifest V3 pueden romper la inyección de scripts.	Abstracción de los selectores DOM en capas de desacoplamiento y mantenimiento periódico de la extensión en el repositorio.
Mantenibilidad	Dificultad para incorporar nuevas señas	El reentrenamiento del clasificador y la captura de nuevos archivos `.pose` requiere trabajo manual intensivo.	Construcción de herramientas de captura automatizadas y canalizaciones de integración continua (*MLOps*) para modelos de visión.
Traducción	Traducción incorrecta	El modelo en servicio acierta de forma exacta el 94,96 % de las oraciones del canal directo y el 91,18 % del inverso. Una salida distinta de la referencia puede cambiar quién interviene, el tiempo, la negación o la pregunta.	Revisión con personas sordas de las salidas que no conservan la proposición prevista.
Reconocimiento	Seña desconocida aceptada	Una seña ausente del vocabulario puede parecerse a una clase conocida y salir con confianza de al menos 0,75. El indicador la muestra en cian y la glosa incorrecta entra a la frase.	Confirmación de quien seña antes de incorporar esa glosa. El umbral de 0,75 solo frena la confianza baja.

Cuadro 8.2 (continuación)

Categoría	**Riesgo identificado**	**Descripción e impacto**	**Estrategia de mitigación / Línea de mejora**
Reproducción	Mensaje recortado o descartado	Cada enunciado anima hasta 24 glosas y 1200 fotogramas; el resto queda en texto. La cola guarda hasta 20 animaciones, no interrumpe la que está en curso y, al llenarse, descarta la más antigua.	Aviso visible cuando el dibujo se recorta o la cola descarta un bloque. El texto de las glosas que no entran al dibujo se conserva.
Emergencia	Avatar no entendido en un auxilio	En un pedido de auxilio, el esqueleto o el deletreo pueden no entenderse. Esa comprensión no se evaluó, y el deletreo no asegura el significado.	Prueba con personas sordas de un pedido de auxilio visto en el avatar. Las glosas de ese pedido quedan también en texto sobre el video.

Capítulo 9

Conclusiones y trabajos futuros

El presente capítulo sintetiza los resultados técnicos del proyecto, evalúa el cumplimiento de los objetivos y analiza el impacto social de la solución. Asimismo, proyecta las líneas de continuidad e investigación futura derivadas de las limitaciones y aprendizajes observados durante el desarrollo del sistema.

9.1. Conclusiones

El desarrollo de este Trabajo Final de Grado permitió validar la factibilidad técnica y operativa de un sistema de traducción bidireccional entre la LSA y el español rioplatense en tiempo pseudo-real, concebido para operar sobre plataformas de videoconferencia de uso masivo como Google Meet. La arquitectura híbrida propuesta dejó el procesamiento de visión en el equipo cliente y la inferencia de lenguaje en la nube. Las pruebas técnicas de la Sección 7.3.1 midieron las latencias de la sesión integrada. La experiencia de los usuarios no se evaluó.

9.1.1. Síntesis del cumplimiento de objetivos

La integración del sistema se contrasta aquí con el objetivo general y las metas específicas planteadas al inicio del proyecto. El detalle de cada medición está en el Capítulo 8.

- **Canal directo (LSA → Español):** El pipeline de visión local sustentado en *TinySkeletonClassifier* demostró una capacidad de reconocimiento gestual sobresaliente (99,6 % de acierto Top-1 en cámara real bajo iluminación adecuada). Por su parte, la adaptación del modelo de lenguaje *Qwen2.5 1,5B* resolvió con fluidez la brecha sintáctica entre las glosas y el español rioplatense (94,96 % de acierto exacto). En la sesión integrada de la Sección 7.3.1, con la extensión en una videollamada activa y el servicio ya en marcha, la inferencia remota de ese canal promedió 1444,4 ms sobre 9 oraciones. El cumplimiento de RNF-01 es parcial: 7 de 9 quedaron bajo 2 s y 2 superaron ese límite, hasta 2372,9 ms. Esa meta cubre solo la inferencia.
- **Canal inverso (Español → LSA) y visualización:** El módulo semántico de retorno reestructuró la voz o el texto del interlocutor oyente hacia el orden manual simplificado del prototipo (91,18 % de acierto exacto). Los clips del avatar se reproducen a 30 fotogramas por segundo. El empalme es un puente interpolado: 4 fotogramas cuando las muñecas del final de una seña y del inicio de la siguiente están cerca, y 8 cuando están lejos (Sección 6.5.2). La entrada desde la postura de espera

y la salida hacia ella duran cerca de seis fotogramas. La comprensión del mensaje se evalúa aparte de esa cadencia, y este trabajo no la incluye. Si falta el archivo de una glosa, el sistema pasa al deletreo dactilológico.

- **Arquitectura, privacidad e integración web:** Se concretó la ejecución sobre Google Meet mediante una extensión Manifest V3 acoplada al motor local ILSA. Meet recibe el video ya compuesto, con la cámara, el subtítulo y, en modo oyente, el esqueleto, y el audio de la llamada. La extracción de puntos clave corre en la misma computadora. El traductor en la nube recibe solo texto, glosas u oración, y el servidor de movimientos recibe el nombre de una glosa o las letras de un deletreo. La imagen y el video no salen hacia esos dos servicios, como fija RNF-02. Si bien el despliegue en la nube se limitó a una instancia única por servicio en Google Cloud Run en cumplimiento del alcance presupuestario de las pruebas, se dejó completamente documentado el diseño elástico para producción.

9.1.2. Aportes técnicos e ingenieriles

El proyecto aporta decisiones de diseño y arquitecturas aplicadas que resuelven problemas concretos de la ingeniería de *software* y el aprendizaje profundo en dispositivos de borde:

- **Partición eficiente de cómputo híbrido:** La delegación de los modelos de lenguaje hacia la nube y la conservación del pipeline de visión en el cliente permitió ejecutar el sistema en hardware estándar sin placa gráfica dedicada. Los 96 a 231 MB son `RAM_mb` del proceso ILSA en la laptop (Sección 7.3.2): el working set de ese PID, 96 MB en el canal directo y cerca de 231 MB en el inverso. En ese proceso están el clasificador y, en el canal inverso, Vosk. Chrome no entra en esa columna. `js_heap_mb` es otro campo: el heap de JavaScript de la página, no la RAM total del navegador, de MediaPipe ni de la videollamada.

- **Mecanismos de integración transparente en la Web:** La arquitectura basada en entornos *offscreen* y *sandbox* dentro de Manifest V3 resolvió las restricciones de ejecución de código dinámico en navegadores modernos. La inyección de un lienzo sintético (*Canvas captureStream*) permitió interceptar la llamada de Google Meet sin alterar la estabilidad del dispositivo de captura original ni requerir permisos especiales en la plataforma.

- **Estrategias de robustez en el procesamiento temporal:** La compuerta de filtrado de repetición (*RepeatGate*) frena la repetición inmediata y la corrección de relación de aspecto alinea el marco de captura con el de entrenamiento. En la prueba de cámara de la Sección 7.2.1 quedan errores: cuatro con luz adecuada y 54 con luz reducida. Con luz reducida la confianza media de esos errores es 0,771 y supera el umbral de 0,75.

9.1.3. Impacto social y marco regulatorio

El aporte técnico de este trabajo es el prototipo de traducción bidireccional en tiempo pseudo-real sobre videoconferencia. La Ley 27.710 reconoce a la LSA como lengua natural y el derecho de las personas sordas a comunicarse en su lengua en los ámbitos sociales [1]. Una mayor autonomía en salud, trámites y educación virtual, y un acceso más equitativo

cuando falta la interpretación humana, son beneficios sociales posibles de ese aporte. Sin pruebas con usuarios, esa mejora de autonomía y de acceso todavía no está demostrada.

9.2. Trabajos futuros

Las líneas de continuidad y expansión del sistema surgen de manera coherente a partir de las limitaciones identificadas en la etapa de evaluación y de la necesidad de escalar la solución hacia entornos de producción masiva. Se proponen los siguientes ejes de desarrollo:

9.2.1. Inclusión de la comunidad sorda y validación participativa

Incorporar a personas nativas de la comunidad sorda y a especialistas en LSA en el diseño, la anotación del *corpus* y la validación del sistema. Esa participación es una mejora que deberá comprobarse con dos pruebas hechas con personas sordas. En la de comprensión, la persona verá la salida del prototipo sin la oración en español y dirá quién interviene, de qué se habla, en qué tiempo, si hay negación y si hay pregunta. En la de aceptación, indicará si acepta esa seña como LSA para ese sentido. El caso será satisfactorio cuando lo que dice coincida con la proposición prevista y acepte la seña.

9.2.2. Ampliación del dataset y captura multicámara

Regrabar y expandir el conjunto de datos gestual mediante tomas simultáneas desde múltiples ángulos y perspectivas espaciales. La integración de registros multicámara permitirá entrenar clasificadores con mayor invariancia frente a oclusiones mutuas de las manos, cambios en el punto de vista de la lente y variaciones en la postura del usuario respecto del plano frontal.

9.2.3. Incorporación de rasgos no manuales y reconocimiento continuo

Evolucionar desde la clasificación de señas aisladas hacia modelos de reconocimiento de lenguaje de señas continuo (CSLR). Esta etapa contempla la incorporación de la malla facial para capturar la gesticulación de boca, cejas e inclinación de torso, elementos indispensables para interpretar la modulación interrogativa, afectiva y temporal completa de la LSA.

9.2.4. Evolución hacia un avatar tridimensional (3D)

Sustituir la representación esquelética bidimensional actual por un avatar tridimensional con renderizado volumétrico y dinámicas de deformación de malla. Motores como Three.js o Babylon.js dibujan la profundidad, la rotación de las muñecas y la orientación de las palmas. La mano se comprobará al poner la configuración de los dedos, la rotación de la muñeca y la orientación de la palma junto a la pose de referencia. La profundidad se tomará de las tres tomas ya previstas para reconstruirla. La comprensión se comprobará con personas sordas y especialistas en LSA: verán el clip sin el texto en español y nombrarán la seña. El acierto exige que ese nombre coincida con la glosa prevista.

9.2.5. Despliegue del clúster elástico multinodo en la nube

Implementar físicamente la infraestructura elástica proyectada en el diseño conceptual. Esto abarca la configuración de balanceadores de carga, la orquestación de contenedores sin estado con autosescalado horizontal según la longitud de cola de peticiones, y la verificación de la capacidad de respuesta frente a cargas de tráfico concurrente simultáneo.

9.2.6. Generalización a nuevas plataformas y ejecución local de SLM

Extender la compatibilidad de la extensión hacia otras herramientas de videoconferencia (como Zoom, Microsoft Teams o Webex) mediante capas de abstracción sobre el DOM. Asimismo, se plantea explorar la cuantización de modelos de lenguaje para ejecución local mediante WebGPU en el navegador, con el objetivo de eliminar la dependencia de servicios en la nube para el canal semántico y anular los tiempos de transporte de red.

Bibliografía

[1] República Argentina. *Ley 27.710: Reconocimiento de la Lengua de Señas Argentina (LSA) como lengua natural y originaria*. Boletín Oficial de la República Argentina. Sancionada el 13 de abril de 2023. B.O. Nº 35.150. Abr. de 2023. URL: https://servicios.infoleg.gob.ar/infolegInternet/anexos/380000-384999/382022/norma.htm.

[2] M. I. Massone, M. Simón y C. Gutiérrez. *Mano a mano: Lengua de Señas Argentina. Análisis lingüístico y texto*. Buenos Aires, Argentina: Universidad de Buenos Aires, 2003.

[3] Confederación Argentina de Sordos. *Señario de términos y expresiones en Lengua de Señas Argentina*. Plataforma digital interactiva. Accedido en octubre de 2026. 2024. URL: https://cas.org.ar/tag/senario/.

[4] Proyecto DANE. *LSA en Familia*. Aplicación móvil. Ficha de la aplicación: http://www.proyectodane.org/aplicacion/aplicacionlsaenfamilia/. Accedido en octubre de 2026. 2024. URL: https://proyectodane.org.ar/el-proyecto.

[5] Asociación de Sordos de Mar del Plata. *Asociación de Sordos de Mar del Plata (ASMAR)*. Sitio institucional. Accedido en octubre de 2026. 2024. URL: https://sites.google.com/view/asmardelplata/asociaci%5C%C3%5C%B3n-de-sordos-de-mar-del-plata.

[6] M. I. Massone y R. A. Martínez. *Curso de Lengua de Señas Argentina. Parte II*. Accedido en octubre de 2026. Cultura Sorda, 2012, págs. 12-43. URL: https://www.cultura-sorda.org/wp-content/uploads/2015/03/Massone_Martinez_curso_LSA_PARTE_II_2012.pdf.

[7] M. I. Massone. «El número y el género en la Lengua de Señas Argentina». En: *Signo & Seña* 2 (1993), págs. 73-99. ISSN: 0327-8085.

[8] C. Lugaresi et al. «MediaPipe: A Framework for Building Perception Pipelines». En: *arXiv preprint arXiv:1906.08172* (2019). arXiv: 1906.08172 [cs.CV]. URL: https://arxiv.org/abs/1906.08172.

[9] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang y M. Grundmann. *BlazePose: On-Device Real-Time Body Pose Tracking*. Accedido en octubre de 2026. 2020. arXiv: 2006.10204 [cs.CV]. URL: https://arxiv.org/abs/2006.10204.

[10] F. Zhang et al. *MediaPipe Hands: On-Device Real-Time Hand Tracking*. Accedido en octubre de 2026. 2020. arXiv: 2006.10214 [cs.CV]. URL: https://arxiv.org/abs/2006.10214.

[11] A. Moryossef, M. Müller y R. Fahrni. *pose-format: Library for Viewing, Augmenting, and Handling .pose Files*. Repositorio de GitHub. Accedido en octubre de 2026. 2021. URL: https://github.com/sign-language-processing/pose.

[12] A. Vaswani et al. «Attention Is All You Need». En: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017, págs. 5998-6008. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[13] E. J. Hu et al. «LoRA: Low-Rank Adaptation of Large Language Models». En: *10th International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2106.09685 [cs.CL]. URL: https://arxiv.org/abs/2106.09685.

[14] G. Gerganov. *llama.cpp: Port of Facebook's LLaMA model in C/C++*. Repositorio de GitHub. Accedido en octubre de 2026. 2023. URL: https://github.com/ggerganov/llama.cpp.

[15] ggml-org. *Tensor Encoding Schemes*. Documentación de llama.cpp. Esquemas k-quant. En `Q4_K` cada bloque guarda la escala y el mínimo. Accedido en octubre de 2026. 2023. URL: https://github.com/ggml-org/llama.cpp/wiki/Tensor-Encoding-Schemes.

[16] W3C. *Media Capture and Streams*. Estandardización web (W3C Candidate Recommendation Snapshot). Accedido en octubre de 2026. 2024. URL: https://www.w3.org/TR/mediacapture-streams/.

[17] W3C. *Media Capture from DOM Elements*. Estandardización web (W3C Candidate Recommendation Snapshot). Accedido en octubre de 2026. 2024. URL: https://www.w3.org/TR/mediacapture-fromelement/.

[18] Google Chrome Developers. *Overview of Manifest V3*. Chrome Extension Documentation. Accedido en octubre de 2026. 2024. URL: https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3.

[19] Google Chrome Developers. *chrome.offscreen*. Chrome Extension Documentation. Accedido en octubre de 2026. 2024. URL: https://developer.chrome.com/docs/extensions/reference/api/offscreen.

[20] Google Chrome Developers. *sandbox (Manifest V3)*. Chrome Extension Documentation. Accedido en octubre de 2026. 2024. URL: https://developer.chrome.com/docs/extensions/reference/manifest/sandbox.

[21] V. Barán. *LSApp: Uniendo dos mundos*. Plataforma web y móvil. Accedido en octubre de 2026. 2018. URL: https://lsapp.com.ar/.

[22] Blessit. *Interpretación Lengua de Señas — Central de intérpretes*. Plataforma de servicios de accesibilidad. Accedido en octubre de 2026. 2024. URL: https://www.blessit.net/interpretacion-lengua-de-senas.

[23] Ministerio de Justicia de la Nación. *Características de la comunicación por videollamadas — Línea 144*. Portal oficial del Estado Argentino (Argentina.gob.ar). Accedido en octubre de 2026. 2024. URL: https://www.argentina.gob.ar/caracteristicas-de-la-comunicacion-por-videollamadas.

[24] F. Ronchetti, F. Quiroga, C. Estrebou, L. Lanzarini y A. Rosete. «LSA64: An Argentinian Sign Language Dataset». En: *Proceedings of the XXII Congreso Argentino de Ciencias de la Computación (CACIC)*. San Luis, Argentina, 2016, págs. 794-803. arXiv: 2310.17429 [cs.CL]. URL: https://arxiv.org/abs/2310.17429.

[25] I. Mindlin. «Reconocimiento de Lengua de Señas con redes neuronales recurrentes». Tesis de grado. Universidad Nacional de La Plata, Facultad de Informática, 2021. URL: http://sedici.unlp.edu.ar/handle/10915/129853.

[26] D. Retek, D. Palhazi, M. Kajtar, M. Trosztel, Z. Robotka y J. Rovnyai. «Computer Vision Based Sign Language Interpreter». Pat. estadounidense 11847426. SignAll Technologies Zrt. Patente estadounidense US11847426B2, concedida el 19 de diciembre de 2023. Describe la captura visual de ASL y la salida en inglés escrito. Dic. de 2023. URL: https://patents.google.com/patent/US11847426B2.

[27] D. Li, C. Rodriguez, X. Yu y H. Li. «Word-Level Deep Sign Language Recognition from Video: A New Large-Scale Dataset and Methods Comparison». En: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. Snowmass, CO, USA, 2020, págs. 1459-1469. DOI: 10.1109/WACV45572.2020.9093512.

[28] Hand Talk. *Acessibilidade digital em línguas de sinais*. Sitio web oficial. Accedido en octubre de 2026. 2025. URL: https://www.handtalk.me/br/.

[29] R. S. Pressman y B. R. Maxim. *Software Engineering: A Practitioner's Approach*. 8th. New York: McGraw-Hill, 2014.

[30] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama. «Optuna: A Next-Generation Hyperparameter Optimization Framework». En: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, págs. 2623-2631. DOI: 10.1145/3292500.3330701.

[31] Qwen Team. «Qwen2.5 Technical Report». En: *arXiv preprint arXiv:2412.15115* (2024). arXiv: 2412.15115 [cs.CL]. URL: https://arxiv.org/abs/2412.15115.

[32] L. B. Allal, A. Lozhkov, E. Bakouch, L. von Werra y T. Wolf. «SmolLM2: When Small Goes Big — Data-Centric Training of a Small Language Model». En: *arXiv preprint arXiv:2502.02737* (2025). arXiv: 2502.02737 [cs.CL]. URL: https://arxiv.org/abs/2502.02737.

[33] Meta. *Llama 3.2 1B Instruct*. Hugging Face Hub. Ficha del modelo. Publicación: 25 de septiembre de 2024. Accedido en octubre de 2026. 2024. URL: https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct.

[34] JetBrains. *YouTrack: Issue Tracker and Project Management Tool*. Documentación oficial de JetBrains. Accedido en octubre de 2026. 2024. URL: https://www.jetbrains.com/youtrack/.

[35] K. Papineni, S. Roukos, T. Ward y W.-J. Zhu. «BLEU: a Method for Automatic Evaluation of Machine Translation». En: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. Philadelphia, Pennsylvania, USA, 2002, págs. 311-318. DOI: 10.3115/1073083.1073135. URL: https://aclanthology.org/P02-1040.

[36] C.-Y. Lin. «ROUGE: A Package for Automatic Evaluation of Summaries». En: *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. Barcelona, Spain: Association for Computational Linguistics, jul. de 2004, págs. 74-81. URL: https://aclanthology.org/W04-1013.

[37] S. Banerjee y A. Lavie. «METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments». En: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. Ann Arbor, Michigan: Association for Computational Linguistics, jun. de 2005, págs. 65-72. URL: https://aclanthology.org/W05-0909.

[38] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger e Y. Artzi. «BERTScore: Evaluating Text Generation with BERT». En: *8th International Conference on Learning Representations (ICLR)*. 2020. arXiv: 1904.09675 [cs.CL]. URL: https://arxiv.org/abs/1904.09675.

[39] R. Rei, C. Stewart, A. C. Farinha y A. Lavie. «COMET: A Neural Framework for MT Evaluation». En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, nov. de 2020, págs. 2685-2702. DOI: 10.18653/v1/2020.emnlp-main.213. URL: https://aclanthology.org/2020.emnlp-main.213.

[40] S. Ramírez. *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*. Repositorio de GitHub. Accedido en octubre de 2026. 2018. URL: https://github.com/fastapi/fastapi.

[41] G. Van Rossum, B. Warsaw y N. Coghlan. *PEP 8 – Style Guide for Python Code*. Python Software Foundation. 2001. URL: https://peps.python.org/pep-0008/ (visitado 31-10-2026).

[42] Ecma International. *ECMA-262: ECMAScript 2015 Language Specification*. Geneva, Switzerland, 2015. URL: https://www.ecma-international.org/publications-and-standards/standards/ecma-262/ (visitado 31-10-2026).

[43] W. Kwon et al. «Efficient Memory Management for Large Language Model Serving with PagedAttention». En: *Proceedings of the 29th ACM Symposium on Operating Systems Principles*. SOSP '23. Análisis analítico del footprint de memoria: pesos, KV Cache y activaciones. New York, NY, USA: Association for Computing Machinery, 2023, págs. 611-626. DOI: 10.1145/3600006.3613165.

[44] V. A. Korthikanti, M. Shoeybi, A. Kakarmath, B. Catanzaro y D. Narayanan. «Reducing Activation Recomputation in Large Transformer Models». En: *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 5. 2023, págs. 348-358. arXiv: 2205.05198 [cs.CL]. URL: https://arxiv.org/abs/2205.05198.

[45] R. Pope et al. «Efficiently Scaling Transformer Inference». En: *Proceedings of the 6th Machine Learning and Systems Conference (MLSys)*. Cálculo de la caché KV: dos tensores, capas, cabezas de KV, dimensión de cabeza y bytes por elemento. Miami Beach, FL, USA, 2023. URL: https://arxiv.org/abs/2211.05102.

Apéndice A

Vocabulario del clasificador

El clasificador distingue 100 clases. Cada clase cuenta con 100 clips. Miriam Rolls ayudó con la validación del conjunto. El cuadro A.1 separa el alfabeto, los dígitos y el léxico en grupos chicos, y cuenta cuántas clases hay en cada uno. La celda de señas trae la etiqueta exacta, tal como está en `mapeo_clases.json`. `ñ` y `años` conservan la ñ. `esposo a` conserva el espacio entre las dos palabras. Las demás etiquetas van sin tilde.

Cuadro A.1: Señas del clasificador por categoría.

Categoría	**Cantidad**	**Señas**
Alfabeto manual	27	`A`, `B`, `C`, `D`, `E`, `F`, `G`, `H`, `I`, `J`, `K`, `L`, `M`, `N`, `Ñ`, `O`, `P`, `Q`, `R`, `S`, `T`, `U`, `V`, `W`, `X`, `Y`, `Z`
Dígitos	10	`0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`
Interrogativos	6	`como`, `cuando`, `donde`, `que`, `quien`, `cuantos`
Afirmación y negación	2	`si`, `no`
Valoración	2	`bien`, `mal`
Pronombres	5	`yo`, `vos`, `el_ella`, `nosotros`, `ellos`
Posesivos	2	`mio`, `tuyo`
Saludos y cortesía	3	`hola`, `chau`, `gracias`
Tiempo	6	`pasado`, `futuro`, `dia`, `hora`, `ahora_hoy`, `ayer`
Días de la semana	7	`lunes`, `martes`, `miercoles`, `jueves`, `viernes`, `sabado`, `domingo`
Lugares	4	`lugar`, `casa`, `calle`, `plaza`
Identificación	5	`nombre`, `apellido`, `documento`, `numero`, `años`
Parentesco	6	`familia`, `mama`, `papa`, `hermano_a`, `hijo_a`, `esposo_a`
Partes del cuerpo	3	`brazo`, `cara`, `ojo`
Objeto	1	`cuchillo`
Verbos	8	`tener`, `poder`, `vivir`, `vivir_en`, `ver`, `llamar`, `llevar`, `repetir`
Auxilio	3	`ayudar`, `robar`, `policia`
Total	**100**	

En la prueba de cámara de la Sección 7.2.1, el Top-1 informado acepta dos pares como la misma seña. `O` y `O` cuentan juntas. `V` y `viernes` cuentan juntas. Con luz adecuada son 18 tomas, ocho de `O` leídas como `O` y diez de `V` leídas como `viernes`, y el acierto pasa de 978/1000 a 996/1000. Con luz reducida son 24 tomas y el acierto pasa de 922/1000 a

946/1000. La validación fuera de línea exige la misma etiqueta.

Apéndice B

Ramas de entrenamiento y de servicios

Este apéndice reúne las otras ramas. En cada una figuran el commit, los comandos de instalación y de evaluación, y el SHA-256 de los pesos usados. El detalle de directorios queda en el `README` de cada una.

B.1. *scratch-mediapipe-v3*: Repositorio de entrenamiento del clasificador

Este repositorio entrena y evalúa el clasificador de señas aisladas. Vive en la rama `scratch-mediapipe-v3`. El archivo de pesos que sale de ese entrenamiento es el que después carga el motor local.

Todo el código está en `src/`. `train.py` ajusta el modelo y guarda el peso, `tune_optuna.py` busca hiperparámetros, y `preprocessing.py` convierte los videos MP4 en coordenadas. `camera.py` mide el acierto frente a la cámara. La arquitectura está en `model_arch.py`. `config.py` fija las rutas, las 100 clases y la ventana de 16 fotogramas. La extracción usa el Holistic del navegador, el mismo paquete que la extensión, a través de `holistic_web.py`. Los videos quedan en `dataset/` y las coordenadas en `dataset_landmarks_16frames/`. Los pesos y los registros salen a `src/model_new_2026_10_20/`, con el checkpoint `tinyskeleton_best.pth`.

El commit de la rama es https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/af16107261a2627de46fb125933bb02768c78eb0. La instalación parte de la raíz. El entrenamiento y la evaluación parten de `src/`.

```
conda env create -f environment.yml
conda activate lsa_gpu
cd src/holistic_web
npm install
npx playwright install chromium
cd ..
python check_env.py
python train.py
python camera.py --eval
```

El checkpoint de esa evaluación es `tinyskeleton_best.pth`. Está en el commit. Su SHA-256 es

6ac921623778b17c186c13f0165eee0580950e4c336ed625e8a30fdf090d8a5b.
El motor local carga ese mismo archivo.

B.2. *semantic-cloud*: Servicio semántico en la nube

Este repositorio es el servicio que el motor local consulta para traducir. Atiende las dos direcciones, de glosas hacia español y de español hacia glosas, con los binarios que entregan los repositorios de entrenamiento. Su tarea es responder un enunciado ya armado, y no forma parte de lo que se instala en la computadora de quien llama.

El arranque está en `run_semantic_server.py`. El servicio vive en `src/semantic/`: `api.py` expone las rutas, `service.py` separa los dos canales y `engine.py` ejecuta el binario. Los pesos no comparten carpeta. `models/sign/` guarda el canal de glosas hacia español y `models/hearing/` el canal inverso. En cada una hay un binario principal, un respaldo y el `sys_prompt.txt` de esa dirección.

El commit de la rama es https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/d5bf916c55aab805338f6621b7de592b988dc0ce. La instalación y la prueba local parten de la raíz. Git no versiona los `.gguf`. Los archivos finales de los dos canales están en https://huggingface.co/maite123/lsa-semantic-gguf. El canal directo ocupa `sordo/` y el inverso, `oyente/`. El SHA-256 de Hugging Face coincide con el de cada archivo de abajo.

```
pip install -r requirements.txt
python run_semantic_server.py
curl -s http://127.0.0.1:8787/health
curl -s -X POST http://127.0.0.1:8787/sign -H "Content-Type:
   application/json" -d "{\"glosses\":\"HOLA\"}"
curl -s -X POST http://127.0.0.1:8787/hearing -H "Content-Type:
   application/json" -d "{\"spanish\":\"Hola\"}"
```

Cada canal carga el Qwen2.5 1,5B como principal y el Llama 3.2 1B como respaldo. El archivo del servicio es una copia del binario evaluado en la rama de entrenamiento de ese canal.

- Canal directo, `models/sign/qwen2.5-1.5b.gguf`.
  SHA-256 `fd90f6103dea2512d2f84bb57fb2781869711411279571d3b12d7c84c8e830dd`.
- Canal directo, `models/sign/llama-3.2-1b.gguf`.
  SHA-256 `2fa6bb8347c0b55d3cafa62ac501e262ba4a170562d8f936ee054a050c8d1068`.
- Canal inverso, `models/hearing/qwen2.5-1.5b.gguf`.
  SHA-256 `aca23988584e4d3dafce6682ea6de009c0b1b5bdf940816c99441a56434b3a55`.
- Canal inverso, `models/hearing/llama-3.2-1b.gguf`.
  SHA-256 `4a35a81e48130866b778a947427ec70f92b87175472b0fa266551e62cea2408e`.

B.3. *semantic-v2*: Entrenamiento del canal directo

Este repositorio ajusta y evalúa los modelos que pasan de glosas a una oración en español. El binario que resulta de ese entrenamiento es el que después carga el servicio en la nube para el canal directo.

Todo el código está en `src/semantic/`. `train.py` ajusta un modelo y exporta el binario, `eval_trial.py` lo compara sobre oraciones fuera del corte, y `models.py` es el catálogo: sumar una familia es agregar una entrada ahí. El *corpus* está en `dataset_glosses.json`, las oraciones de prueba en `dataset_trial.json` y las instrucciones del modelo en `prompts/sys_prompt.txt`. Los pesos y los registros salen a `outputs/`. Cada familia se identifica con un id estable, como `qwen2.5-1.5b` o `llama-3.2-1b`, y ese mismo id nombra su carpeta de salida.

El commit de la rama es https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/9c55106abaf96bf802459ebee6b3212c3957bb3e. Los comandos parten de `src/semantic`. Git no versiona los binarios: quedan en `outputs/`. Los cinco archivos finales están en https://huggingface.co/maite123/lsa-semantic-gguf/tree/main/sordo. Allí el Qwen2.5 1,5B se llama `qwen2.5-1.5b.gguf`, y el resto sigue ese patrón. El SHA-256 publicado es el de la lista.

```
conda create -n lsa_semantic python=3.10 -y
conda activate lsa_semantic
pip install torch torchvision --index-url https://download.
   pytorch.org/whl/cu128
pip install -r requirements.txt
pip install unsloth unsloth_zoo --no-deps
pip install unbabel-comet==2.2.7 --no-deps
python train_all.py --continue-on-error
```

El SHA-256 de cada binario `Q4_K_M` evaluado es el siguiente.

- Qwen2.5 0,5B, `qwen2.5-0.5b-instruct.Q4_K_M.gguf`.
  SHA-256 `1fc907c15f398ea5e709b9abb76430dedb7cfe231ed3d0e1fd4de9041b03a8de`.
- Qwen2.5 1,5B, `qwen2.5-1.5b-instruct.Q4_K_M.gguf`.
  SHA-256 `fd90f6103dea2512d2f84bb57fb2781869711411279571d3b12d7c84c8e830dd`.
- Qwen2.5 3B, `qwen2.5-3b-instruct.Q4_K_M.gguf`.
  SHA-256 `927022b3717266b1b20441ac0fff02b20e56b361c3fb1d4261d97473c7067aa6`.
- Llama 3.2 1B, `llama-3.2-1b-instruct.Q4_K_M.gguf`.
  SHA-256 `2fa6bb8347c0b55d3cafa62ac501e262ba4a170562d8f936ee054a050c8d1068`.
- SmolLM2 1,7B, `SmolLM2-1.7B-Instruct.Q4_K_M.gguf`.
  SHA-256 `22e4fe904a7facd19595002fb187c01c52f5417ff52998b4ac282dae087449b4`.

B.4. *semantic-reverse*: Entrenamiento del canal inverso

Este repositorio ajusta y evalúa los modelos que pasan de una oración en español a una secuencia de glosas. El *corpus* y el binario son propios de esta dirección, y el servicio en la nube los usa para el canal inverso.

La disposición repite la del canal directo, también bajo `src/semantic/`: `train.py` entrena, `eval_trial.py` compara y `config.py` reúne rutas, hiperparámetros y la lista de modelos. El *corpus*, el archivo de prueba y el prompt ocupan los mismos nombres de archivo que en el otro laboratorio. Los id de modelo también coinciden, de modo que `qwen2.5-1.5b` designa la misma familia en las dos direcciones, con pesos distintos.

El commit de la rama es https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/e5285fa2c08cde46bf41d6ffb5b5fe0541d6bb72. Los comandos parten de `src/semantic`. Git tampoco versiona estos binarios. Los cinco archivos finales están en https://huggingface.co/maite123/lsa-semantic-gguf/tree/main/oyente. Allí el Qwen2.5 1,5B se llama `qwen2.5-1.5b.gguf`, y el resto sigue ese patrón. El SHA-256 publicado es el de la lista.

```
conda create -n lsa-train python=3.11 -y
conda activate lsa-train
pip install torch torchvision --index-url https://download.
   pytorch.org/whl/cu126
pip install -r requirements_train.txt
pip install unsloth unsloth_zoo --no-deps
pip install unbabel-comet==2.2.7 --no-deps
python train.py --continue-on-error
python eval_trial.py
```

El SHA-256 de cada binario `Q4_K_M` evaluado es el siguiente.

- Qwen2.5 0,5B, `qwen2.5-0.5b-instruct.Q4_K_M.gguf`.
  SHA-256 `46786ccb127860669824d55e6328ac5d25f72a0023edb7d47de4c216955049ad`.
- Qwen2.5 1,5B, `qwen2.5-1.5b-instruct.Q4_K_M.gguf`.
  SHA-256 `aca23988584e4d3dafce6682ea6de009c0b1b5bdf940816c99441a56434b3a55`.
- Qwen2.5 3B, `qwen2.5-3b-instruct.Q4_K_M.gguf`.
  SHA-256 `3f8e868c9ded360e2ec3849b1a7f33435affafee975c3af499c847bed122a186`.
- Llama 3.2 1B, `llama-3.2-1b-instruct.Q4_K_M.gguf`.
  SHA-256 `4a35a81e48130866b778a947427ec70f92b87175472b0fa266551e62cea2408e`.
- SmolLM2 1,7B, `SmolLM2-1.7B-Instruct.Q4_K_M.gguf`.
  SHA-256 `51f3e6ac945e5386806c06806c729f30ddf98da326c0254872d39bfe3e3b6a1f`.

B.5. *avatar-cloud*: Servicio de poses del avatar

Este repositorio publica los archivos de pose que el motor local pide para dibujar el avatar. No entrena modelos ni traduce texto. El cliente de la videollamada solicita esas poses a este servicio.

El código cabe en `app/`: `main.py` arranca el servicio, `config.py` fija rutas y tipos de archivo, y `fingerspell.py` resuelve el alfabeto manual. Las poses están en `poses/`. El servidor devuelve el archivo cuyo nombre coincide con el pedido. Ese nombre, sin extensión, va en mayúsculas y en ASCII: sin tildes y sin ñ.

Al armar el pedido, ILSA transforma la "ñ" en "n": `MAÑANA` busca `MANANA.pose`. La letra Ñ, cuando es la glosa entera, se pide como `ENIE`. Antes del subtítulo, ILSA restituye la Ñ.

Una etiqueta compuesta tampoco es el nombre del archivo. El clasificador reescribe `hermano_a` como `HERMANO`, `esposo_a` como `ESPOSO` y `hijo_a` como `HIJO`: el sufijo `_a` marca el género y no entra al archivo. `vivir_en` se reescribe como `VIVIR_EN`. Al pedir la pose, ILSA borra los guiones bajos, así `VIVIR_EN` busca `VIVIREN.pose`.

El commit de la rama es https://github.com/FranciscoVeronING/2026_Proyecto_LSA/commit/a713860b071bceacb2e7b1ed4145ecd1a73f0b55. Esta rama no entrena un modelo y no tiene archivo de pesos. Los comandos parten de la raíz. La comprobación es `GET /health` y un `GET /tposes/{glosa}.pose` en el puerto 8080.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m app.main
```

Apéndice C

Arquitectura escalable de producción en la nube

La Figura C.1 pertenece a la Sección 5.6. La hoja va en horizontal para que el ingreso, el traductor y el servicio de archivos queden en una sola vista. El diagrama describe la proyección para varias videollamadas a la vez.

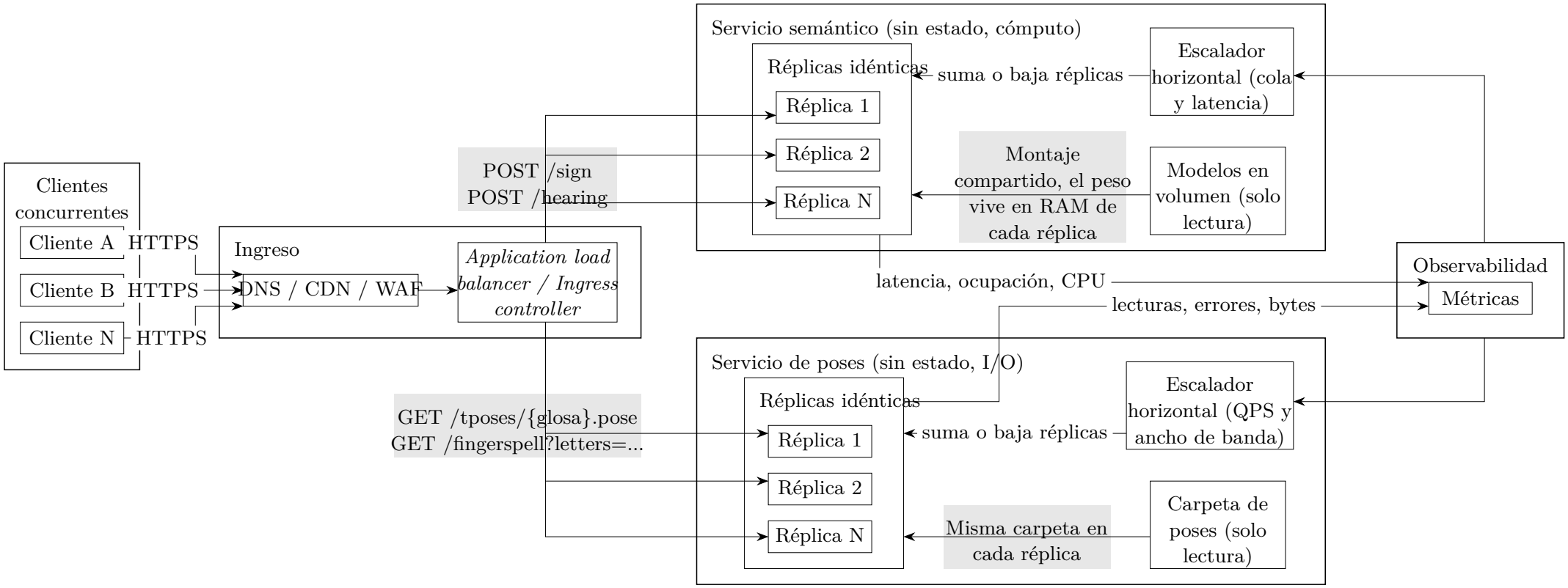


Figura C.1: Diagrama de la arquitectura del servidor, balanceo de carga y escalabilidad de servicios sin estado.